

Contents

Part I: Foundations for Analytics	2
1. SQL Refresher	2
2. Joins and Set Operations	4
3. Aggregate Functions	8
4. Window Functions	11
Part II: Advanced SQL for Analytics	14
1. Advanced Filtering and Logic	14
2. Advanced Join Techniques	19
4. User-Defined Functions for Analytics	31
Part III: Data Modeling for Analytics	37
1. Dimensional Modeling Concepts	37
2. Star and Snowflake Schemas	43
3. Time and Date Dimensions	48
Part IV: Performance Optimization	57
1. Indexes and Query Plans	57
2. Query Optimization Techniques	65
3. Identifying and Resolving Bottlenecks	72
Part V: Distributed SQL and Large-Scale Analytics	80
1. Columnar Storage Principles	80
2. Partitioning and Distribution Strategies	82
3. Large-Scale Aggregation Techniques	85
4. Incremental Processing	88
5. Modern Data Formats and Integration	95
6. Practical Optimization Techniques for Distributed Analytics	99
Part II: Advanced SQL for Analytics	103
1. Advanced Filtering and Logic	103
2. Advanced Join Techniques	107
4. User-Defined Functions for Analytics	120
Part III: Data Modeling for Analytics	126
1. Dimensional Modeling Concepts	126
2. Star and Snowflake Schemas	132
3. Time and Date Dimensions	136
SQL for Analytics: A Comprehensive Guide	145
Part IV: Performance Optimization	145
1. Indexes and Query Plans	145
2. Query Optimization Techniques	154
3. Identifying and Resolving Bottlenecks	161
Part V: Distributed SQL and Large-Scale Analytics	169
1. Columnar Storage Principles	169
2. Partitioning and Distribution Strategies	171
3. Large-Scale Aggregation Techniques	174
4. Incremental Processing	177
5. Modern Data Formats and Integration	184

“# SQL for Analytics: A Comprehensive Guide

Part I: Foundations for Analytics

1. SQL Refresher

SQL (Structured Query Language) is the foundation for data analysis in relational databases. In analytics contexts, we primarily use SQL to extract, transform, and aggregate data from large datasets.

Basic Query Structure

```
-- Basic SELECT statement with filtering, grouping, and ordering
SELECT
    column1,
    column2,
    COUNT(*) as record_count,
    SUM(numeric_column) as total_value
FROM
    table_name
WHERE
    condition = 'value'
    AND numeric_column > 0
GROUP BY
    column1, column2
HAVING
    COUNT(*) > 10
ORDER BY
    total_value DESC
LIMIT 100;
```

Essential Clauses for Analytics Analytics work requires heavy use of filtering, grouping, and sorting:

```
-- SELECT: Specify columns to retrieve
SELECT
    customer_segment,
    product_category,
    SUM(sales_amount) as total_sales

-- FROM: Specify data source
FROM
    sales_transactions

-- WHERE: Filter rows before grouping
```

```

WHERE
    transaction_date >= '2023-01-01'
    AND transaction_date < '2024-01-01'
    AND sales_amount > 0

-- GROUP BY: Aggregate data
GROUP BY
    customer_segment,
    product_category

-- HAVING: Filter groups after aggregation
HAVING
    SUM(sales_amount) > 10000

-- ORDER BY: Sort results
ORDER BY
    total_sales DESC

-- LIMIT: Restrict number of rows returned
LIMIT 20;

```

Analytical Expressions Analytics frequently requires calculated fields:

```

-- Math operations
SELECT
    product_id,
    sales_amount,
    cost_amount,
    sales_amount - cost_amount AS profit_amount,
    (sales_amount - cost_amount) / NULLIF(cost_amount, 0) * 100 AS profit_margin_pct
FROM
    sales_transactions;

-- Date expressions
SELECT
    transaction_date,
    EXTRACT(YEAR FROM transaction_date) AS year,
    EXTRACT(QUARTER FROM transaction_date) AS quarter,
    EXTRACT(MONTH FROM transaction_date) AS month,
    EXTRACT(DAY FROM transaction_date) AS day,
    EXTRACT(DOW FROM transaction_date) AS day_of_week
FROM
    sales_transactions;

-- Conditional logic for categorization (CASE statements)
SELECT

```

```

customer_id,
annual_income,
CASE
    WHEN annual_income < 50000 THEN 'Low'
    WHEN annual_income BETWEEN 50000 AND 100000 THEN 'Medium'
    WHEN annual_income > 100000 THEN 'High'
    ELSE 'Unknown'
END AS income_segment
FROM
customers;

```

2. Joins and Set Operations

In analytics, you'll frequently need to combine data from multiple tables to get a complete picture.

Types of Joins

```

-- Sample table structure for examples
/*
Table: sales_transactions
- transaction_id (PK)
- customer_id (FK to customers)
- product_id (FK to products)
- transaction_date
- sales_amount

Table: customers
- customer_id (PK)
- customer_name
- customer_segment

Table: products
- product_id (PK)
- product_name
- product_category
*/

-- INNER JOIN: Returns rows when there is a match in both tables
SELECT
    t.transaction_id,
    t.transaction_date,
    c.customer_name,
    p.product_name,
    t.sales_amount
FROM

```

```

        sales_transactions t
INNER JOIN
    customers c ON t.customer_id = c.customer_id
INNER JOIN
    products p ON t.product_id = p.product_id;

-- LEFT JOIN: Returns all rows from the left table and matches from the right
-- Useful for finding all sales transactions including those with missing customer data
SELECT
    t.transaction_id,
    t.customer_id,
    c.customer_name
FROM
    sales_transactions t
LEFT JOIN
    customers c ON t.customer_id = c.customer_id;

-- RIGHT JOIN: Returns all rows from the right table and matches from the left
-- Useful for finding all customers including those with no transactions
SELECT
    c.customer_id,
    c.customer_name,
    COUNT(t.transaction_id) as transaction_count
FROM
    sales_transactions t
RIGHT JOIN
    customers c ON t.customer_id = c.customer_id
GROUP BY
    c.customer_id, c.customer_name;

-- FULL OUTER JOIN: Returns rows when there is a match in either table
-- Useful for comprehensive data discovery and finding data problems
SELECT
    t.customer_id as transaction_customer_id,
    c.customer_id as customer_table_id,
    c.customer_name,
    COUNT(t.transaction_id) as transaction_count
FROM
    sales_transactions t
FULL OUTER JOIN
    customers c ON t.customer_id = c.customer_id
GROUP BY
    t.customer_id, c.customer_id, c.customer_name;

```

Analytics-Focused Join Patterns

```

-- Time-based join for trend analysis
SELECT
    d.date,
    COALESCE(SUM(t.sales_amount), 0) as daily_sales
FROM
    date_dimension d
LEFT JOIN
    sales_transactions t ON d.date = t.transaction_date
WHERE
    d.date BETWEEN '2023-01-01' AND '2023-12-31'
GROUP BY
    d.date
ORDER BY
    d.date;

-- Multiple fact table join
-- Joining sales and marketing spend for ROI analysis
SELECT
    p.product_category,
    SUM(s.sales_amount) as total_sales,
    SUM(m.marketing_spend) as total_marketing_spend,
    SUM(s.sales_amount) / NULLIF(SUM(m.marketing_spend), 0) as roi
FROM
    products p
LEFT JOIN
    sales_transactions s ON p.product_id = s.product_id
LEFT JOIN
    marketing_spend m ON p.product_id = m.product_id
WHERE
    s.transaction_date BETWEEN '2023-01-01' AND '2023-12-31'
    AND m.spend_date BETWEEN '2023-01-01' AND '2023-12-31'
GROUP BY
    p.product_category;

```

Set Operations Set operations combine results from multiple queries:

```

-- UNION: Combines results and removes duplicates
-- Example: Unifying customer lists from different sources
SELECT customer_id, customer_name, 'Active' as status
FROM active_customers
UNION
SELECT customer_id, customer_name, 'Inactive' as status
FROM inactive_customers
ORDER BY customer_id;

-- INTERSECT: Returns only rows that appear in both result sets

```

```

-- Example: Finding products that exist in both inventory and sales
SELECT product_id
FROM inventory
INTERSECT
SELECT product_id
FROM sales_transactions;

-- EXCEPT: Returns rows from the first query that don't appear in the second
-- Example: Finding customers who haven't made a purchase
SELECT customer_id
FROM all_customers
EXCEPT
SELECT customer_id
FROM sales_transactions;

```

Join Optimization for Analytics Analytics often involves joining large tables, making optimization crucial:

```

-- 1. Use column statistics to inform join order
-- Start with the most filtered table when appropriate
EXPLAIN
SELECT
    p.product_name,
    c.customer_name,
    SUM(t.sales_amount) as total_sales
FROM
    sales_transactions t
JOIN
    products p ON t.product_id = p.product_id
JOIN
    customers c ON t.customer_id = c.customer_id
WHERE
    t.transaction_date >= '2023-01-01'
    AND p.product_category = 'Electronics'
GROUP BY
    p.product_name, c.customer_name;

-- 2. Pre-filter large tables before joining
-- Efficient: Filter first, then join
EXPLAIN
WITH filtered_transactions AS (
    SELECT *
    FROM sales_transactions
    WHERE transaction_date >= '2023-01-01'
)
SELECT

```

```

        p.product_name,
        SUM(t.sales_amount) as total_sales
FROM
    filtered_transactions t
JOIN
    products p ON t.product_id = p.product_id
GROUP BY
    p.product_name;

-- 3. Use EXISTS for existence checks instead of joins
-- Finding customers who made a purchase in a specific category
EXPLAIN
SELECT
    c.customer_id,
    c.customer_name
FROM
    customers c
WHERE
    EXISTS (
        SELECT 1
        FROM sales_transactions t
        JOIN products p ON t.product_id = p.product_id
        WHERE t.customer_id = c.customer_id
        AND p.product_category = 'Electronics'
    );

```

3. Aggregate Functions

Aggregation is at the heart of analytics, transforming detailed data into meaningful summaries.

Basic Aggregation Functions

```

-- COUNT: Number of rows or non-NULL values
SELECT COUNT(*) as total_transactions
FROM sales_transactions;

SELECT COUNT(DISTINCT customer_id) as unique_customers
FROM sales_transactions;

-- SUM: Calculate the sum of values
SELECT SUM(sales_amount) as total_sales
FROM sales_transactions;

-- AVG: Calculate the average of values
SELECT AVG(sales_amount) as average_sale

```



```

FROM sales_transactions;

-- MIN and MAX: Find the extremes
SELECT
    MIN(sales_amount) as smallest_sale,
    MAX(sales_amount) as largest_sale
FROM sales_transactions;

-- Statistical functions
SELECT
    STDDEV(sales_amount) as sales_std_dev,
    VARIANCE(sales_amount) as sales_variance,
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY sales_amount) as median_sale
FROM sales_transactions;

```

Grouped Aggregations

```

-- Basic grouping
SELECT
    product_category,
    COUNT(*) as transaction_count,
    SUM(sales_amount) as total_sales,
    AVG(sales_amount) as average_sale
FROM
    sales_transactions s
JOIN
    products p ON s.product_id = p.product_id
GROUP BY
    product_category
ORDER BY
    total_sales DESC;

-- Multi-level grouping
SELECT
    EXTRACT(YEAR FROM transaction_date) as year,
    EXTRACT(MONTH FROM transaction_date) as month,
    product_category,
    SUM(sales_amount) as total_sales
FROM
    sales_transactions s
JOIN
    products p ON s.product_id = p.product_id
GROUP BY
    EXTRACT(YEAR FROM transaction_date),
    EXTRACT(MONTH FROM transaction_date),
    product_category

```

```

ORDER BY
    year, month, total_sales DESC;

-- Filtering groups with HAVING
SELECT
    customer_id,
    COUNT(*) as transaction_count,
    SUM(sales_amount) as total_spend
FROM
    sales_transactions
GROUP BY
    customer_id
HAVING
    COUNT(*) > 10 AND SUM(sales_amount) > 1000
ORDER BY
    total_spend DESC;

```

Conditional Aggregation

```

-- CASE inside aggregation
SELECT
    product_category,
    SUM(CASE WHEN sales_amount >= 1000 THEN 1 ELSE 0 END) as high_value_sales,
    SUM(CASE WHEN sales_amount < 1000 THEN 1 ELSE 0 END) as regular_sales,
    SUM(CASE WHEN sales_amount >= 1000 THEN sales_amount ELSE 0 END) as high_value_revenue,
    SUM(CASE WHEN sales_amount < 1000 THEN sales_amount ELSE 0 END) as regular_revenue
FROM
    sales_transactions s
JOIN
    products p ON s.product_id = p.product_id
GROUP BY
    product_category;

-- Creating cohorts with conditional aggregation
SELECT
    customer_segment,
    SUM(CASE WHEN EXTRACT(YEAR FROM transaction_date) = 2022 THEN sales_amount ELSE 0 END) as 2022_sales,
    SUM(CASE WHEN EXTRACT(YEAR FROM transaction_date) = 2023 THEN sales_amount ELSE 0 END) as 2023_sales,
    SUM(CASE WHEN EXTRACT(YEAR FROM transaction_date) = 2022 THEN sales_amount ELSE 0 END) as 2022_revenue,
    SUM(CASE WHEN EXTRACT(YEAR FROM transaction_date) = 2023 THEN sales_amount ELSE 0 END) as 2023_revenue
FROM
    sales_transactions s
JOIN
    customers c ON s.customer_id = c.customer_id
GROUP BY
    customer_segment;

```

Filter vs. Having Understanding the difference between WHERE and HAVING is crucial for efficient analytics queries:

```
-- WHERE filters rows before aggregation (more efficient)
SELECT
    product_category,
    SUM(sales_amount) as total_sales
FROM
    sales_transactions s
JOIN
    products p ON s.product_id = p.product_id
WHERE
    sales_amount > 100 -- Filter before aggregation
GROUP BY
    product_category;

-- HAVING filters groups after aggregation (necessary for aggregate conditions)
SELECT
    product_category,
    SUM(sales_amount) as total_sales
FROM
    sales_transactions s
JOIN
    products p ON s.product_id = p.product_id
GROUP BY
    product_category
HAVING
    SUM(sales_amount) > 10000; -- Filter after aggregation
```

4. Window Functions

Window functions are incredibly powerful for analytics, allowing calculations across rows related to the current row without grouping rows into a single output row.

Basic Window Functions

```
-- ROW_NUMBER: Assign a unique sequential integer to rows
SELECT
    customer_id,
    transaction_date,
    sales_amount,
    ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY transaction_date) as purchase_number
FROM
    sales_transactions;

-- RANK and DENSE_RANK: Assign rankings with and without gaps
```

```

SELECT
    product_id,
    sales_amount,
    RANK() OVER (ORDER BY sales_amount DESC) as sales_rank,
    DENSE_RANK() OVER (ORDER BY sales_amount DESC) as sales_dense_rank
FROM
    sales_transactions;

-- NTILE: Divide rows into a specified number of groups
SELECT
    customer_id,
    sales_amount,
    NTILE(4) OVER (ORDER BY sales_amount) as sales_quartile
FROM
    sales_transactions;

-- Common use: find top performers
SELECT *
FROM (
    SELECT
        product_id,
        product_name,
        total_sales,
        DENSE_RANK() OVER (ORDER BY total_sales DESC) as sales_rank
    FROM (
        SELECT
            p.product_id,
            p.product_name,
            SUM(s.sales_amount) as total_sales
        FROM
            products p
        JOIN
            sales_transactions s ON p.product_id = s.product_id
        GROUP BY
            p.product_id, p.product_name
    ) product_sales
    ) ranked_products
WHERE sales_rank <= 5; -- Top 5 products

```

Analytic Window Functions

```

-- Moving average (trailing 7 days)
SELECT
    transaction_date,
    sales_amount,
    AVG(sales_amount) OVER (

```

```

        ORDER BY transaction_date
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    ) as moving_avg_7day
FROM
    daily_sales;

-- Cumulative sum
SELECT
    transaction_date,
    sales_amount,
    SUM(sales_amount) OVER (
        ORDER BY transaction_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) as cumulative_sales
FROM
    daily_sales;

-- Percent of total calculation
SELECT
    product_category,
    sales_amount,
    sales_amount / SUM(sales_amount) OVER () * 100 as percent_of_total,
    sales_amount / SUM(sales_amount) OVER (PARTITION BY EXTRACT(YEAR FROM transaction_date))
FROM
    sales_by_category;

```

Analytical Comparisons

```

-- Period over period comparison
SELECT
    current_period.year,
    current_period.month,
    current_period.sales as current_sales,
    previous_period.sales as previous_sales,
    (current_period.sales - previous_period.sales) / previous_period.sales * 100 as growth_p
FROM
    monthly_sales current_period
LEFT JOIN
    monthly_sales previous_period
    ON current_period.year = previous_period.year + 1
    AND current_period.month = previous_period.month;

-- Alternative using LAG function
SELECT
    year,
    month,

```

```

    sales as current_sales,
    LAG(sales) OVER (ORDER BY year, month) as previous_month_sales,
    LAG(sales, 12) OVER (ORDER BY year, month) as previous_year_sales,
    (sales - LAG(sales, 12) OVER (ORDER BY year, month)) /
    NULLIF(LAG(sales, 12) OVER (ORDER BY year, month), 0) * 100 as yoy_growth_percent
FROM
    monthly_sales;

```

Performance Analysis

-- Analyzing customer purchase frequency

```

SELECT
    customer_id,
    transaction_date,
    LAG(transaction_date) OVER (PARTITION BY customer_id ORDER BY transaction_date) as prev_transaction_date,
    transaction_date - LAG(transaction_date) OVER (PARTITION BY customer_id ORDER BY transaction_date) as days_between_transactions
FROM
    sales_transactions;

```

-- Finding top customers by category

```

SELECT *
FROM (
    SELECT
        c.customer_id,
        c.customer_name,
        p.product_category,
        SUM(s.sales_amount) as category_spend,
        DENSE_RANK() OVER (PARTITION BY p.product_category ORDER BY SUM(s.sales_amount) DESC) as category_rank
    FROM
        customers c
    JOIN
        sales_transactions s ON c.customer_id = s.customer_id
    JOIN
        products p ON s.product_id = p.product_id
    GROUP BY
        c.customer_id, c.customer_name, p.product_category
) ranked_customers
WHERE
    category_rank <= 3; -- Top 3 customers per category

```

Part II: Advanced SQL for Analytics

1. Advanced Filtering and Logic

Advanced filtering is essential for isolating exactly the data subsets needed for analysis.

Complex Filtering with CASE, COALESCE, and NULLIF

-- CASE for complex categorization

```
SELECT
    transaction_id,
    sales_amount,
    CASE
        WHEN sales_amount < 100 THEN 'Small'
        WHEN sales_amount BETWEEN 100 AND 999 THEN 'Medium'
        WHEN sales_amount BETWEEN 1000 AND 9999 THEN 'Large'
        WHEN sales_amount >= 10000 THEN 'Enterprise'
        ELSE 'Unknown'
    END as sale_size_category
FROM
    sales_transactions;
```

-- COALESCE: Returns the first non-NULL expression

-- Useful for handling missing values

```
SELECT
    customer_id,
    COALESCE(email, phone, 'No contact info') as contact_info,
    COALESCE(preferred_name, first_name || ' ' || last_name) as display_name,
    COALESCE(annual_spend, 0) as customer_spend
FROM
    customers;
```

-- NULLIF: Returns NULL if the two expressions are equal

-- Useful for avoiding division by zero and detecting unchanged values

```
SELECT
    product_id,
    current_month_sales,
    previous_month_sales,
    current_month_sales / NULLIF(previous_month_sales, 0) as sales_ratio,
    NULLIF(current_price, original_price) as price_changed
FROM
    product_performance;
```

Advanced Subquery Techniques

-- Scalar subqueries (return a single value)

```
SELECT
    product_id,
    product_name,
    sales_amount,
    sales_amount / (SELECT AVG(sales_amount) FROM sales_transactions) as ratio_to_avg
FROM
```

```

        sales_transactions
JOIN
    products USING (product_id);

-- Correlated subqueries (reference the outer query)
SELECT
    customer_id,
    customer_name,
    annual_spend,
    (SELECT AVG(annual_spend)
     FROM customers c2
     WHERE c2.customer_segment = c1.customer_segment) as segment_avg_spend
FROM
    customers c1;

-- Subqueries in the FROM clause
SELECT
    customer_segment,
    AVG(customer_ltv) as avg_segment_ltv
FROM (
    SELECT
        c.customer_id,
        c.customer_name,
        c.customer_segment,
        SUM(s.sales_amount) as customer_ltv
    FROM
        customers c
    JOIN
        sales_transactions s ON c.customer_id = s.customer_id
    GROUP BY
        c.customer_id, c.customer_name, c.customer_segment
) customer_lifetime_value
GROUP BY
    customer_segment;

-- Subqueries with aggregation
SELECT
    product_id,
    product_name
FROM
    products
WHERE
    product_id IN (
        SELECT product_id
        FROM sales_transactions
        GROUP BY product_id
    )

```



```

        HAVING COUNT(*) > 100
    );

```

Exists, Not Exists, and Anti-joins

```

-- EXISTS: Check for the existence of related rows
-- Find customers who purchased a specific product category
SELECT
    c.customer_id,
    c.customer_name
FROM
    customers c
WHERE
    EXISTS (
        SELECT 1
        FROM sales_transactions s
        JOIN products p ON s.product_id = p.product_id
        WHERE s.customer_id = c.customer_id
        AND p.product_category = 'Electronics'
    );

-- NOT EXISTS: Check for the absence of related rows
-- Find products with no sales in the last 90 days
SELECT
    p.product_id,
    p.product_name
FROM
    products p
WHERE
    NOT EXISTS (
        SELECT 1
        FROM sales_transactions s
        WHERE s.product_id = p.product_id
        AND s.transaction_date >= CURRENT_DATE - INTERVAL '90 days'
    );

-- Anti-join: Finding rows in one table with no match in another
-- Alternative to NOT EXISTS
SELECT
    p.product_id,
    p.product_name
FROM
    products p
LEFT JOIN
    sales_transactions s ON p.product_id = s.product_id
    AND s.transaction_date >= CURRENT_DATE - INTERVAL '90 days'

```

```
WHERE
    s.transaction_id IS NULL;
```

Analytic Boolean Logic

-- Complex filtering logic for segmentation

```
SELECT
    customer_id,
    customer_name,
    CASE
        WHEN
            (annual_spend > 10000 OR lifetime_purchases > 50)
            AND (last_purchase_date >= CURRENT_DATE - INTERVAL '90 days')
            AND customer_status = 'Active'
        THEN 'High Value'
        WHEN
            annual_spend BETWEEN 5000 AND 10000
            AND last_purchase_date >= CURRENT_DATE - INTERVAL '180 days'
        THEN 'Medium Value'
        WHEN customer_status = 'Active' THEN 'Regular'
        ELSE 'At Risk'
    END as customer_value_segment
FROM
    customers;
```

-- Multiple conditions with grouping

```
SELECT
    product_category,
    COUNT(*) as product_count,
    SUM(CASE
        WHEN price > 100 AND stock_quantity > 0 THEN 1
        ELSE 0
    END) as premium_in_stock,
    SUM(CASE
        WHEN price <= 100 AND stock_quantity > 0 THEN 1
        ELSE 0
    END) as budget_in_stock,
    SUM(CASE
        WHEN stock_quantity = 0 THEN 1
        ELSE 0
    END) as out_of_stock
FROM
    products
GROUP BY
    product_category;
```

2. Advanced Join Techniques

Analytics often requires sophisticated approaches to joining data from multiple sources.

Self Joins for Hierarchical and Sequential Analysis

```
-- Self join for employee hierarchy analysis
SELECT
    e.employee_id,
    e.employee_name,
    e.title,
    m.employee_name as manager_name,
    m.title as manager_title
FROM
    employees e
LEFT JOIN
    employees m ON e.manager_id = m.employee_id;

-- Sequential events analysis
-- Finding time between a customer's purchases
SELECT
    current.customer_id,
    current.transaction_id as current_transaction,
    current.transaction_date as current_date,
    previous.transaction_id as previous_transaction,
    previous.transaction_date as previous_date,
    current.transaction_date - previous.transaction_date as days_between_purchases
FROM
    sales_transactions current
LEFT JOIN
    sales_transactions previous
    ON current.customer_id = previous.customer_id
    AND previous.transaction_date < current.transaction_date
    AND NOT EXISTS (
        SELECT 1
        FROM sales_transactions middle
        WHERE middle.customer_id = current.customer_id
        AND middle.transaction_date > previous.transaction_date
        AND middle.transaction_date < current.transaction_date
    );
```

Lateral Joins Lateral joins allow subqueries in the FROM clause to reference columns from preceding tables, enabling powerful analytics patterns:

```
-- Finding top 3 products for each customer
SELECT
```

```

        c.customer_id,
        c.customer_name,
        top_products.product_id,
        top_products.product_name,
        top_products.total_spent,
        top_products.product_rank
FROM
    customers c
CROSS JOIN LATERAL (
    SELECT
        p.product_id,
        p.product_name,
        SUM(s.sales_amount) as total_spent,
        RANK() OVER (ORDER BY SUM(s.sales_amount) DESC) as product_rank
    FROM
        sales_transactions s
    JOIN
        products p ON s.product_id = p.product_id
    WHERE
        s.customer_id = c.customer_id
    GROUP BY
        p.product_id, p.product_name
    ORDER BY
        total_spent DESC
    LIMIT 3
) top_products;

-- Time-based cohort analysis with lateral joins
SELECT
    first_month,
    COUNT(DISTINCT customer_id) as cohort_size,
    retention.month_number,
    retention.active_customers,
    ROUND(retention.active_customers * 100.0 / COUNT(DISTINCT customer_id), 2) as retention
FROM (
    SELECT
        customer_id,
        DATE_TRUNC('month', MIN(transaction_date)) as first_month
    FROM
        sales_transactions
    GROUP BY
        customer_id
) cohorts
CROSS JOIN LATERAL (
    SELECT
        months.month_number,

```

```

        COUNT(DISTINCT s.customer_id) as active_customers
    FROM (
        SELECT generate_series(0, 11) as month_number
    ) months
    LEFT JOIN
        sales_transactions s
        ON s.customer_id = cohorts.customer_id
        AND DATE_TRUNC('month', s.transaction_date) = cohorts.first_month + (months.month_number)
    GROUP BY
        months.month_number
) retention
GROUP BY
    first_month, retention.month_number, retention.active_customers
ORDER BY
    first_month, retention.month_number;

```

Advanced Set Operations for Analysis

```

-- Finding customer overlap between product categories
WITH electronics_customers AS (
    SELECT DISTINCT customer_id
    FROM sales_transactions s
    JOIN products p ON s.product_id = p.product_id
    WHERE p.product_category = 'Electronics'
),
appliance_customers AS (
    SELECT DISTINCT customer_id
    FROM sales_transactions s
    JOIN products p ON s.product_id = p.product_id
    WHERE p.product_category = 'Appliances'
)
SELECT
    'Electronics only' as customer_segment,
    COUNT(*) as customer_count
FROM
    electronics_customers
WHERE
    customer_id NOT IN (SELECT customer_id FROM appliance_customers)

UNION ALL

SELECT
    'Appliances only' as customer_segment,
    COUNT(*) as customer_count
FROM
    appliance_customers

```

```

WHERE
    customer_id NOT IN (SELECT customer_id FROM electronics_customers)

UNION ALL

SELECT
    'Both categories' as customer_segment,
    COUNT(*) as customer_count
FROM
    electronics_customers
WHERE
    customer_id IN (SELECT customer_id FROM appliance_customers);

-- Time-period comparison using UNION ALL
SELECT
    'Current Quarter' as period,
    product_category,
    SUM(sales_amount) as sales_amount
FROM
    sales_transactions s
JOIN
    products p ON s.product_id = p.product_id
WHERE
    transaction_date BETWEEN '2023-01-01' AND '2023-03-31'
GROUP BY
    product_category

UNION ALL

SELECT
    'Previous Quarter' as period,
    product_category,
    SUM(sales_amount) as sales_amount
FROM
    sales_transactions s
JOIN
    products p ON s.product_id = p.product_id
WHERE
    transaction_date BETWEEN '2022-10-01' AND '2022-12-31'
GROUP BY
    product_category

UNION ALL

SELECT
    'Year-over-Year' as period,

```

```

        product_category,
        SUM(sales_amount) as sales_amount
FROM
    sales_transactions s
JOIN
    products p ON s.product_id = p.product_id
WHERE
    transaction_date BETWEEN '2022-01-01' AND '2022-03-31'
GROUP BY
    product_category

ORDER BY
    product_category,
    CASE
        WHEN period = 'Current Quarter' THEN 1
        WHEN period = 'Previous Quarter' THEN 2
        WHEN period = 'Year-over-Year' THEN 3
    END;

```

3. Time-Based Operations

Time-based analysis is central to analytics, enabling trend identification, seasonality analysis, and forecasting.

Date and Time Functions

```

```sql
-- Date extraction and manipulation
SELECT
 transaction_date,
 -- Extract components
 EXTRACT(YEAR FROM transaction_date) AS year,
 EXTRACT(QUARTER FROM transaction_date) AS quarter,
 EXTRACT(MONTH FROM transaction_date) AS month,
 EXTRACT(DAY FROM transaction_date) AS day,
 EXTRACT(DOW FROM transaction_date) AS day_of_week, -- 0 (Sunday) to 6 (Saturday)
 EXTRACT(ISODOW FROM transaction_date) AS iso_day_of_week, -- 1 (Monday) to 7 (Sunday)
 EXTRACT(DOY FROM transaction_date) AS day_of_year,
 EXTRACT(WEEK FROM transaction_date) AS week,

 -- Truncation to specific precision
 DATE_TRUNC('year', transaction_date) AS year_start,
 DATE_TRUNC('quarter', transaction_date) AS quarter_start,
 DATE_TRUNC('month', transaction_date) AS month_start,
 DATE_TRUNC('week', transaction_date) AS week_start,
 DATE_TRUNC('day', transaction_date) AS day_start,

```

```

-- Date math
transaction_date + INTERVAL '1 month' AS one_month_later,
transaction_date - INTERVAL '1 year' AS one_year_ago,

-- Date difference
CURRENT_DATE - transaction_date AS days_since_transaction,
EXTRACT(EPOCH FROM (CURRENT_TIMESTAMP - transaction_date)) / 86400 AS days_since_decimal
FROM
 sales_transactions;

```

## Time Series Analysis

```

-- Creating a complete time series without gaps
WITH date_spine AS (
 SELECT
 generate_series(
 DATE_TRUNC('month', MIN(transaction_date)),
 DATE_TRUNC('month', MAX(transaction_date)),
 INTERVAL '1 month'
) AS month_start
 FROM
 sales_transactions
)
SELECT
 date_spine.month_start,
 COALESCE(SUM(s.sales_amount), 0) AS monthly_sales
FROM
 date_spine
LEFT JOIN
 sales_transactions s
 ON DATE_TRUNC('month', s.transaction_date) = date_spine.month_start
GROUP BY
 date_spine.month_start
ORDER BY
 date_spine.month_start;

-- Moving averages (3-month rolling average)
WITH monthly_sales AS (
 SELECT
 DATE_TRUNC('month', transaction_date) AS month,
 SUM(sales_amount) AS total_sales
 FROM
 sales_transactions
 GROUP BY
 DATE_TRUNC('month', transaction_date)
)

```



```

SELECT
 month,
 total_sales,
 AVG(total_sales) OVER (
 ORDER BY month
 ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
) AS moving_avg_3month
FROM
 monthly_sales
ORDER BY
 month;

-- Year-over-year comparison
WITH monthly_sales AS (
 SELECT
 EXTRACT(YEAR FROM transaction_date) AS year,
 EXTRACT(MONTH FROM transaction_date) AS month,
 SUM(sales_amount) AS monthly_sales
 FROM
 sales_transactions
 GROUP BY
 EXTRACT(YEAR FROM transaction_date),
 EXTRACT(MONTH FROM transaction_date)
)
SELECT
 current_year.year,
 current_year.month,
 current_year.monthly_sales AS current_year_sales,
 previous_year.monthly_sales AS previous_year_sales,
 current_year.monthly_sales - previous_year.monthly_sales AS absolute_difference,
 CASE
 WHEN previous_year.monthly_sales = 0 THEN NULL
 ELSE ROUND((current_year.monthly_sales - previous_year.monthly_sales) * 100.0 / previous_year.monthly_sales)
 END AS percentage_growth
FROM
 monthly_sales current_year
LEFT JOIN
 monthly_sales previous_year
 ON current_year.month = previous_year.month
 AND current_year.year = previous_year.year + 1
ORDER BY
 current_year.year, current_year.month;

```

## Cohort Analysis

```

-- Customer cohort retention analysis
WITH first_purchases AS (
 -- Get the first purchase month for each customer
 SELECT
 customer_id,
 DATE_TRUNC('month', MIN(transaction_date)) AS first_purchase_month
 FROM
 sales_transactions
 GROUP BY
 customer_id
),
cohort_activity AS (
 -- Calculate activity for each customer by month
 SELECT
 fp.customer_id,
 fp.first_purchase_month AS cohort_month,
 DATE_TRUNC('month', s.transaction_date) AS activity_month,
 -- Calculate the number of months between first purchase and this activity
 EXTRACT(YEAR FROM DATE_TRUNC('month', s.transaction_date)) * 12 +
 EXTRACT(MONTH FROM DATE_TRUNC('month', s.transaction_date)) -
 (EXTRACT(YEAR FROM fp.first_purchase_month) * 12 +
 EXTRACT(MONTH FROM fp.first_purchase_month)) AS months_since_first_purchase
 FROM
 first_purchases fp
 JOIN
 sales_transactions s ON fp.customer_id = s.customer_id
),
cohort_size AS (
 -- Count of customers in each cohort
 SELECT
 cohort_month,
 COUNT(DISTINCT customer_id) AS num_customers
 FROM
 first_purchases
 GROUP BY
 cohort_month
),
retention_table AS (
 -- Count distinct active customers for each cohort and month
 SELECT
 ca.cohort_month,
 ca.months_since_first_purchase,
 COUNT(DISTINCT ca.customer_id) AS num_customers
 FROM
 cohort_activity ca
 GROUP BY

```

```

 ca.cohort_month,
 ca.months_since_first_purchase
)
SELECT
 rt.cohort_month,
 cs.num_customers AS original_cohort_size,
 rt.months_since_first_purchase,
 rt.num_customers AS active_customers,
 ROUND(rt.num_customers * 100.0 / cs.num_customers, 2) AS retention_rate
FROM
 retention_table rt
JOIN
 cohort_size cs ON rt.cohort_month = cs.cohort_month
ORDER BY
 rt.cohort_month,
 rt.months_since_first_purchase;

```

## Seasonal Analysis

```

-- Monthly seasonality analysis
SELECT
 EXTRACT(MONTH FROM transaction_date) AS month,
 TO_CHAR(DATE_TRUNC('month', transaction_date), 'Month') AS month_name,
 EXTRACT(YEAR FROM transaction_date) AS year,
 SUM(sales_amount) AS monthly_sales,

 -- Compare to overall monthly average
 SUM(sales_amount) /
 AVG(SUM(sales_amount)) OVER (PARTITION BY EXTRACT(YEAR FROM transaction_date))
 AS year_seasonality_index
FROM
 sales_transactions
GROUP BY
 EXTRACT(MONTH FROM transaction_date),
 TO_CHAR(DATE_TRUNC('month', transaction_date), 'Month'),
 EXTRACT(YEAR FROM transaction_date)
ORDER BY
 year, month;

-- Day-of-week patterns
SELECT
 EXTRACT(ISODOW FROM transaction_date) AS day_of_week,
 TO_CHAR(transaction_date, 'Day') AS day_name,
 COUNT(*) AS transaction_count,
 SUM(sales_amount) AS total_sales,
 AVG(sales_amount) AS avg_transaction_value

```

```

FROM
 sales_transactions
GROUP BY
 EXTRACT(ISODOW FROM transaction_date),
 TO_CHAR(transaction_date, 'Day')
ORDER BY
 day_of_week;

-- Hourly patterns
SELECT
 EXTRACT(HOUR FROM transaction_timestamp) AS hour_of_day,
 COUNT(*) AS transaction_count,
 SUM(sales_amount) AS total_sales,
 ROUND(100.0 * COUNT(*) / SUM(COUNT(*) OVER (), 2) AS percent_of_daily_transactions
FROM
 sales_transactions
GROUP BY
 EXTRACT(HOUR FROM transaction_timestamp)
ORDER BY
 hour_of_day;

```

## Time Windows and Intervals

```

-- Activity in the last 30, 60, 90 days
SELECT
 'Last 30 days' AS time_window,
 COUNT(DISTINCT customer_id) AS active_customers,
 COUNT(*) AS transaction_count,
 SUM(sales_amount) AS total_sales
FROM
 sales_transactions
WHERE
 transaction_date >= CURRENT_DATE - INTERVAL '30 days'

UNION ALL

SELECT
 'Last 60 days' AS time_window,
 COUNT(DISTINCT customer_id) AS active_customers,
 COUNT(*) AS transaction_count,
 SUM(sales_amount) AS total_sales
FROM
 sales_transactions
WHERE
 transaction_date >= CURRENT_DATE - INTERVAL '60 days'

```

```

UNION ALL

SELECT
 'Last 90 days' AS time_window,
 COUNT(DISTINCT customer_id) AS active_customers,
 COUNT(*) AS transaction_count,
 SUM(sales_amount) AS total_sales
FROM
 sales_transactions
WHERE
 transaction_date >= CURRENT_DATE - INTERVAL '90 days'

ORDER BY
 CASE
 WHEN time_window = 'Last 30 days' THEN 1
 WHEN time_window = 'Last 60 days' THEN 2
 WHEN time_window = 'Last 90 days' THEN 3
 END;

-- Comparing custom time periods
WITH current_period AS (
 SELECT
 product_category,
 SUM(sales_amount) AS period_sales
 FROM
 sales_transactions s
 JOIN
 products p ON s.product_id = p.product_id
 WHERE
 transaction_date BETWEEN '2023-03-01' AND '2023-03-31'
 GROUP BY
 product_category
),
previous_period AS (
 SELECT
 product_category,
 SUM(sales_amount) AS period_sales
 FROM
 sales_transactions s
 JOIN
 products p ON s.product_id = p.product_id
 WHERE
 transaction_date BETWEEN '2023-02-01' AND '2023-02-28'
 GROUP BY
 product_category
),

```

```

year_ago_period AS (
 SELECT
 product_category,
 SUM(sales_amount) AS period_sales
 FROM
 sales_transactions s
 JOIN
 products p ON s.product_id = p.product_id
 WHERE
 transaction_date BETWEEN '2022-03-01' AND '2022-03-31'
 GROUP BY
 product_category
)
SELECT
 cp.product_category,
 cp.period_sales AS current_period_sales,
 pp.period_sales AS previous_period_sales,
 yap.period_sales AS year_ago_sales,

 -- Month-over-month change
 cp.period_sales - pp.period_sales AS mom_change,
 CASE
 WHEN pp.period_sales = 0 THEN NULL
 ELSE (cp.period_sales - pp.period_sales) * 100.0 / pp.period_sales
 END AS mom_percent_change,

 -- Year-over-year change
 cp.period_sales - yap.period_sales AS yoy_change,
 CASE
 WHEN yap.period_sales = 0 THEN NULL
 ELSE (cp.period_sales - yap.period_sales) * 100.0 / yap.period_sales
 END AS yoy_percent_change
FROM
 current_period cp
LEFT JOIN
 previous_period pp ON cp.product_category = pp.product_category
LEFT JOIN
 year_ago_period yap ON cp.product_category = yap.product_category
ORDER BY
 cp.period_sales DESC;

```

## Working with Timestamps and Time Zones

```

-- Converting between time zones
SELECT
 transaction_timestamp,

```

```

 transaction_timestamp AT TIME ZONE 'UTC' AS utc_time,
 transaction_timestamp AT TIME ZONE 'America/New_York' AS eastern_time,
 transaction_timestamp AT TIME ZONE 'America/Los_Angeles' AS pacific_time
FROM
 sales_transactions
LIMIT 10;

-- Aggregating across time zones to a standard time
SELECT
 DATE_TRUNC('day', transaction_timestamp AT TIME ZONE 'UTC') AS day_utc,
 SUM(sales_amount) AS total_sales,
 COUNT(*) AS transaction_count
FROM
 sales_transactions
GROUP BY
 DATE_TRUNC('day', transaction_timestamp AT TIME ZONE 'UTC')
ORDER BY
 day_utc;

-- Finding peak hours by location's local time
SELECT
 store_timezone,
 EXTRACT(HOUR FROM transaction_timestamp AT TIME ZONE store_timezone) AS local_hour,
 COUNT(*) AS transaction_count,
 SUM(sales_amount) AS total_sales
FROM
 sales_transactions s
JOIN
 stores st ON s.store_id = st.store_id
GROUP BY
 store_timezone,
 EXTRACT(HOUR FROM transaction_timestamp AT TIME ZONE store_timezone)
ORDER BY
 store_timezone,
 local_hour;

```

#### 4. User-Defined Functions for Analytics

User-defined functions (UDFs) can significantly enhance your analytical capabilities by encapsulating complex logic into reusable components.

##### Creating Analytical UDFs

```

-- Simple UDF for age calculation
CREATE OR REPLACE FUNCTION calculate_age(birth_date DATE)
RETURNS INTEGER AS $

```

```

BEGIN
 RETURN EXTRACT(YEAR FROM AGE(CURRENT_DATE, birth_date));
END;
$ LANGUAGE plpgsql;

-- Usage
SELECT
 customer_id,
 customer_name,
 birth_date,
 calculate_age(birth_date) AS customer_age
FROM
 customers;

-- Customer lifetime value function
CREATE OR REPLACE FUNCTION customer_lifetime_value(
 customer_id_param INTEGER,
 start_date DATE DEFAULT '1900-01-01',
 end_date DATE DEFAULT CURRENT_DATE
)
RETURNS NUMERIC AS $
DECLARE
 total_value NUMERIC;
BEGIN
 SELECT COALESCE(SUM(sales_amount), 0)
 INTO total_value
 FROM sales_transactions
 WHERE customer_id = customer_id_param
 AND transaction_date BETWEEN start_date AND end_date;

 RETURN total_value;
END;
$ LANGUAGE plpgsql;

-- Usage
SELECT
 customer_id,
 customer_name,
 customer_lifetime_value(customer_id) AS all_time_value,
 customer_lifetime_value(customer_id, '2023-01-01', '2023-12-31') AS current_year_value
FROM
 customers;

```

## Table-Valued Functions for Analytics



```

-- Product performance analysis function
CREATE OR REPLACE FUNCTION product_performance(
 start_date DATE,
 end_date DATE,
 category_filter TEXT DEFAULT NULL
)
RETURNS TABLE (
 product_id INTEGER,
 product_name TEXT,
 product_category TEXT,
 total_sales NUMERIC,
 transaction_count BIGINT,
 unique_customers BIGINT,
 avg_sale_value NUMERIC
) AS $
BEGIN
 RETURN QUERY
 SELECT
 p.product_id,
 p.product_name,
 p.product_category,
 COALESCE(SUM(s.sales_amount), 0) AS total_sales,
 COUNT(s.transaction_id) AS transaction_count,
 COUNT(DISTINCT s.customer_id) AS unique_customers,
 CASE
 WHEN COUNT(s.transaction_id) = 0 THEN 0
 ELSE COALESCE(SUM(s.sales_amount), 0) / COUNT(s.transaction_id)
 END AS avg_sale_value
 FROM
 products p
 LEFT JOIN
 sales_transactions s ON p.product_id = s.product_id
 AND s.transaction_date BETWEEN start_date AND end_date
 WHERE
 (category_filter IS NULL OR p.product_category = category_filter)
 GROUP BY
 p.product_id, p.product_name, p.product_category
 ORDER BY
 total_sales DESC;
END;
$ LANGUAGE plpgsql;

-- Usage
SELECT * FROM product_performance('2023-01-01', '2023-03-31', 'Electronics');
SELECT * FROM product_performance('2023-01-01', '2023-03-31');

```

## Statistical UDFs for Advanced Analytics

```
-- Z-score calculation (standardization)
CREATE OR REPLACE FUNCTION z_score(value NUMERIC, mean NUMERIC, std_dev NUMERIC)
RETURNS NUMERIC AS $
BEGIN
 IF std_dev = 0 THEN
 RETURN 0;
 ELSE
 RETURN (value - mean) / std_dev;
 END IF;
END;
$ LANGUAGE plpgsql;

-- Using the z-score function to identify outliers
WITH product_stats AS (
 SELECT
 product_category,
 AVG(sales_amount) AS avg_sale,
 STDDEV(sales_amount) AS stddev_sale
 FROM
 sales_transactions s
 JOIN
 products p ON s.product_id = p.product_id
 GROUP BY
 product_category
)
SELECT
 s.transaction_id,
 p.product_id,
 p.product_name,
 p.product_category,
 s.sales_amount,
 ps.avg_sale,
 ps.stddev_sale,
 z_score(s.sales_amount, ps.avg_sale, ps.stddev_sale) AS z_score_value
FROM
 sales_transactions s
JOIN
 products p ON s.product_id = p.product_id
JOIN
 product_stats ps ON p.product_category = ps.product_category
WHERE
 ABS(z_score(s.sales_amount, ps.avg_sale, ps.stddev_sale)) > 2
ORDER BY
 ABS(z_score(s.sales_amount, ps.avg_sale, ps.stddev_sale)) DESC;
```

```

-- Percentile calculation function
CREATE OR REPLACE FUNCTION percentile_disc_array(
 values_array NUMERIC[],
 percentile NUMERIC
)
RETURNS NUMERIC AS $
DECLARE
 sorted_array NUMERIC[];
 array_length INTEGER;
 percentile_position INTEGER;
BEGIN
 -- Sort the array
 SELECT array_agg(val ORDER BY val)
 INTO sorted_array
 FROM unnest(values_array) AS val;

 -- Get the array length
 array_length := array_length(sorted_array, 1);

 -- Calculate the position for the percentile
 percentile_position := CEIL(percentile * array_length);

 -- Return the value at the percentile position
 IF percentile_position = 0 THEN
 RETURN sorted_array[1];
 ELSE
 RETURN sorted_array[percentile_position];
 END IF;
END;
$ LANGUAGE plpgsql;

-- Using the percentile function for RFM (Recency, Frequency, Monetary) segmentation
WITH customer_rfm AS (
 SELECT
 customer_id,
 CURRENT_DATE - MAX(transaction_date) AS recency,
 COUNT(*) AS frequency,
 SUM(sales_amount) AS monetary
 FROM
 sales_transactions
 GROUP BY
 customer_id
),
rfm_stats AS (
 SELECT

```

```

percentile_disc_array(ARRAY_AGG(recency), 0.2) AS r_20,
percentile_disc_array(ARRAY_AGG(recency), 0.4) AS r_40,
percentile_disc_array(ARRAY_AGG(recency), 0.6) AS r_60,
percentile_disc_array(ARRAY_AGG(recency), 0.8) AS r_80,

percentile_disc_array(ARRAY_AGG(frequency), 0.2) AS f_20,
percentile_disc_array(ARRAY_AGG(frequency), 0.4) AS f_40,
percentile_disc_array(ARRAY_AGG(frequency), 0.6) AS f_60,
percentile_disc_array(ARRAY_AGG(frequency), 0.8) AS f_80,

percentile_disc_array(ARRAY_AGG(monetary), 0.2) AS m_20,
percentile_disc_array(ARRAY_AGG(monetary), 0.4) AS m_40,
percentile_disc_array(ARRAY_AGG(monetary), 0.6) AS m_60,
percentile_disc_array(ARRAY_AGG(monetary), 0.8) AS m_80
FROM
 customer_rfm
)
SELECT
 c.customer_id,
 c.customer_name,
 rfm.recency,
 rfm.frequency,
 rfm.monetary,

 -- Recency score (lower is better)
 CASE
 WHEN rfm.recency <= rs.r_20 THEN 5
 WHEN rfm.recency <= rs.r_40 THEN 4
 WHEN rfm.recency <= rs.r_60 THEN 3
 WHEN rfm.recency <= rs.r_80 THEN 2
 ELSE 1
 END AS r_score,

 -- Frequency score (higher is better)
 CASE
 WHEN rfm.frequency >= rs.f_80 THEN 5
 WHEN rfm.frequency >= rs.f_60 THEN 4
 WHEN rfm.frequency >= rs.f_40 THEN 3
 WHEN rfm.frequency >= rs.f_20 THEN 2
 ELSE 1
 END AS f_score,

 -- Monetary score (higher is better)
 CASE
 WHEN rfm.monetary >= rs.m_80 THEN 5
 WHEN rfm.monetary >= rs.m_60 THEN 4

```

```

 WHEN rfm.monetary >= rs.m_40 THEN 3
 WHEN rfm.monetary >= rs.m_20 THEN 2
 ELSE 1
 END AS m_score
FROM
 customer_rfm rfm
JOIN
 customers c ON rfm.customer_id = c.customer_id
CROSS JOIN
 rfm_stats rs
ORDER BY
 (r_score + f_score + m_score) DESC;

```

## Part III: Data Modeling for Analytics

### 1. Dimensional Modeling Concepts

Dimensional modeling is a database design technique specifically optimized for data warehouses and analytics. It focuses on delivering data that is intuitive to business users and provides high performance for analytical queries.

#### Facts and Dimensions

*-- Core concepts of dimensional modeling:*  
*-- 1. Fact tables: contain measurements, metrics, or facts about business processes*  
*-- 2. Dimension tables: contain descriptive attributes about business entities*

*-- Example fact table: sales\_fact*

```

CREATE TABLE sales_fact (
 sale_id INTEGER PRIMARY KEY,
 date_key INTEGER REFERENCES date_dimension(date_key),
 product_key INTEGER REFERENCES product_dimension(product_key),
 customer_key INTEGER REFERENCES customer_dimension(customer_key),
 store_key INTEGER REFERENCES store_dimension(store_key),
 promotion_key INTEGER REFERENCES promotion_dimension(promotion_key),
 quantity INTEGER NOT NULL,
 unit_price NUMERIC(12,2) NOT NULL,
 discount_amount NUMERIC(12,2) NOT NULL,
 sales_amount NUMERIC(12,2) NOT NULL,
 cost_amount NUMERIC(12,2) NOT NULL,
 profit_amount NUMERIC(12,2) NOT NULL
);

```

*-- Example dimension table: product\_dimension*

```

CREATE TABLE product_dimension (
 product_key INTEGER PRIMARY KEY,
 product_id INTEGER NOT NULL, -- business key

```

```

 product_name VARCHAR(100) NOT NULL,
 product_description TEXT,
 brand VARCHAR(50),
 category VARCHAR(50) NOT NULL,
 subcategory VARCHAR(50),
 department VARCHAR(50),
 size VARCHAR(20),
 color VARCHAR(20),
 weight NUMERIC(8,2),
 cost NUMERIC(12,2),
 retail_price NUMERIC(12,2),
 effective_date DATE NOT NULL,
 expiration_date DATE,
 is_current BOOLEAN NOT NULL
);

```

#### Key Concepts in Dimensional Modeling

**\*\*1. Grain\*\***

The grain defines the level of detail in a fact table. It answers the question: "What does a

```

```sql
-- Examples of different grain levels:
-- Transaction grain (most detailed)
CREATE TABLE sales_transaction_fact (
    transaction_id INTEGER,
    transaction_line_id INTEGER,
    date_key INTEGER,
    product_key INTEGER,
    customer_key INTEGER,
    quantity INTEGER,
    sales_amount NUMERIC(12,2),
    PRIMARY KEY (transaction_id, transaction_line_id)
);

```

```

-- Daily product sales grain (aggregated)
CREATE TABLE daily_product_sales_fact (
    date_key INTEGER,
    product_key INTEGER,
    store_key INTEGER,
    total_quantity INTEGER,
    total_sales_amount NUMERIC(12,2),
    PRIMARY KEY (date_key, product_key, store_key)
);

```

```

-- Monthly customer grain (highly aggregated)

```

```

CREATE TABLE monthly_customer_sales_fact (
    year_month INTEGER, -- YYYYMM format
    customer_key INTEGER,
    total_transactions INTEGER,
    total_quantity INTEGER,
    total_sales_amount NUMERIC(12,2),
    PRIMARY KEY (year_month, customer_key)
);

```

2. Dimension Table Design

Dimensions provide the context for facts and support filtering, grouping, and labeling.

```

-- Shared dimension pattern (conformed dimension)
-- This customer dimension can be used across multiple fact tables
CREATE TABLE customer_dimension (
    customer_key INTEGER PRIMARY KEY,
    customer_id VARCHAR(20) NOT NULL, -- business key
    customer_name VARCHAR(100) NOT NULL,
    email VARCHAR(100),
    phone VARCHAR(20),
    address_line1 VARCHAR(100),
    address_line2 VARCHAR(100),
    city VARCHAR(50),
    state_province VARCHAR(50),
    postal_code VARCHAR(20),
    country VARCHAR(50),
    customer_since_date DATE,
    customer_segment VARCHAR(30),
    credit_rating VARCHAR(10),
    lifetime_value NUMERIC(12,2)
);

-- Role-playing dimension (same dimension used in different contexts)
CREATE TABLE date_dimension (
    date_key INTEGER PRIMARY KEY,
    full_date DATE UNIQUE NOT NULL,
    day_of_week INTEGER NOT NULL, -- 1-7
    day_of_week_name VARCHAR(10) NOT NULL, -- Monday, Tuesday, etc.
    day_of_month INTEGER NOT NULL, -- 1-31
    day_of_year INTEGER NOT NULL, -- 1-366
    week_of_month INTEGER NOT NULL,
    week_of_year INTEGER NOT NULL,
    month_number INTEGER NOT NULL, -- 1-12
    month_name VARCHAR(10) NOT NULL, -- January, February, etc.
    quarter INTEGER NOT NULL, -- 1-4

```

```

        quarter_name VARCHAR(10) NOT NULL, -- Q1, Q2, Q3, Q4
        year INTEGER NOT NULL
    );

-- Using the same date dimension in different roles
SELECT
    od.full_date AS order_date,
    sd.full_date AS ship_date,
    pd.full_date AS payment_date,
    f.order_amount,
    f.ship_amount,
    f.payment_amount
FROM
    order_fact f
JOIN
    date_dimension od ON f.order_date_key = od.date_key
JOIN
    date_dimension sd ON f.ship_date_key = sd.date_key
JOIN
    date_dimension pd ON f.payment_date_key = pd.date_key;

```

3. Fact Table Types

There are three main types of fact tables, each serving different analytical needs:

```

-- 1. Transaction fact table (records individual events)
CREATE TABLE sales_transaction_fact (
    transaction_id INTEGER,
    line_item_id INTEGER,
    date_key INTEGER,
    product_key INTEGER,
    customer_key INTEGER,
    quantity INTEGER NOT NULL,
    sales_amount NUMERIC(12,2) NOT NULL,
    PRIMARY KEY (transaction_id, line_item_id)
);

-- 2. Periodic snapshot fact table (records regular snapshots)
CREATE TABLE monthly_inventory_fact (
    date_key INTEGER, -- last day of month
    product_key INTEGER,
    store_key INTEGER,
    quantity_on_hand INTEGER,
    quantity_on_order INTEGER,
    inventory_value NUMERIC(12,2),
    PRIMARY KEY (date_key, product_key, store_key)
);

```


-- 3. Accumulating snapshot fact table (records process milestones)

```
CREATE TABLE order_fulfillment_fact (  
    order_key INTEGER PRIMARY KEY,  
    customer_key INTEGER,  
    product_key INTEGER,  
    order_date_key INTEGER,  
    approved_date_key INTEGER,  
    picked_date_key INTEGER,  
    shipped_date_key INTEGER,  
    delivered_date_key INTEGER,  
    order_amount NUMERIC(12,2),  
    shipping_amount NUMERIC(12,2),  
    total_amount NUMERIC(12,2),  
    order_status VARCHAR(20)  
);
```

4. Slowly Changing Dimensions

Dimensions can change over time, and there are different strategies for handling these changes:

-- Type 1 SCD: Overwrite the old value (no history)

```
UPDATE product_dimension  
SET  
    category = 'Home Electronics',  
    subcategory = 'Smart Home'  
WHERE  
    product_id = 12345;
```

-- Type 2 SCD: Add a new row with the updated values (preserve history)

-- 1. Set expiration on the current record

```
UPDATE product_dimension  
SET  
    expiration_date = CURRENT_DATE,  
    is_current = FALSE  
WHERE  
    product_id = 12345  
    AND is_current = TRUE;
```

-- 2. Insert the new version

```
INSERT INTO product_dimension (  
    product_key, product_id, product_name, category, subcategory,  
    effective_date, expiration_date, is_current  
)  
VALUES (  
    NEXTVAL('product_key_seq'), 12345, 'Smart Thermostat',
```

```

        'Home Electronics', 'Smart Home',
        CURRENT_DATE, NULL, TRUE
    );

    -- Query current product information
    SELECT * FROM product_dimension WHERE is_current = TRUE;

    -- Query historical product information
    SELECT * FROM product_dimension
    WHERE product_id = 12345
    ORDER BY effective_date;

    -- Type 3 SCD: Add new columns to track a limited history
    CREATE TABLE product_dimension_type3 (
        product_key INTEGER PRIMARY KEY,
        product_id INTEGER NOT NULL,
        product_name VARCHAR(100) NOT NULL,
        current_category VARCHAR(50) NOT NULL,
        previous_category VARCHAR(50),
        category_change_date DATE,
        -- other attributes
        is_current BOOLEAN NOT NULL
    );

```

5. Junk Dimensions

A junk dimension combines several low-cardinality flags or attributes into a single dimension to avoid dimension bloat:

```

    -- Creating a junk dimension for order flags
    CREATE TABLE order_flags_dimension (
        flag_key INTEGER PRIMARY KEY,
        is_rush_order BOOLEAN NOT NULL,
        is_gift BOOLEAN NOT NULL,
        has_gift_wrapping BOOLEAN NOT NULL,
        has_special_instructions BOOLEAN NOT NULL,
        delivery_method VARCHAR(20) NOT NULL,
        payment_method VARCHAR(20) NOT NULL
    );

    -- Populate with all possible combinations
    INSERT INTO order_flags_dimension
    (flag_key, is_rush_order, is_gift, has_gift_wrapping,
     has_special_instructions, delivery_method, payment_method)
    VALUES
    (1, FALSE, FALSE, FALSE, FALSE, 'Standard', 'Credit Card'),
    (2, TRUE, FALSE, FALSE, FALSE, 'Express', 'Credit Card'),

```

```

(3, FALSE, TRUE, TRUE, FALSE, 'Standard', 'Credit Card'),
-- Add all other relevant combinations
(16, TRUE, TRUE, TRUE, TRUE, 'Express', 'PayPal');

-- Use in fact table
CREATE TABLE order_fact (
    order_key INTEGER PRIMARY KEY,
    date_key INTEGER,
    customer_key INTEGER,
    flag_key INTEGER REFERENCES order_flags_dimension(flag_key),
    order_amount NUMERIC(12,2)
);

```

6. Degenerate Dimensions

Degenerate dimensions are attributes that are neither measures (facts) nor dimension attributes but are still useful for analysis:

```

CREATE TABLE sales_fact (
    date_key INTEGER,
    product_key INTEGER,
    store_key INTEGER,
    invoice_number VARCHAR(20), -- degenerate dimension
    po_number VARCHAR(20), -- degenerate dimension
    quantity INTEGER,
    sales_amount NUMERIC(12,2),
    PRIMARY KEY (date_key, product_key, store_key, invoice_number)
);

```

2. Star and Snowflake Schemas

The two primary dimensional modeling patterns are star schemas and snowflake schemas. Both are designed for analytical workloads but have different approaches to dimension normalization.

Star Schema A star schema features a central fact table surrounded by denormalized dimension tables. Each dimension table has a primary key that directly relates to the fact table, creating a star-like pattern.

-- Star Schema Example

```

-- Dimension tables
CREATE TABLE date_dimension (
    date_key INTEGER PRIMARY KEY,
    full_date DATE,
    day_of_week VARCHAR(10),
    day_of_month INTEGER,
    month VARCHAR(10),

```

```

        quarter VARCHAR(2),
        year INTEGER
    );

CREATE TABLE product_dimension (
    product_key INTEGER PRIMARY KEY,
    product_id VARCHAR(20),
    product_name VARCHAR(100),
    brand VARCHAR(50),
    category VARCHAR(50),
    subcategory VARCHAR(50),
    unit_cost NUMERIC(12,2),
    unit_price NUMERIC(12,2)
);

CREATE TABLE store_dimension (
    store_key INTEGER PRIMARY KEY,
    store_id VARCHAR(20),
    store_name VARCHAR(100),
    store_type VARCHAR(30),
    region VARCHAR(30),
    city VARCHAR(50),
    state VARCHAR(2),
    country VARCHAR(30),
    open_date DATE
);

-- Fact table
CREATE TABLE sales_fact (
    sale_key SERIAL PRIMARY KEY,
    date_key INTEGER REFERENCES date_dimension(date_key),
    product_key INTEGER REFERENCES product_dimension(product_key),
    store_key INTEGER REFERENCES store_dimension(store_key),
    quantity INTEGER,
    unit_price NUMERIC(12,2),
    extended_price NUMERIC(12,2),
    cost NUMERIC(12,2),
    profit NUMERIC(12,2)
);

-- Typical star schema query
SELECT
    d.year,
    d.quarter,
    p.category,
    p.brand,

```

```

        s.region,
        SUM(f.quantity) AS total_units,
        SUM(f.extended_price) AS total_sales,
        SUM(f.profit) AS total_profit
FROM
    sales_fact f
JOIN
    date_dimension d ON f.date_key = d.date_key
JOIN
    product_dimension p ON f.product_key = p.product_key
JOIN
    store_dimension s ON f.store_key = s.store_key
WHERE
    d.year = 2023
GROUP BY
    d.year, d.quarter, p.category, p.brand, s.region
ORDER BY
    d.quarter, total_sales DESC;

```

Snowflake Schema A snowflake schema normalizes dimension tables by creating additional tables that relate to the dimension tables, rather than directly to the fact table. This creates a more complex, snowflake-like structure.

-- Snowflake Schema Example

-- Normalized dimension tables

```

CREATE TABLE category_dimension (
    category_key INTEGER PRIMARY KEY,
    category_name VARCHAR(50)
);

CREATE TABLE subcategory_dimension (
    subcategory_key INTEGER PRIMARY KEY,
    category_key INTEGER REFERENCES category_dimension(category_key),
    subcategory_name VARCHAR(50)
);

CREATE TABLE brand_dimension (
    brand_key INTEGER PRIMARY KEY,
    brand_name VARCHAR(50),
    manufacturer VARCHAR(100)
);

CREATE TABLE product_dimension (
    product_key INTEGER PRIMARY KEY,
    product_id VARCHAR(20),

```

```

        product_name VARCHAR(100),
        brand_key INTEGER REFERENCES brand_dimension(brand_key),
        subcategory_key INTEGER REFERENCES subcategory_dimension(subcategory_key),
        unit_cost NUMERIC(12,2),
        unit_price NUMERIC(12,2)
    );

CREATE TABLE country_dimension (
    country_key INTEGER PRIMARY KEY,
    country_name VARCHAR(50)
);

CREATE TABLE state_dimension (
    state_key INTEGER PRIMARY KEY,
    country_key INTEGER REFERENCES country_dimension(country_key),
    state_name VARCHAR(50),
    state_code VARCHAR(10)
);

CREATE TABLE city_dimension (
    city_key INTEGER PRIMARY KEY,
    state_key INTEGER REFERENCES state_dimension(state_key),
    city_name VARCHAR(50)
);

CREATE TABLE store_dimension (
    store_key INTEGER PRIMARY KEY,
    store_id VARCHAR(20),
    store_name VARCHAR(100),
    store_type VARCHAR(30),
    city_key INTEGER REFERENCES city_dimension(city_key),
    open_date DATE
);

-- Fact table (same as in star schema)
CREATE TABLE sales_fact (
    sale_key SERIAL PRIMARY KEY,
    date_key INTEGER REFERENCES date_dimension(date_key),
    product_key INTEGER REFERENCES product_dimension(product_key),
    store_key INTEGER REFERENCES store_dimension(store_key),
    quantity INTEGER,
    unit_price NUMERIC(12,2),
    extended_price NUMERIC(12,2),
    cost NUMERIC(12,2),
    profit NUMERIC(12,2)
);

```

```

-- Snowflake schema query (more joins but same result)
SELECT
    d.year,
    d.quarter,
    c.category_name,
    b.brand_name,
    country.country_name,
    SUM(f.quantity) AS total_units,
    SUM(f.extended_price) AS total_sales,
    SUM(f.profit) AS total_profit
FROM
    sales_fact f
JOIN
    date_dimension d ON f.date_key = d.date_key
JOIN
    product_dimension p ON f.product_key = p.product_key
JOIN
    brand_dimension b ON p.brand_key = b.brand_key
JOIN
    subcategory_dimension sub ON p.subcategory_key = sub.subcategory_key
JOIN
    category_dimension c ON sub.category_key = c.category_key
JOIN
    store_dimension s ON f.store_key = s.store_key
JOIN
    city_dimension city ON s.city_key = city.city_key
JOIN
    state_dimension state ON city.state_key = state.state_key
JOIN
    country_dimension country ON state.country_key = country.country_key
WHERE
    d.year = 2023
GROUP BY
    d.year, d.quarter, c.category_name, b.brand_name, country.country_name
ORDER BY
    d.quarter, total_sales DESC;

```

Star vs. Snowflake Comparison **Star Schema Advantages:** - Simpler structure with fewer joins required for queries - Generally better query performance - Easier for business users to understand - Simpler ETL processes

Snowflake Schema Advantages: - More normalized structure uses less storage space - Better dimension consistency when attributes are shared - Can better represent complex hierarchical relationships - Often easier to maintain and update dimensions

3. Time and Date Dimensions

Time dimensions are among the most important in an analytical database. Nearly every fact table connects to at least one time dimension, and most analyses include time-based comparisons.

Creating a Comprehensive Date Dimension A well-designed date dimension makes time-based analysis much simpler:

```
-- Create the date dimension table
CREATE TABLE date_dimension (
    date_key INTEGER PRIMARY KEY,
    date_value DATE UNIQUE NOT NULL,

    -- Calendar hierarchical components
    year INTEGER NOT NULL,
    quarter INTEGER NOT NULL,
    quarter_name VARCHAR(10) NOT NULL, -- e.g., "Q1", "Q2"
    month_number INTEGER NOT NULL, -- 1 to 12
    month_name VARCHAR(10) NOT NULL, -- January, February, etc.
    month_name_short VARCHAR(3) NOT NULL, -- Jan, Feb, etc.
    week_of_year INTEGER NOT NULL, -- 1 to 53
    day_of_year INTEGER NOT NULL, -- 1 to 366
    day_of_month INTEGER NOT NULL, -- 1 to 31
    day_of_week INTEGER NOT NULL, -- 1 (Monday) to 7 (Sunday)
    day_of_week_name VARCHAR(10) NOT NULL, -- Monday, Tuesday, etc.
    day_of_week_name_short VARCHAR(3) NOT NULL, -- Mon, Tue, etc.

    -- Fiscal components (if fiscal year is different from calendar year)
    fiscal_year INTEGER NOT NULL,
    fiscal_quarter INTEGER NOT NULL,
    fiscal_month INTEGER NOT NULL,

    -- Period start/end indicators
    is_first_day_of_month BOOLEAN NOT NULL,
    is_last_day_of_month BOOLEAN NOT NULL,
    is_first_day_of_quarter BOOLEAN NOT NULL,
    is_last_day_of_quarter BOOLEAN NOT NULL,
    is_first_day_of_year BOOLEAN NOT NULL,
    is_last_day_of_year BOOLEAN NOT NULL,

    -- Holiday and business indicators
    is_weekend BOOLEAN NOT NULL,
    is_holiday BOOLEAN NOT NULL,
    holiday_name VARCHAR(50),
    is_business_day BOOLEAN NOT NULL,
```



```

-- Seasons and special periods
season VARCHAR(10), -- Spring, Summer, Fall, Winter

-- Special retail periods
is_black_friday BOOLEAN NOT NULL DEFAULT FALSE,
is_cyber_monday BOOLEAN NOT NULL DEFAULT FALSE,
is_holiday_season BOOLEAN NOT NULL DEFAULT FALSE -- Nov 15 - Dec 31
);

-- Populate the date dimension for 10 years (can be done with a script or procedure)
INSERT INTO date_dimension
SELECT
    TO_CHAR(dt, 'YYYYMMDD')::INTEGER AS date_key,
    dt AS date_value,
    EXTRACT(YEAR FROM dt) AS year,
    EXTRACT(QUARTER FROM dt) AS quarter,
    'Q' || EXTRACT(QUARTER FROM dt) AS quarter_name,
    EXTRACT(MONTH FROM dt) AS month_number,
    TO_CHAR(dt, 'Month') AS month_name,
    TO_CHAR(dt, 'Mon') AS month_name_short,
    EXTRACT(WEEK FROM dt) AS week_of_year,
    EXTRACT(DOY FROM dt) AS day_of_year,
    EXTRACT(DAY FROM dt) AS day_of_month,
    EXTRACT(ISODOW FROM dt) AS day_of_week,
    TO_CHAR(dt, 'Day') AS day_of_week_name,
    TO_CHAR(dt, 'Dy') AS day_of_week_name_short,

    -- Fiscal year (example assumes fiscal year starts in July)
    CASE
        WHEN EXTRACT(MONTH FROM dt) >= 7 THEN EXTRACT(YEAR FROM dt) + 1
        ELSE EXTRACT(YEAR FROM dt)
    END AS fiscal_year,

    -- Fiscal quarter
    CASE
        WHEN EXTRACT(MONTH FROM dt) BETWEEN 7 AND 9 THEN 1
        WHEN EXTRACT(MONTH FROM dt) BETWEEN 10 AND 12 THEN 2
        WHEN EXTRACT(MONTH FROM dt) BETWEEN 1 AND 3 THEN 3
        ELSE 4
    END AS fiscal_quarter,

    -- Fiscal month
    CASE
        WHEN EXTRACT(MONTH FROM dt) >= 7 THEN EXTRACT(MONTH FROM dt) - 6
        ELSE EXTRACT(MONTH FROM dt) + 6
    END AS fiscal_month

```

```

END AS fiscal_month,

-- First/last day indicators
EXTRACT(DAY FROM dt) = 1 AS is_first_day_of_month,
EXTRACT(DAY FROM dt) = EXTRACT(DAY FROM (DATE_TRUNC('MONTH', dt) + INTERVAL '1 MONTH - 1 DAY')) AS is_last_day_of_month,

EXTRACT(DAY FROM dt) = 1 AND EXTRACT(MONTH FROM dt) IN (1, 4, 7, 10) AS is_first_day_of_year,
EXTRACT(DAY FROM dt) = EXTRACT(DAY FROM (DATE_TRUNC('MONTH', dt) + INTERVAL '1 MONTH - 1 DAY')) AND EXTRACT(MONTH FROM dt) IN (3, 6, 9, 12) AS is_last_day_of_year,

EXTRACT(DOY FROM dt) = 1 AS is_first_day_of_year,
EXTRACT(MONTH FROM dt) = 12 AND
EXTRACT(DAY FROM dt) = 31 AS is_last_day_of_year,

-- Weekend indicator
EXTRACT(ISODOW FROM dt) >= 6 AS is_weekend,

-- Holiday indicators would be populated based on region-specific logic
FALSE AS is_holiday, -- Placeholder, needs more complex logic
NULL AS holiday_name, -- Placeholder

EXTRACT(ISODOW FROM dt) < 6 AS is_business_day, -- Simplified, should also exclude holidays

-- Season (Northern Hemisphere)
CASE
    WHEN EXTRACT(MONTH FROM dt) BETWEEN 3 AND 5 THEN 'Spring'
    WHEN EXTRACT(MONTH FROM dt) BETWEEN 6 AND 8 THEN 'Summer'
    WHEN EXTRACT(MONTH FROM dt) BETWEEN 9 AND 11 THEN 'Fall'
    ELSE 'Winter'
END AS season,

-- Retail periods
(EXTRACT(MONTH FROM dt) = 11 AND EXTRACT(DAY FROM dt) =
    (SELECT EXTRACT(DAY FROM (DATE_TRUNC('MONTH', DATE '2023-11-01') +
        ((EXTRACT(ISODOW FROM DATE_TRUNC('MONTH', DATE '2023-11-01') + INTERVAL '21 day')) + 1)
    )) AS is_black_friday,

(EXTRACT(MONTH FROM dt) = 11 AND EXTRACT(DAY FROM dt) =
    (SELECT EXTRACT(DAY FROM (DATE_TRUNC('MONTH', DATE '2023-11-01') +
        ((EXTRACT(ISODOW FROM DATE_TRUNC('MONTH', DATE '2023-11-01') + INTERVAL '21 day')) + 1)
    )) AS is_cyber_monday,

(EXTRACT(MONTH FROM dt) = 11 AND EXTRACT(DAY FROM dt) >= 15) OR
(EXTRACT(MONTH FROM dt) = 12) AS is_holiday_season
FROM
generate_series(

```

```

        '2020-01-01'::date,
        '2030-12-31'::date,
        '1 day'::interval
    ) AS dt;

```

Using the Date Dimension for Analysis

-- Year-over-year comparison by quarter

```

SELECT
    current_year.quarter_name,
    SUM(current_facts.sales_amount) AS current_year_sales,
    SUM(previous_facts.sales_amount) AS previous_year_sales,
    (SUM(current_facts.sales_amount) - SUM(previous_facts.sales_amount)) /
        NULLIF(SUM(previous_facts.sales_amount), 0) * 100 AS percent_change
FROM
    sales_fact current_facts
JOIN
    date_dimension current_year ON current_facts.date_key = current_year.date_key
JOIN
    date_dimension previous_year ON previous_year.date_value = current_year.date_value - INT
LEFT JOIN
    sales_fact previous_facts ON previous_facts.date_key = previous_year.date_key
WHERE
    current_year.year = 2023
GROUP BY
    current_year.quarter, current_year.quarter_name
ORDER BY
    current_year.quarter;

```

-- Sales by day of week

```

SELECT
    d.day_of_week_name,
    SUM(f.sales_amount) AS total_sales,
    COUNT(*) AS transaction_count,
    SUM(f.sales_amount) / COUNT(*) AS avg_transaction_value
FROM
    sales_fact f
JOIN
    date_dimension d ON f.date_key = d.date_key
WHERE
    d.year = 2023
GROUP BY
    d.day_of_week, d.day_of_week_name
ORDER BY
    d.day_of_week;

```

```

-- Holiday vs. non-holiday performance
SELECT
    CASE WHEN d.is_holiday THEN 'Holiday' ELSE 'Regular Day' END AS day_type,
    COUNT(DISTINCT d.date_value) AS number_of_days,
    SUM(f.sales_amount) AS total_sales,
    SUM(f.sales_amount) / COUNT(DISTINCT d.date_value) AS avg_daily_sales
FROM
    sales_fact f
JOIN
    date_dimension d ON f.date_key = d.date_key
WHERE
    d.year = 2023
GROUP BY
    CASE WHEN d.is_holiday THEN 'Holiday' ELSE 'Regular Day' END;

-- Business day vs. weekend performance by category
SELECT
    p.category,
    CASE WHEN d.is_weekend THEN 'Weekend' ELSE 'Business Day' END AS day_type,
    COUNT(DISTINCT d.date_value) AS number_of_days,
    SUM(f.sales_amount) AS total_sales,
    SUM(f.sales_amount) / COUNT(DISTINCT d.date_value) AS avg_daily_sales,
    COUNT(f.sale_id) AS transaction_count,
    COUNT(f.sale_id) / COUNT(DISTINCT d.date_value) AS avg_daily_transactions
FROM
    sales_fact f
JOIN
    date_dimension d ON f.date_key = d.date_key
JOIN
    product_dimension p ON f.product_key = p.product_key
WHERE
    d.year = 2023
GROUP BY
    p.category,
    CASE WHEN d.is_weekend THEN 'Weekend' ELSE 'Business Day' END
ORDER BY
    p.category, day_type;

```

Time-of-Day Dimension For more granular analysis of intraday patterns, a time-of-day dimension can be useful:

```

-- Create the time dimension table
CREATE TABLE time_dimension (
    time_key INTEGER PRIMARY KEY,
    time_value TIME UNIQUE NOT NULL,
    hour_24 INTEGER NOT NULL, -- 0-23

```

```

hour_12 INTEGER NOT NULL, -- 1-12
am_pm VARCHAR(2) NOT NULL, -- AM/PM
minute INTEGER NOT NULL, -- 0-59
second INTEGER NOT NULL, -- 0-59

-- Time periods
day_period VARCHAR(20) NOT NULL, -- Morning, Afternoon, Evening, Night
business_hour BOOLEAN NOT NULL, -- True if during business hours (e.g., 9am-5pm)
peak_hour BOOLEAN NOT NULL, -- True if during peak hours (defined by business)

-- Formatted values
time_format_24 VARCHAR(8) NOT NULL, -- HH:MM:SS
time_format_12 VARCHAR(11) NOT NULL -- HH:MM:SS AM/PM
);

-- Populate the time dimension for each minute of the day
INSERT INTO time_dimension
SELECT
    TO_CHAR(tm, 'HH24MI')::INTEGER AS time_key,
    tm::TIME AS time_value,
    EXTRACT(HOUR FROM tm) AS hour_24,
    CASE
        WHEN EXTRACT(HOUR FROM tm) % 12 = 0 THEN 12
        ELSE EXTRACT(HOUR FROM tm) % 12
    END AS hour_12,
    CASE
        WHEN EXTRACT(HOUR FROM tm) < 12 THEN 'AM'
        ELSE 'PM'
    END AS am_pm,
    EXTRACT(MINUTE FROM tm) AS minute,
    EXTRACT(SECOND FROM tm) AS second,

    CASE
        WHEN EXTRACT(HOUR FROM tm) BETWEEN 5 AND 11 THEN 'Morning'
        WHEN EXTRACT(HOUR FROM tm) BETWEEN 12 AND 16 THEN 'Afternoon'
        WHEN EXTRACT(HOUR FROM tm) BETWEEN 17 AND 20 THEN 'Evening'
        ELSE 'Night'
    END AS day_period,

    EXTRACT(HOUR FROM tm) BETWEEN 9 AND 17 AS business_hour,

    (EXTRACT(HOUR FROM tm) BETWEEN 11 AND 13) OR
    (EXTRACT(HOUR FROM tm) BETWEEN 17 AND 19) AS peak_hour,

    TO_CHAR(tm, 'HH24:MI:SS') AS time_format_24,
    TO_CHAR(tm, 'HH12:MI:SS AM') AS time_format_12

```

```

FROM
    generate_series(
        '2000-01-01 00:00:00'::timestamp,
        '2000-01-01 23:59:00'::timestamp,
        '1 minute'::interval
    ) AS tm;

```

Combining Date and Time Dimensions To analyze both date and time components together:

```

-- Creating a fact table with both date and time keys
CREATE TABLE transaction_fact (
    transaction_id INTEGER PRIMARY KEY,
    date_key INTEGER REFERENCES date_dimension(date_key),
    time_key INTEGER REFERENCES time_dimension(time_key),
    store_key INTEGER REFERENCES store_dimension(store_key),
    product_key INTEGER REFERENCES product_dimension(product_key),
    customer_key INTEGER REFERENCES customer_dimension(customer_key),
    quantity INTEGER,
    sales_amount NUMERIC(12,2)
);

-- Analyzing sales by time period throughout the week
SELECT
    d.day_of_week_name,
    t.day_period,
    COUNT(*) AS transaction_count,
    SUM(f.sales_amount) AS total_sales
FROM
    transaction_fact f
JOIN
    date_dimension d ON f.date_key = d.date_key
JOIN
    time_dimension t ON f.time_key = t.time_key
WHERE
    d.month_number = 3 -- March
    AND d.year = 2023
GROUP BY
    d.day_of_week, d.day_of_week_name, t.day_period
ORDER BY
    d.day_of_week,
    CASE
        WHEN t.day_period = 'Morning' THEN 1
        WHEN t.day_period = 'Afternoon' THEN 2
        WHEN t.day_period = 'Evening' THEN 3
        WHEN t.day_period = 'Night' THEN 4
    
```

```

END;

-- Identifying peak business hours by product category
SELECT
    p.category,
    t.hour_24,
    t.am_pm,
    COUNT(*) AS transaction_count,
    SUM(f.quantity) AS units_sold,
    SUM(f.sales_amount) AS total_sales
FROM
    transaction_fact f
JOIN
    product_dimension p ON f.product_key = p.product_key
JOIN
    time_dimension t ON f.time_key = t.time_key
JOIN
    date_dimension d ON f.date_key = d.date_key
WHERE
    d.is_business_day = TRUE
    AND d.year = 2023
GROUP BY
    p.category, t.hour_24, t.am_pm
ORDER BY
    p.category, total_sales DESC;

```

Holiday and Special Event Analysis

```

-- Impact of holidays on sales
SELECT
    d.holiday_name,
    SUM(f.sales_amount) AS total_sales,
    AVG(f.sales_amount) AS avg_transaction_value,
    COUNT(*) AS transaction_count
FROM
    sales_fact f
JOIN
    date_dimension d ON f.date_key = d.date_key
WHERE
    d.is_holiday = TRUE
    AND d.year = 2023
GROUP BY
    d.holiday_name
ORDER BY
    total_sales DESC;

```

```

-- Comparing Black Friday to regular Fridays
SELECT
    CASE
        WHEN d.is_black_friday THEN 'Black Friday'
        ELSE 'Regular Friday'
    END AS friday_type,
    COUNT(DISTINCT d.date_key) AS day_count,
    SUM(f.sales_amount) AS total_sales,
    SUM(f.sales_amount) / COUNT(DISTINCT d.date_key) AS avg_daily_sales,
    COUNT(*) / COUNT(DISTINCT d.date_key) AS avg_daily_transactions
FROM
    sales_fact f
JOIN
    date_dimension d ON f.date_key = d.date_key
WHERE
    d.day_of_week_name = 'Friday'
    AND d.year = 2023
GROUP BY
    CASE WHEN d.is_black_friday THEN 'Black Friday' ELSE 'Regular Friday' END;

-- Holiday season vs. non-holiday season performance by category
SELECT
    p.category,
    CASE WHEN d.is_holiday_season THEN 'Holiday Season' ELSE 'Regular Season' END AS season,
    COUNT(DISTINCT d.date_key) AS number_of_days,
    SUM(f.sales_amount) AS total_sales,
    SUM(f.sales_amount) / COUNT(DISTINCT d.date_key) AS avg_daily_sales
FROM
    sales_fact f
JOIN
    date_dimension d ON f.date_key = d.date_key
JOIN
    product_dimension p ON f.product_key = p.product_key
WHERE
    d.year = 2023
GROUP BY
    p.category,
    CASE WHEN d.is_holiday_season THEN 'Holiday Season' ELSE 'Regular Season' END
ORDER BY
    p.category,
    CASE WHEN season_type = 'Holiday Season' THEN 1 ELSE 2 END;

```


Part IV: Performance Optimization

1. Indexes and Query Plans

Optimizing query performance is critical for analytics, especially when dealing with large datasets. Understanding how to interpret execution plans and use indexes effectively is an essential skill.

Index Types for Analytics While transaction-oriented databases primarily use B-tree indexes, analytical databases employ a wider variety of index types optimized for different query patterns:

```
-- B-tree index (standard index type)
-- Good for: equality, range queries, and sorting
CREATE INDEX idx_product_category
ON product_dimension(category);

-- Bitmap index
-- Good for: low cardinality columns (columns with few unique values)
-- Note: Explicitly available in some databases like Oracle and PostgreSQL
CREATE INDEX idx_product_category_bitmap
ON product_dimension USING BITMAP(category);

-- Covering index (includes additional columns)
-- Good for: avoiding table lookups by including all needed columns
CREATE INDEX idx_sales_by_date_product
ON sales_fact(date_key, product_key)
INCLUDE (sales_amount);

-- Partial/filtered index (only indexes a subset of rows)
-- Good for: queries that always filter on the same condition
CREATE INDEX idx_high_value_sales
ON sales_fact(date_key, customer_key)
WHERE sales_amount > 1000;

-- Expression index (indexes a computed value)
-- Good for: queries that use functions or expressions
CREATE INDEX idx_sale_month
ON sales_fact(EXTRACT(MONTH FROM transaction_date));

-- GIN (Generalized Inverted Index)
-- Good for: full-text search and array operations
CREATE INDEX idx_product_description_search
ON product_dimension USING GIN (to_tsvector('english', product_description));
```

Understanding EXPLAIN Output The EXPLAIN command shows how the database will execute your query. For analytics, understanding these plans is

crucial for optimizing long-running queries:

```
-- Basic EXPLAIN
EXPLAIN
SELECT
    p.category,
    COUNT(*) as sale_count,
    SUM(s.sales_amount) as total_sales
FROM
    sales_fact s
JOIN
    product_dimension p ON s.product_key = p.product_key
WHERE
    s.date_key BETWEEN 20230101 AND 20230131
GROUP BY
    p.category;

-- Sample output:
/*
HashAggregate (cost=1234.56..1235.67 rows=89 width=24)
  Group Key: p.category
    -> Hash Join (cost=87.65..1122.33 rows=10000 width=16)
        Hash Cond: (s.product_key = p.product_key)
        -> Seq Scan on sales_fact s (cost=0.00..877.00 rows=10000 width=12)
            Filter: ((date_key >= 20230101) AND (date_key <= 20230131))
        -> Hash (cost=54.00..54.00 rows=2692 width=12)
            -> Seq Scan on product_dimension p (cost=0.00..54.00 rows=2692 width=12)
*/

-- EXPLAIN with execution statistics
EXPLAIN ANALYZE
SELECT
    p.category,
    COUNT(*) as sale_count,
    SUM(s.sales_amount) as total_sales
FROM
    sales_fact s
JOIN
    product_dimension p ON s.product_key = p.product_key
WHERE
    s.date_key BETWEEN 20230101 AND 20230131
GROUP BY
    p.category;

-- Sample output with runtime information:
/*
```

```

HashAggregate (cost=1234.56..1235.67 rows=89 width=24) (actual time=22.548..22.592 rows=15)
  Group Key: p.category
  -> Hash Join (cost=87.65..1122.33 rows=10000 width=16) (actual time=0.978..19.462 rows=10000)
    Hash Cond: (s.product_key = p.product_key)
    -> Seq Scan on sales_fact s (cost=0.00..877.00 rows=10000 width=12) (actual time=0.000..19.458 rows=10000)
      Filter: ((date_key >= 20230101) AND (date_key <= 20230131))
      Rows Removed by Filter: 23432
    -> Hash (cost=54.00..54.00 rows=2692 width=12) (actual time=0.909..0.909 rows=2692)
      Buckets: 4096 Batches: 1 Memory Usage: 175kB
      -> Seq Scan on product_dimension p (cost=0.00..54.00 rows=2692 width=12) (actual time=0.000..0.909 rows=2692)
Planning Time: 0.654 ms
Execution Time: 22.744 ms
*/

```

Common Plan Operations and What They Mean Understanding the operations in a query plan helps identify optimization opportunities:

```

-- Sequential Scan: Full table scan
EXPLAIN
SELECT * FROM large_table WHERE column1 > 1000;
-- Output contains: "Seq Scan on large_table"

-- Index Scan: Uses an index to find rows, then fetches data
EXPLAIN
SELECT * FROM large_table WHERE indexed_column = 'value';
-- Output might contain: "Index Scan using idx_column on large_table"

-- Index Only Scan: Gets all data directly from the index
EXPLAIN
SELECT indexed_column FROM large_table WHERE indexed_column = 'value';
-- Output might contain: "Index Only Scan using idx_column on large_table"

-- Bitmap Index Scan: Uses a bitmap to find matching rows
EXPLAIN
SELECT * FROM large_table WHERE category IN ('A', 'B', 'C');
-- Output might contain: "Bitmap Index Scan on idx_category"

-- Hash Join: Build a hash table from one side, probe with the other
EXPLAIN
SELECT * FROM table1 JOIN table2 ON table1.id = table2.id;
-- Output might contain: "Hash Join" with "Hash" and "Hash Cond"

-- Merge Join: Merge two sorted inputs
EXPLAIN
SELECT * FROM table1 JOIN table2 ON table1.id = table2.id
ORDER BY table1.id;

```

```

-- Output might contain: "Merge Join" with "Sort" operations

-- Nested Loop: For each row from one table, scan the other table
EXPLAIN
SELECT * FROM small_table s JOIN large_table l ON s.id = l.id;
-- Output might contain: "Nested Loop"

-- Aggregate: Group and calculate aggregate functions
EXPLAIN
SELECT category, SUM(amount) FROM table GROUP BY category;
-- Output might contain: "HashAggregate" or "GroupAggregate"

-- Sort: Sort the result set
EXPLAIN
SELECT * FROM table ORDER BY column;
-- Output might contain: "Sort"

-- Limit: Restrict the number of returned rows
EXPLAIN
SELECT * FROM table LIMIT 10;
-- Output might contain: "Limit"

```

Comparing Different Query Plans Let's examine how the same analytical query can have different execution plans:

```

-- Query: Find total sales by category for January 2023

-- Version 1: Simple Query
EXPLAIN ANALYZE
SELECT
    p.category,
    SUM(s.sales_amount) as total_sales
FROM
    sales_fact s
JOIN
    product_dimension p ON s.product_key = p.product_key
WHERE
    s.date_key BETWEEN 20230101 AND 20230131
GROUP BY
    p.category;

-- Sample output:
/*
HashAggregate (cost=1234.56..1235.67 rows=15 width=24) (actual time=22.548..22.592 rows=15)
  Group Key: p.category
  -> Hash Join (cost=87.65..1122.33 rows=11204 width=16) (actual time=0.978..19.462 rows=)

```

```

Hash Cond: (s.product_key = p.product_key)
-> Seq Scan on sales_fact s (cost=0.00..877.00 rows=11204 width=12) (actual time=0.909..0.909 rows=2692)
    Filter: ((date_key >= 20230101) AND (date_key <= 20230131))
    Rows Removed by Filter: 23432
-> Hash (cost=54.00..54.00 rows=2692 width=12) (actual time=0.909..0.909 rows=2692)
    Buckets: 4096 Batches: 1 Memory Usage: 175kB
    -> Seq Scan on product_dimension p (cost=0.00..54.00 rows=2692 width=12) (actual time=0.000..0.000 rows=2692)
Planning Time: 0.654 ms
Execution Time: 22.744 ms
*/

-- Version 2: Pre-aggregation in a CTE
EXPLAIN ANALYZE
WITH date_filtered_sales AS (
    SELECT
        product_key,
        SUM(sales_amount) as total_sales
    FROM
        sales_fact
    WHERE
        date_key BETWEEN 20230101 AND 20230131
    GROUP BY
        product_key
)
SELECT
    p.category,
    SUM(s.total_sales) as total_sales
FROM
    date_filtered_sales s
JOIN
    product_dimension p ON s.product_key = p.product_key
GROUP BY
    p.category;

-- Sample output:
/*
HashAggregate (cost=923.45..924.56 rows=15 width=24) (actual time=19.332..19.376 rows=15 loops=1)
    Group Key: p.category
    -> Hash Join (cost=787.65..899.33 rows=1204 width=16) (actual time=10.978..17.462 rows=15 loops=1)
        Hash Cond: (s.product_key = p.product_key)
        -> HashAggregate (cost=700.00..725.00 rows=1204 width=12) (actual time=10.014..12.014 rows=15 loops=1)
            Group Key: sales_fact.product_key
            -> Seq Scan on sales_fact (cost=0.00..600.00 rows=11204 width=12) (actual time=0.000..0.000 rows=11204)
                Filter: ((date_key >= 20230101) AND (date_key <= 20230131))
                Rows Removed by Filter: 23432
            -> Hash (cost=54.00..54.00 rows=2692 width=12) (actual time=0.909..0.909 rows=2692)

```

```

        Buckets: 4096 Batches: 1 Memory Usage: 175kB
        -> Seq Scan on product_dimension p (cost=0.00..54.00 rows=2692 width=12) (a
Planning Time: 0.712 ms
Execution Time: 19.513 ms
*/

-- Version 3: Using a pre-created index
-- Assume we created: CREATE INDEX idx_sales_date_product ON sales_fact(date_key, product_k
EXPLAIN ANALYZE
SELECT
    p.category,
    SUM(s.sales_amount) as total_sales
FROM
    sales_fact s
JOIN
    product_dimension p ON s.product_key = p.product_key
WHERE
    s.date_key BETWEEN 20230101 AND 20230131
GROUP BY
    p.category;

-- Sample output with index:
/*
HashAggregate (cost=723.45..724.56 rows=15 width=24) (actual time=15.548..15.592 rows=15 l
Group Key: p.category
-> Hash Join (cost=87.65..622.33 rows=11204 width=16) (actual time=0.978..12.462 rows=1
Hash Cond: (s.product_key = p.product_key)
-> Index Scan using idx_sales_date_product on sales_fact s (cost=0.00..377.00 rows=
Index Cond: ((date_key >= 20230101) AND (date_key <= 20230131))
-> Hash (cost=54.00..54.00 rows=2692 width=12) (actual time=0.909..0.909 rows=2692
Buckets: 4096 Batches: 1 Memory Usage: 175kB
-> Seq Scan on product_dimension p (cost=0.00..54.00 rows=2692 width=12) (a
Planning Time: 0.654 ms
Execution Time: 15.744 ms
*/

```

Analyzing Join Order Impact One of the most important aspects of query optimization is join order. Let's examine how it affects performance:

```

-- Query: Sales by store, product, and date

-- Version 1: Optimizer-determined join order
EXPLAIN
SELECT
    s.store_name,
    p.product_name,

```

```

        d.month_name,
        SUM(f.sales_amount) as total_sales
FROM
    sales_fact f
JOIN
    date_dimension d ON f.date_key = d.date_key
JOIN
    product_dimension p ON f.product_key = p.product_key
JOIN
    store_dimension s ON f.store_key = s.store_key
WHERE
    d.year = 2023
    AND s.region = 'Northeast'
    AND p.category = 'Electronics'
GROUP BY
    s.store_name, p.product_name, d.month_name;

-- Version 2: Explicitly ordered joins (start with the most filtered table)
EXPLAIN
SELECT
    s.store_name,
    p.product_name,
    d.month_name,
    SUM(f.sales_amount) as total_sales
FROM
    store_dimension s
JOIN
    sales_fact f ON s.store_key = f.store_key
JOIN
    product_dimension p ON f.product_key = p.product_key
JOIN
    date_dimension d ON f.date_key = d.date_key
WHERE
    d.year = 2023
    AND s.region = 'Northeast'
    AND p.category = 'Electronics'
GROUP BY
    s.store_name, p.product_name, d.month_name;

```

Examining the Impact of WHERE Clause Order The order of conditions in a WHERE clause can impact the execution plan:

```

-- Query: Find high-value electronics sales in 2023

-- Version 1: Date filter first
EXPLAIN ANALYZE

```

```

SELECT
    p.product_name,
    SUM(f.sales_amount) as total_sales
FROM
    sales_fact f
JOIN
    product_dimension p ON f.product_key = p.product_key
JOIN
    date_dimension d ON f.date_key = d.date_key
WHERE
    d.year = 2023
    AND p.category = 'Electronics'
    AND f.sales_amount > 1000
GROUP BY
    p.product_name;

```

-- Version 2: Category filter first

```

EXPLAIN ANALYZE
SELECT
    p.product_name,
    SUM(f.sales_amount) as total_sales
FROM
    sales_fact f
JOIN
    product_dimension p ON f.product_key = p.product_key
JOIN
    date_dimension d ON f.date_key = d.date_key
WHERE
    p.category = 'Electronics'
    AND d.year = 2023
    AND f.sales_amount > 1000
GROUP BY
    p.product_name;

```

Understanding Cardinality and Selectivity For analytics workloads, knowing how many rows your queries will process is crucial:

-- Examine the actual vs. estimated row counts

```

EXPLAIN ANALYZE
SELECT
    p.category,
    COUNT(*) as product_count
FROM
    product_dimension p
WHERE
    p.retail_price > 100

```



```

GROUP BY
    p.category;

-- Collect statistics on the table to improve estimates
ANALYZE product_dimension;

-- Re-run the explain to see updated estimates
EXPLAIN ANALYZE
SELECT
    p.category,
    COUNT(*) as product_count
FROM
    product_dimension p
WHERE
    p.retail_price > 100
GROUP BY
    p.category;

```

2. Query Optimization Techniques

Beyond understanding execution plans, there are specific techniques to optimize analytical queries.

Filtering Early Reducing the amount of data processed as early as possible is one of the most effective optimization strategies:

```

-- Inefficient: Filtering after the join
EXPLAIN
SELECT
    c.customer_name,
    SUM(s.sales_amount) as total_sales
FROM
    sales_fact s
JOIN
    customer_dimension c ON s.customer_key = c.customer_key
WHERE
    s.date_key BETWEEN 20230101 AND 20230131
    AND c.customer_segment = 'Enterprise'
GROUP BY
    c.customer_name;

-- More efficient: Pre-filtering the tables before joining
EXPLAIN
WITH filtered_sales AS (
    SELECT
        customer_key,

```

```

        sales_amount
    FROM
        sales_fact
    WHERE
        date_key BETWEEN 20230101 AND 20230131
),
filtered_customers AS (
    SELECT
        customer_key,
        customer_name
    FROM
        customer_dimension
    WHERE
        customer_segment = 'Enterprise'
)
SELECT
    c.customer_name,
    SUM(s.sales_amount) as total_sales
FROM
    filtered_sales s
JOIN
    filtered_customers c ON s.customer_key = c.customer_key
GROUP BY
    c.customer_name;

```

Aggregating Early For analytical queries, reducing data volume through early aggregation can significantly improve performance:

-- Inefficient: Joining first, then aggregating

```

EXPLAIN
SELECT
    d.year,
    d.quarter,
    p.category,
    SUM(s.sales_amount) as total_sales
FROM
    sales_fact s
JOIN
    date_dimension d ON s.date_key = d.date_key
JOIN
    product_dimension p ON s.product_key = p.product_key
WHERE
    d.year BETWEEN 2020 AND 2023
GROUP BY
    d.year, d.quarter, p.category;

```

```

-- More efficient: Aggregate before joining when possible
EXPLAIN
WITH sales_by_date_product AS (
    SELECT
        date_key,
        product_key,
        SUM(sales_amount) as total_sales
    FROM
        sales_fact
    GROUP BY
        date_key, product_key
)
SELECT
    d.year,
    d.quarter,
    p.category,
    SUM(s.total_sales) as total_sales
FROM
    sales_by_date_product s
JOIN
    date_dimension d ON s.date_key = d.date_key
JOIN
    product_dimension p ON s.product_key = p.product_key
WHERE
    d.year BETWEEN 2020 AND 2023
GROUP BY
    d.year, d.quarter, p.category;

```

Materialized Views for Analytics Materialized views pre-compute and store the results of a query for faster access:

```

-- Create a materialized view for monthly sales by product category
CREATE MATERIALIZED VIEW monthly_category_sales AS
SELECT
    d.year,
    d.month_number,
    p.category,
    SUM(s.sales_amount) as total_sales,
    COUNT(*) as transaction_count
FROM
    sales_fact s
JOIN
    date_dimension d ON s.date_key = d.date_key
JOIN
    product_dimension p ON s.product_key = p.product_key
GROUP BY

```

```

        d.year, d.month_number, p.category;

-- Create an index on the materialized view for faster queries
CREATE INDEX idx_mv_monthly_sales
ON monthly_category_sales(year, month_number, category);

-- Use the materialized view
EXPLAIN
SELECT
    year,
    month_number,
    category,
    total_sales
FROM
    monthly_category_sales
WHERE
    year = 2023
    AND month_number BETWEEN 1 AND 3
ORDER BY
    month_number, total_sales DESC;

-- Refresh the materialized view when source data changes
REFRESH MATERIALIZED VIEW monthly_category_sales;

```

Query Rewriting Techniques Sometimes, restructuring a query can dramatically improve performance:

```

-- Original query with multiple joins and subqueries
EXPLAIN
SELECT
    c.customer_name,
    SUM(s.sales_amount) as total_sales,
    (
        SELECT AVG(sales_amount)
        FROM sales_fact
        WHERE date_key BETWEEN 20230101 AND 20230131
    ) as average_sale
FROM
    sales_fact s
JOIN
    customer_dimension c ON s.customer_key = c.customer_key
WHERE
    s.date_key BETWEEN 20230101 AND 20230131
    AND s.sales_amount > (
        SELECT AVG(sales_amount) * 2
        FROM sales_fact
    )

```

```

        WHERE date_key BETWEEN 20230101 AND 20230131
    )
GROUP BY
    c.customer_name;

-- Rewritten query with CTEs to avoid repeated subqueries
EXPLAIN
WITH period_stats AS (
    SELECT
        AVG(sales_amount) as avg_sale,
        AVG(sales_amount) * 2 as high_value_threshold
    FROM
        sales_fact
    WHERE
        date_key BETWEEN 20230101 AND 20230131
)
SELECT
    c.customer_name,
    SUM(s.sales_amount) as total_sales,
    ps.avg_sale as average_sale
FROM
    sales_fact s
JOIN
    customer_dimension c ON s.customer_key = c.customer_key
CROSS JOIN
    period_stats ps
WHERE
    s.date_key BETWEEN 20230101 AND 20230131
    AND s.sales_amount > ps.high_value_threshold
GROUP BY
    c.customer_name, ps.avg_sale;

```

Using Window Functions Instead of Self-Joins Window functions can often replace complex self-joins for analytical calculations:

```

-- Inefficient: Self-join to calculate year-over-year growth
EXPLAIN
SELECT
    current.year,
    current.month_number,
    current.category,
    current.total_sales,
    previous.total_sales as prev_year_sales,
    (current.total_sales - previous.total_sales) / previous.total_sales * 100 as yoy_growth
FROM
    monthly_category_sales current

```

```

LEFT JOIN
    monthly_category_sales previous
    ON current.month_number = previous.month_number
    AND current.category = previous.category
    AND previous.year = current.year - 1
WHERE
    current.year = 2023;

-- More efficient: Using window functions
EXPLAIN
SELECT
    year,
    month_number,
    category,
    total_sales,
    LAG(total_sales, 12) OVER (
        PARTITION BY category, month_number
        ORDER BY year, month_number
    ) as prev_year_sales,
    CASE
        WHEN LAG(total_sales, 12) OVER (
            PARTITION BY category, month_number
            ORDER BY year, month_number
        ) IS NULL OR LAG(total_sales, 12) OVER (
            PARTITION BY category, month_number
            ORDER BY year, month_number
        ) = 0
        THEN NULL
        ELSE (total_sales - LAG(total_sales, 12) OVER (
            PARTITION BY category, month_number
            ORDER BY year, month_number
        )) / LAG(total_sales, 12) OVER (
            PARTITION BY category, month_number
            ORDER BY year, month_number
        ) * 100
    END as yoy_growth
FROM
    monthly_category_sales
WHERE
    year = 2023;

```

Using EXISTS Instead of IN for Subquery Filtering For large datasets, EXISTS can perform better than IN with subqueries:

```

-- Using IN with a subquery
EXPLAIN

```

```

SELECT
    p.product_id,
    p.product_name,
    p.category
FROM
    product_dimension p
WHERE
    p.product_id IN (
        SELECT DISTINCT product_id
        FROM sales_fact
        WHERE date_key BETWEEN 20230101 AND 20230131
    );

-- Using EXISTS instead
EXPLAIN
SELECT
    p.product_id,
    p.product_name,
    p.category
FROM
    product_dimension p
WHERE
    EXISTS (
        SELECT 1
        FROM sales_fact s
        WHERE s.product_id = p.product_id
        AND s.date_key BETWEEN 20230101 AND 20230131
    );

```

Query Hints Some databases allow query hints to override the optimizer's decisions. Use these sparingly and only when necessary:

```

-- PostgreSQL: Force a specific join order
SELECT /*+ Leading(dimension1 fact dimension2) */
    d1.attribute,
    d2.attribute,
    SUM(f.measure)
FROM
    dimension1 d1
JOIN
    fact f ON d1.key = f.dimension1_key
JOIN
    dimension2 d2 ON f.dimension2_key = d2.key
GROUP BY
    d1.attribute, d2.attribute;

```

```

-- PostgreSQL: Disable specific join types
SELECT /*+ NoHashJoin(f d1) NoMergeJoin(f d2) */
       d1.attribute,
       d2.attribute,
       SUM(f.measure)
FROM
       fact f
JOIN
       dimension1 d1 ON f.dimension1_key = d1.key
JOIN
       dimension2 d2 ON f.dimension2_key = d2.key
GROUP BY
       d1.attribute, d2.attribute;

```

3. Identifying and Resolving Bottlenecks

Even with well-designed queries, performance issues can arise. Knowing how to identify and resolve bottlenecks is essential.

Identifying Slow Queries First, you need to find problematic queries:

```

-- PostgreSQL: Find slow queries from logs
SELECT
       substring(query, 1, 100) as query_snippet,
       calls,
       total_time,
       mean_time,
       max_time
FROM
       pg_stat_statements
ORDER BY
       total_time DESC
LIMIT 20;

-- PostgreSQL: Find currently running long queries
SELECT
       pid,
       now() - query_start as duration,
       query
FROM
       pg_stat_activity
WHERE
       state = 'active'
       AND now() - query_start > interval '5 minutes'
ORDER BY
       duration DESC;

```


Common Bottlenecks in Analytical Queries Let's examine some common issues and solutions:

1. Inefficient Joins

-- Problem: Cartesian join (missing join condition)

```
EXPLAIN
SELECT
    c.customer_name,
    p.product_name,
    SUM(s.sales_amount) as total_sales
FROM
    sales_fact s,
    customer_dimension c,
    product_dimension p
WHERE
    s.customer_key = c.customer_key
    -- Missing join condition for product_dimension
GROUP BY
    c.customer_name, p.product_name;
```

-- Solution: Add proper join conditions

```
EXPLAIN
SELECT
    c.customer_name,
    p.product_name,
    SUM(s.sales_amount) as total_sales
FROM
    sales_fact s
JOIN
    customer_dimension c ON s.customer_key = c.customer_key
JOIN
    product_dimension p ON s.product_key = p.product_key
GROUP BY
    c.customer_name, p.product_name;
```

2. Too Many Joins

-- Problem: Excessive joins for the required analysis

```
EXPLAIN
SELECT
    d.year,
    d.quarter,
    SUM(s.sales_amount) as total_sales
FROM
    sales_fact s
JOIN
    date_dimension d ON s.date_key = d.date_key
```

```

JOIN
    product_dimension p ON s.product_key = p.product_key
JOIN
    customer_dimension c ON s.customer_key = c.customer_key
JOIN
    store_dimension st ON s.store_key = st.store_key
JOIN
    promotion_dimension pr ON s.promotion_key = pr.promotion_key
WHERE
    d.year = 2023
GROUP BY
    d.year, d.quarter;

```

-- Solution: Only join the necessary tables

```

EXPLAIN
SELECT
    d.year,
    d.quarter,
    SUM(s.sales_amount) as total_sales
FROM
    sales_fact s
JOIN
    date_dimension d ON s.date_key = d.date_key
WHERE
    d.year = 2023
GROUP BY
    d.year, d.quarter;

```

3. Inefficient GROUP BY Operations

-- Problem: Grouping by high-cardinality columns

```

EXPLAIN
SELECT
    p.product_id, -- High cardinality
    SUM(s.sales_amount) as total_sales
FROM
    sales_fact s
JOIN
    product_dimension p ON s.product_key = p.product_key
GROUP BY
    p.product_id;

```

-- Solution: Group by appropriate business level

```

EXPLAIN
SELECT
    p.category, -- Lower cardinality
    SUM(s.sales_amount) as total_sales

```

```

FROM
    sales_fact s
JOIN
    product_dimension p ON s.product_key = p.product_key
GROUP BY
    p.category;

```

4. Suboptimal Function Usage

-- Problem: Functions in WHERE clause prevent index usage

```

EXPLAIN
SELECT
    customer_key,
    sales_amount
FROM
    sales_fact
WHERE
    EXTRACT(YEAR FROM transaction_date) = 2023;

```

-- Solution: Rewrite to allow index usage

```

EXPLAIN
SELECT
    customer_key,
    sales_amount
FROM
    sales_fact
WHERE
    transaction_date >= '2023-01-01' AND
    transaction_date < '2024-01-01';

```

-- Alternatively, use a function-based index

```

CREATE INDEX idx_sales_year
ON sales_fact(EXTRACT(YEAR FROM transaction_date));

```

5. Insufficient Memory for Operations

-- Problem: Large sort or hash operations might exceed work_mem

```

EXPLAIN
SELECT
    c.customer_id,
    p.product_id,
    SUM(s.sales_amount) as total_sales
FROM
    sales_fact s
JOIN
    customer_dimension c ON s.customer_key = c.customer_key
JOIN
    product_dimension p ON s.product_key = p.product_key

```

```

GROUP BY
    c.customer_id, p.product_id;

-- Solution: Break into smaller queries or increase work_mem temporarily
SET work_mem = '256MB'; -- Adjust based on your server's capacity

-- Alternative solution: Use CTEs to break down the operations
WITH customer_sales AS (
    SELECT
        customer_key,
        product_key,
        SUM(sales_amount) as total_sales
    FROM
        sales_fact
    GROUP BY
        customer_key, product_key
)
SELECT
    c.customer_id,
    p.product_id,
    cs.total_sales
FROM
    customer_sales cs
JOIN
    customer_dimension c ON cs.customer_key = c.customer_key
JOIN
    product_dimension p ON cs.product_key = p.product_key;

```

Using EXPLAIN to Identify Issues Let's see how to identify specific problems using EXPLAIN output:

```

-- Identifying a sequential scan when an index should be used
EXPLAIN
SELECT * FROM large_table WHERE indexed_column = 'value';
-- Look for "Seq Scan" instead of "Index Scan"

-- Identifying a hash join when a nested loop would be better for small datasets
EXPLAIN
SELECT * FROM small_table s JOIN large_table l ON s.id = l.id;
-- Look for "Hash Join" instead of "Nested Loop"

-- Identifying poor row count estimates
EXPLAIN
SELECT * FROM table WHERE column1 = 'rare_value';
-- Compare estimated vs. actual rows after running EXPLAIN ANALYZE

```

```

-- Identifying a sort operation when using an index could avoid it
EXPLAIN
SELECT * FROM table ORDER BY column1;
-- Look for "Sort" operation instead of "Index Scan using idx_column1"

```

Performance Tuning Solutions 1. Creating Appropriate Indexes

```

-- For common equality filters
CREATE INDEX idx_product_category ON product_dimension(category);

-- For range queries
CREATE INDEX idx_sales_date ON sales_fact(transaction_date);

-- For common joins
CREATE INDEX idx_sales_product_key ON sales_fact(product_key);

-- Multi-column indexes for compound conditions
CREATE INDEX idx_sales_date_product
ON sales_fact(date_key, product_key);

-- Covering indexes for frequent analytical queries
CREATE INDEX idx_sales_analysis
ON sales_fact(date_key, product_key)
INCLUDE (sales_amount);

-- Indexes for GROUP BY columns
CREATE INDEX idx_sales_store_date
ON sales_fact(store_key, date_key);

```

2. Partitioning Tables

Partitioning large fact tables can significantly improve performance:

```

-- Create a partitioned table by date
CREATE TABLE sales_fact (
    sale_id INTEGER,
    date_key INTEGER,
    product_key INTEGER,
    customer_key INTEGER,
    sales_amount NUMERIC(12,2),
    -- other columns
    PRIMARY KEY (sale_id, date_key)
) PARTITION BY RANGE (date_key);

-- Create partitions for each year
CREATE TABLE sales_fact_2021
PARTITION OF sales_fact

```

```

FOR VALUES FROM (20210101) TO (20220101);

CREATE TABLE sales_fact_2022
PARTITION OF sales_fact
FOR VALUES FROM (20220101) TO (20230101);

CREATE TABLE sales_fact_2023
PARTITION OF sales_fact
FOR VALUES FROM (20230101) TO (20240101);

-- Create indexes on each partition
CREATE INDEX idx_sales_2023_product
ON sales_fact_2023(product_key);

-- Query will automatically use the relevant partition
EXPLAIN
SELECT
    p.category,
    SUM(s.sales_amount) as total_sales
FROM
    sales_fact s
JOIN
    product_dimension p ON s.product_key = p.product_key
WHERE
    s.date_key BETWEEN 20230101 AND 20230331
GROUP BY
    p.category;

```

3. Materialized Views and Summary Tables

Pre-aggregating common analytical queries:

```

-- Create a summary table for daily sales by product category
CREATE TABLE daily_category_sales AS
SELECT
    s.date_key,
    p.category,
    SUM(s.sales_amount) as total_sales,
    COUNT(*) as transaction_count
FROM
    sales_fact s
JOIN
    product_dimension p ON s.product_key = p.product_key
GROUP BY
    s.date_key, p.category;

-- Create indexes on the summary table

```

```

CREATE INDEX idx_daily_category_sales_date
ON daily_category_sales(date_key);

CREATE INDEX idx_daily_category_sales_category
ON daily_category_sales(category);

-- Query the summary table instead of base tables
SELECT
    d.month_name,
    dcs.category,
    SUM(dcs.total_sales) as monthly_sales
FROM
    daily_category_sales dcs
JOIN
    date_dimension d ON dcs.date_key = d.date_key
WHERE
    d.year = 2023
GROUP BY
    d.month_name, dcs.category
ORDER BY
    d.month_name, monthly_sales DESC;

```

4. Query Restructuring

Sometimes, rewriting the query logic can improve performance:

```

-- Original query with complex joins and filtering
EXPLAIN
SELECT
    c.customer_segment,
    p.category,
    d.quarter,
    SUM(f.sales_amount) as total_sales
FROM
    sales_fact f
JOIN
    customer_dimension c ON f.customer_key = c.customer_key
JOIN
    product_dimension p ON f.product_key = p.product_key
JOIN
    date_dimension d ON f.date_key = d.date_key
WHERE
    d.year = 2023
    AND c.customer_segment IN ('Enterprise', 'SMB')
    AND p.category IN ('Electronics', 'Furniture')
GROUP BY
    c.customer_segment, p.category, d.quarter;

```

```

-- Restructured query with pre-filtering and explicit join order
EXPLAIN
WITH filtered_dates AS (
    SELECT date_key, quarter
    FROM date_dimension
    WHERE year = 2023
),
filtered_customers AS (
    SELECT customer_key, customer_segment
    FROM customer_dimension
    WHERE customer_segment IN ('Enterprise', 'SMB')
),
filtered_products AS (
    SELECT product_key, category
    FROM product_dimension
    WHERE category IN ('Electronics', 'Furniture')
)
SELECT
    c.customer_segment,
    p.category,
    d.quarter,
    SUM(f.sales_amount) as total_sales
FROM
    filtered_customers c
JOIN
    sales_fact f ON c.customer_key = f.customer_key
JOIN
    filtered_products p ON f.product_key = p.product_key
JOIN
    filtered_dates d ON f.date_key = d.date_key
GROUP BY
    c.customer_segment, p.category, d.quarter;

```

Part V: Distributed SQL and Large-Scale Analytics

1. Columnar Storage Principles

Modern data warehouses and analytics platforms use columnar storage rather than the row-based storage found in traditional transactional databases. Understanding these principles is essential for optimizing large-scale analytics.

Row-Based vs. Columnar Storage

- Conceptual comparison (not executable SQL)
- Row-based storage (traditional OLTP databases)


```

-- Stores all columns of a row together
TABLE sales_fact (
    /* Physically stored as:
    [sale_id=1, date_key=20230101, product_key=101, sales_amount=125.50],
    [sale_id=2, date_key=20230101, product_key=205, sales_amount=89.99],
    ...
    */
);

-- Columnar storage (analytical databases)
-- Stores all values of a column together
TABLE sales_fact (
    /* Physically stored as:
    sale_id:      [1, 2, 3, 4, 5, ...],
    date_key:     [20230101, 20230101, 20230101, 20230102, 20230102, ...],
    product_key:  [101, 205, 308, 101, 205, ...],
    sales_amount: [125.50, 89.99, 45.25, 130.00, 92.50, ...]
    */
);

```

Benefits of Columnar Storage for Analytics

1. **Data Compression:** Values in the same column often have similar characteristics, allowing for better compression.
2. **I/O Efficiency:** Analytics queries often only need a subset of columns, so you only read what you need.
3. **Vectorized Processing:** Operations can be performed on entire columns at once.
4. **Late Materialization:** Join and filter operations can be pushed down before reconstructing rows.

Columnar Storage Platforms Several major analytical platforms use columnar storage:

- **Amazon Redshift:** Based on PostgreSQL with columnar storage optimizations
- **Snowflake:** Cloud-based data warehouse with columnar storage
- **Google BigQuery:** Serverless columnar data warehouse
- **Apache Parquet:** Columnar file format for data lakes
- **Apache ORC:** Optimized Row Columnar format for Hadoop

Writing SQL for Columnar Systems While the SQL syntax is generally the same, some considerations can help optimize for columnar storage:

```

-- Good practices for columnar databases

-- 1. Select only needed columns (takes advantage of columnar storage)

```

```

-- Efficient for columnar storage
SELECT
    date_key,
    product_key,
    SUM(sales_amount) as total_sales
FROM
    sales_fact
WHERE
    date_key BETWEEN 20230101 AND 20230131
GROUP BY
    date_key, product_key;

-- 2. Use partition pruning to limit data scanning
-- Example for a table partitioned by date_key
SELECT
    product_key,
    SUM(sales_amount) as total_sales
FROM
    sales_fact
WHERE
    date_key BETWEEN 20230101 AND 20230131
GROUP BY
    product_key;

-- 3. Leverage projection pushdown (predicate pushdown)
-- Filtering early reduces the amount of data processed
WITH filtered_sales AS (
    SELECT product_key, sales_amount
    FROM sales_fact
    WHERE date_key BETWEEN 20230101 AND 20230131
)
SELECT
    product_key,
    SUM(sales_amount) as total_sales
FROM
    filtered_sales
GROUP BY
    product_key;

```

2. Partitioning and Distribution Strategies

In distributed environments, how data is split across nodes has a significant impact on query performance.

Table Partitioning Partitioning divides a table into smaller, more manageable pieces based on a specified column:

-- Creating a partitioned table in PostgreSQL/Redshift

```
CREATE TABLE sales_fact (  
    sale_id INTEGER,  
    date_key INTEGER,  
    product_key INTEGER,  
    store_key INTEGER,  
    customer_key INTEGER,  
    sales_amount NUMERIC(12,2),  
    -- other columns  
    PRIMARY KEY (sale_id, date_key)  
) PARTITION BY RANGE (date_key);
```

-- Create yearly partitions

```
CREATE TABLE sales_fact_2021  
PARTITION OF sales_fact  
FOR VALUES FROM (20210101) TO (20220101);
```

```
CREATE TABLE sales_fact_2022  
PARTITION OF sales_fact  
FOR VALUES FROM (20220101) TO (20230101);
```

```
CREATE TABLE sales_fact_2023  
PARTITION OF sales_fact  
FOR VALUES FROM (20230101) TO (20240101);
```

-- Snowflake syntax for clustering

```
CREATE OR REPLACE TABLE sales_fact (  
    sale_id INTEGER,  
    date_key INTEGER,  
    product_key INTEGER,  
    store_key INTEGER,  
    customer_key INTEGER,  
    sales_amount NUMERIC(12,2)  
)  
CLUSTER BY (date_key);
```

Distribution Styles Different platforms offer various distribution strategies for splitting data across compute nodes:

-- Amazon Redshift distribution styles

-- 1. KEY distribution (co-locate related data on the same node)

```
CREATE TABLE sales_fact (  
    sale_id INTEGER,  
    date_key INTEGER,  
    product_key INTEGER,  
    store_key INTEGER,
```

```

        customer_key INTEGER,
        sales_amount NUMERIC(12,2)
    )
    DISTKEY(product_key);

-- 2. EVEN distribution (round-robin distribution)
CREATE TABLE large_fact_table (
    -- columns
)
DISTSTYLE EVEN;

-- 3. ALL distribution (replicate entire table on all nodes)
CREATE TABLE small_dimension_table (
    -- columns
)
DISTSTYLE ALL;

-- Snowflake doesn't require manual distribution configuration
-- (it handles distribution automatically)

```

Optimizing Join Performance with Distribution Co-locating join keys on the same nodes reduces data movement during joins:

```

-- Amazon Redshift optimization for joins
CREATE TABLE sales_fact (
    sale_id INTEGER,
    date_key INTEGER,
    product_key INTEGER,
    -- other columns
)
DISTKEY(product_key);

CREATE TABLE product_dimension (
    product_key INTEGER,
    product_name VARCHAR(100),
    -- other columns
)
DISTKEY(product_key);

-- This join will be more efficient as both tables are distributed on product_key
SELECT
    p.product_name,
    SUM(s.sales_amount) as total_sales
FROM
    sales_fact s
JOIN

```

```

        product_dimension p ON s.product_key = p.product_key
GROUP BY
    p.product_name;

-- Distribution suggestions for fact and dimension tables
-- 1. Distribute fact tables on frequently joined foreign keys
-- 2. Distribute large dimension tables on their primary keys
-- 3. Use ALL distribution for small dimension tables
-- 4. Consider EVEN distribution for staging tables or tables without clear join patterns

```

Sort Keys and Zone Maps Sorting data physically can improve performance for range-based queries:

```

-- Amazon Redshift sort keys
CREATE TABLE sales_fact (
    sale_id INTEGER,
    date_key INTEGER,
    product_key INTEGER,
    -- other columns
)
DISTKEY(product_key)
SORTKEY(date_key); -- Compound sort key (default)

-- Multiple sort keys (compound)
CREATE TABLE sales_fact (
    sale_id INTEGER,
    date_key INTEGER,
    product_key INTEGER,
    -- other columns
)
SORTKEY(date_key, product_key);

-- Interleaved sort keys
CREATE TABLE sales_fact (
    sale_id INTEGER,
    date_key INTEGER,
    product_key INTEGER,
    -- other columns
)
INTERLEAVED SORTKEY(date_key, product_key);

-- Snowflake automatically creates zone maps and doesn't require explicit sort keys

```

3. Large-Scale Aggregation Techniques

Analytics at scale requires efficient aggregation strategies.

Memory-Efficient Aggregation When dealing with very large datasets, standard aggregations may exceed memory limits:

-- Two-stage aggregation to reduce memory requirements
-- Stage 1: Pre-aggregate at a granular level

```
WITH daily_product_sales AS (  
    SELECT  
        date_key,  
        product_key,  
        SUM(sales_amount) as total_sales,  
        COUNT(*) as transaction_count  
    FROM  
        sales_fact  
    WHERE  
        date_key BETWEEN 20230101 AND 20230331  
    GROUP BY  
        date_key, product_key  
)  
-- Stage 2: Aggregate the results further  
SELECT  
    FLOOR(date_key / 100) as month_key,  
    product_key,  
    SUM(total_sales) as monthly_sales,  
    SUM(transaction_count) as monthly_transactions  
FROM  
    daily_product_sales  
GROUP BY  
    FLOOR(date_key / 100), product_key;
```

Approximate Aggregations For very large datasets, approximate calculations can be much faster with acceptable accuracy:

-- Approximating distinct counts (PostgreSQL/Redshift)

```
SELECT  
    product_category,  
    APPROXIMATE COUNT(DISTINCT customer_key) as approx_unique_customers  
FROM  
    sales_fact  
JOIN  
    product_dimension USING (product_key)  
GROUP BY  
    product_category;
```

-- Approximate percentile using APPROX_PERCENTILE (BigQuery)

```
SELECT  
    product_category,
```

```

        APPROX_PERCENTILE(sales_amount, 0.5) as median_sale,
        APPROX_PERCENTILE(sales_amount, 0.95) as p95_sale
FROM
    sales_fact
JOIN
    product_dimension USING (product_key)
GROUP BY
    product_category;

-- Approximate count distinct (Snowflake)
SELECT
    product_category,
    APPROX_COUNT_DISTINCT(customer_key) as approx_unique_customers
FROM
    sales_fact
JOIN
    product_dimension USING (product_key)
GROUP BY
    product_category;

```

Handling Skew in Distributed Aggregations Data skew can cause performance issues when certain nodes have more work than others:

```

-- Detecting skew in Redshift
SELECT
    "tbl",
    "col",
    "distkey",
    "skew_rows",
    "skew_rows_ratio"
FROM
    svv_table_info
WHERE
    "table" = 'sales_fact';

-- Handling skewed joins in Redshift (using temporary redistribution)
SET enable_result_cache_for_session TO OFF;
EXPLAIN
SELECT /*+ SHUFFLE */
    p.product_name,
    SUM(s.sales_amount) as total_sales
FROM
    sales_fact s
JOIN
    product_dimension p ON s.product_key = p.product_key
WHERE

```

```

        p.product_name = 'High Volume Product'
GROUP BY
    p.product_name;

```

Materialized Views and Result Caching Pre-calculate common aggregations for faster queries:

```

-- Create a materialized view for reporting in PostgreSQL/Redshift
CREATE MATERIALIZED VIEW monthly_category_sales AS
SELECT
    DATE_TRUNC('month', d.date_value) as month_date,
    p.category,
    SUM(s.sales_amount) as total_sales,
    COUNT(*) as transaction_count
FROM
    sales_fact s
JOIN
    date_dimension d ON s.date_key = d.date_key
JOIN
    product_dimension p ON s.product_key = p.product_key
GROUP BY
    DATE_TRUNC('month', d.date_value), p.category;

-- Create a caching layer in Snowflake
CREATE OR REPLACE TABLE monthly_category_sales AS
SELECT
    DATE_TRUNC('MONTH', d.date_value) as month_date,
    p.category,
    SUM(s.sales_amount) as total_sales,
    COUNT(*) as transaction_count
FROM
    sales_fact s
JOIN
    date_dimension d ON s.date_key = d.date_key
JOIN
    product_dimension p ON s.product_key = p.product_key
GROUP BY
    DATE_TRUNC('MONTH', d.date_value), p.category;

```

4. Incremental Processing

For large-scale analytics, processing only new or changed data is far more efficient than reprocessing everything.

Incremental Aggregation Pattern


```

-- Update an aggregation table incrementally

-- 1. Create the aggregation table
CREATE TABLE monthly_product_sales (
    month_key INTEGER,
    product_key INTEGER,
    total_sales NUMERIC(12,2),
    transaction_count INTEGER,
    last_updated TIMESTAMP,
    PRIMARY KEY (month_key, product_key)
);

-- 2. Track the last processed date
CREATE TABLE processing_metadata (
    table_name VARCHAR(100) PRIMARY KEY,
    last_processed_date_key INTEGER,
    last_processed_timestamp TIMESTAMP
);

-- 3. Incremental update procedure
-- Pseudocode for incremental update
/*
BEGIN TRANSACTION;

-- Get the last processed date
SELECT last_processed_date_key
INTO last_date
FROM processing_metadata
WHERE table_name = 'monthly_product_sales';

-- Process new data
INSERT INTO monthly_product_sales (month_key, product_key, total_sales, transaction_count, last_updated)
SELECT
    FLOOR(date_key / 100) as month_key,
    product_key,
    SUM(sales_amount) as total_sales,
    COUNT(*) as transaction_count,
    CURRENT_TIMESTAMP as last_updated
FROM
    sales_fact
WHERE
    date_key > last_date
GROUP BY
    FLOOR(date_key / 100), product_key
ON CONFLICT (month_key, product_key)
DO UPDATE SET

```

```

        total_sales = monthly_product_sales.total_sales + EXCLUDED.total_sales,
        transaction_count = monthly_product_sales.transaction_count + EXCLUDED.transaction_count,
        last_updated = CURRENT_TIMESTAMP;

-- Update the metadata
UPDATE processing_metadata
SET
    last_processed_date_key = (SELECT MAX(date_key) FROM sales_fact),
    last_processed_timestamp = CURRENT_TIMESTAMP
WHERE
    table_name = 'monthly_product_sales';

COMMIT;
*/

```

Handling Late-Arriving Data In distributed systems, data can arrive out of order or be delayed:

```

-- Create a table with effective date ranges
CREATE TABLE monthly_product_sales_v2 (
    month_key INTEGER,
    product_key INTEGER,
    total_sales NUMERIC(12,2),
    transaction_count INTEGER,
    effective_from TIMESTAMP,
    effective_to TIMESTAMP,
    is_current BOOLEAN,
    PRIMARY KEY (month_key, product_key, effective_from)
);

-- Handle late-arriving data with effective dating
-- Pseudocode for late-arriving data handling
/*
BEGIN TRANSACTION;

-- Mark existing records as not current
UPDATE monthly_product_sales_v2
SET
    effective_to = CURRENT_TIMESTAMP,
    is_current = FALSE
WHERE
    month_key IN (SELECT DISTINCT FLOOR(date_key / 100) FROM staging_sales)
    AND is_current = TRUE;

-- Insert new records with updated totals
INSERT INTO monthly_product_sales_v2 (

```

```

        month_key,
        product_key,
        total_sales,
        transaction_count,
        effective_from,
        effective_to,
        is_current
    )
    SELECT
        FLOOR(date_key / 100) as month_key,
        product_key,
        SUM(sales_amount) as total_sales,
        COUNT(*) as transaction_count,
        CURRENT_TIMESTAMP as effective_from,
        NULL as effective_to,
        TRUE as is_current
    FROM (
        -- Combine existing data with new data for affected months
        SELECT
            date_key,
            product_key,
            sales_amount
        FROM
            sales_fact
        WHERE
            FLOOR(date_key / 100) IN (SELECT DISTINCT FLOOR(date_key / 100) FROM staging_sales)

        UNION ALL

        SELECT
            date_key,
            product_key,
            sales_amount
        FROM
            staging_sales
    ) combined_data
    GROUP BY
        FLOOR(date_key / 100), product_key;

    COMMIT;
*/

```

SCD Type 2 Processing Slowly Changing Dimension Type 2 (SCD2) is a common pattern for tracking changes over time:

```

-- Create a Type 2 SCD table for product dimension
CREATE TABLE product_dimension_scd2 (
    product_key SERIAL,
    product_id INTEGER NOT NULL, -- business key
    product_name VARCHAR(100) NOT NULL,
    category VARCHAR(50) NOT NULL,
    price NUMERIC(12,2) NOT NULL,
    effective_from DATE NOT NULL,
    effective_to DATE,
    is_current BOOLEAN NOT NULL,
    PRIMARY KEY (product_key)
);

-- Create a unique index on business key + effective date
CREATE UNIQUE INDEX idx_product_id_effective
ON product_dimension_scd2 (product_id, effective_from);

-- Insert initial product data
INSERT INTO product_dimension_scd2 (
    product_id, product_name, category, price, effective_from, effective_to, is_current
)
SELECT
    product_id,
    product_name,
    category,
    price,
    '2023-01-01', -- effective date
    NULL, -- no end date for current records
    TRUE -- is current
FROM
    source_products;

-- SCD Type 2 incremental load procedure (pseudocode)
/*
BEGIN TRANSACTION;

-- Identify new products (not in dimension yet)
INSERT INTO product_dimension_scd2 (
    product_id, product_name, category, price, effective_from, effective_to, is_current
)
SELECT
    s.product_id,
    s.product_name,
    s.category,
    s.price,
    CURRENT_DATE, -- effective from today

```

```

        NULL, -- no end date
        TRUE -- is current
FROM
    staging_products s
LEFT JOIN
    product_dimension_scd2 d
    ON s.product_id = d.product_id
    AND d.is_current = TRUE
WHERE
    d.product_id IS NULL;

-- Identify changed products
WITH changed_products AS (
    SELECT
        s.product_id,
        s.product_name,
        s.category,
        s.price
    FROM
        staging_products s
    JOIN
        product_dimension_scd2 d
        ON s.product_id = d.product_id
        AND d.is_current = TRUE
    WHERE
        s.product_name != d.product_name OR
        s.category != d.category OR
        s.price != d.price
)
-- Update current records to be not current anymore
UPDATE product_dimension_scd2 d
SET
    effective_to = CURRENT_DATE - INTERVAL '1 day',
    is_current = FALSE
FROM
    changed_products c
WHERE
    d.product_id = c.product_id
    AND d.is_current = TRUE;

-- Insert new records for changed products
INSERT INTO product_dimension_scd2 (
    product_id, product_name, category, price, effective_from, effective_to, is_current
)
SELECT
    product_id,

```

```

        product_name,
        category,
        price,
        CURRENT_DATE, -- effective from today
        NULL, -- no end date
        TRUE -- is current
FROM
    changed_products;

COMMIT;
*/

```

Merge Patterns for Upserts In data pipelines, you often need to insert new records and update existing ones in a single operation:

```

-- PostgreSQL's MERGE equivalent using ON CONFLICT (upsert)
INSERT INTO monthly_sales_summary (
    month_key,
    product_key,
    total_sales,
    transaction_count,
    last_updated
)
SELECT
    FLOOR(date_key / 100) as month_key,
    product_key,
    SUM(sales_amount) as total_sales,
    COUNT(*) as transaction_count,
    CURRENT_TIMESTAMP as last_updated
FROM
    new_sales_data
GROUP BY
    FLOOR(date_key / 100), product_key
ON CONFLICT (month_key, product_key)
DO UPDATE SET
    total_sales = monthly_sales_summary.total_sales + EXCLUDED.total_sales,
    transaction_count = monthly_sales_summary.transaction_count + EXCLUDED.transaction_count,
    last_updated = CURRENT_TIMESTAMP;

-- Snowflake MERGE statement
MERGE INTO monthly_sales_summary tgt
USING (
    SELECT
        FLOOR(date_key / 100) as month_key,
        product_key,
        SUM(sales_amount) as total_sales,

```

```

        COUNT(*) as transaction_count
    FROM
        new_sales_data
    GROUP BY
        FLOOR(date_key / 100), product_key
) src
ON tgt.month_key = src.month_key AND tgt.product_key = src.product_key
WHEN MATCHED THEN UPDATE SET
    tgt.total_sales = tgt.total_sales + src.total_sales,
    tgt.transaction_count = tgt.transaction_count + src.transaction_count,
    tgt.last_updated = CURRENT_TIMESTAMP()
WHEN NOT MATCHED THEN INSERT (
    month_key,
    product_key,
    total_sales,
    transaction_count,
    last_updated
)
VALUES (
    src.month_key,
    src.product_key,
    src.total_sales,
    src.transaction_count,
    CURRENT_TIMESTAMP()
);

```

5. Modern Data Formats and Integration

Modern analytics platforms often integrate with various data storage formats and systems.

Working with Parquet and ORC Files Many data lake solutions use columnar file formats like Parquet and ORC:

```

-- Redshift Spectrum query on S3 Parquet files
SELECT
    year,
    month,
    product_category,
    SUM(sales_amount) as total_sales
FROM
    spectrum.sales_parquet
WHERE
    year = 2023
    AND month BETWEEN 1 AND 3
GROUP BY

```

```

    year, month, product_category
ORDER BY
    month, total_sales DESC;

-- Creating an external table in Redshift
CREATE EXTERNAL TABLE spectrum.sales_parquet (
    sale_id INTEGER,
    date_key INTEGER,
    product_key INTEGER,
    customer_key INTEGER,
    sales_amount DECIMAL(12,2),
    year INTEGER,
    month INTEGER
)
PARTITIONED BY (year INT, month INT)
STORED AS PARQUET
LOCATION 's3://analytics-bucket/sales/';

-- Adding partitions
ALTER TABLE spectrum.sales_parquet ADD PARTITION(year=2023, month=1)
LOCATION 's3://analytics-bucket/sales/year=2023/month=1/';

-- BigQuery external table on GCS Parquet files
CREATE EXTERNAL TABLE `project.dataset.sales_parquet`
OPTIONS (
    format = 'PARQUET',
    uris = ['gs://analytics-bucket/sales/*.parquet']
);

-- Snowflake external table
CREATE EXTERNAL TABLE sales_external (
    sale_id INTEGER,
    date_key INTEGER,
    product_key INTEGER,
    customer_key INTEGER,
    sales_amount DECIMAL(12,2)
)
STORAGE_INTEGRATION = s3_integration
PARTITION BY (TO_VARCHAR(date_key))
FILE_FORMAT = (TYPE = PARQUET)
LOCATION = 's3://analytics-bucket/sales/';

```

Query Federation Modern analytics platforms can query data from various sources in a single query:


```

-- PostgreSQL foreign table (using postgres_fdw)
CREATE EXTENSION postgres_fdw;

CREATE SERVER remote_server
FOREIGN DATA WRAPPER postgres_fdw
OPTIONS (host 'remote-db-server', port '5432', dbname 'products_db');

CREATE USER MAPPING FOR current_user
SERVER remote_server
OPTIONS (user 'remote_user', password 'remote_password');

CREATE FOREIGN TABLE remote_products (
    product_id INTEGER,
    product_name VARCHAR(100),
    category VARCHAR(50),
    price NUMERIC(12,2)
)
SERVER remote_server
OPTIONS (schema_name 'public', table_name 'products');

-- Query combining local and remote data
SELECT
    rp.product_name,
    SUM(sf.sales_amount) as total_sales
FROM
    sales_fact sf
JOIN
    remote_products rp ON sf.product_key = rp.product_id
WHERE
    sf.date_key BETWEEN 20230101 AND 20230331
GROUP BY
    rp.product_name;

-- Amazon Redshift federated query
-- Query Aurora PostgreSQL and Redshift in a single query
SELECT
    rp.product_name,
    SUM(sf.sales_amount) as total_sales
FROM
    sales_fact sf
JOIN
    mydb.public.products rp ON sf.product_key = rp.product_id
WHERE
    sf.date_key BETWEEN 20230101 AND 20230331
GROUP BY
    rp.product_name;

```

Working with Semi-Structured Data Modern analytics often involves JSON, XML, or other semi-structured data:

```
-- Querying JSON data in PostgreSQL
CREATE TABLE customer_activities (
    customer_id INTEGER,
    activity_date DATE,
    activity_data JSONB
);

-- Query JSON fields
SELECT
    customer_id,
    activity_data->>'activity_type' as activity_type,
    (activity_data->>'amount')::NUMERIC as amount
FROM
    customer_activities
WHERE
    activity_data->>'activity_type' = 'purchase'
    AND activity_date >= '2023-01-01';

-- Aggregate JSON array elements
SELECT
    customer_id,
    jsonb_array_elements(activity_data->'products')->>'product_id' as product_id,
    jsonb_array_elements(activity_data->'products')->>'price' as price
FROM
    customer_activities
WHERE
    activity_data->>'activity_type' = 'purchase'
    AND activity_date >= '2023-01-01';

-- Snowflake semi-structured data
CREATE TABLE customer_activities (
    customer_id INTEGER,
    activity_date DATE,
    activity_data VARIANT
);

-- Query variant data
SELECT
    customer_id,
    activity_data:activity_type::STRING as activity_type,
    activity_data:amount::NUMERIC as amount
FROM
    customer_activities
```

```

WHERE
    activity_data:activity_type::STRING = 'purchase'
    AND activity_date >= '2023-01-01';

```

6. Practical Optimization Techniques for Distributed Analytics

Predicate Pushdown Pushing filters to the data source level reduces the amount of data that needs to be processed:

```

-- Example query without explicit pushdown
SELECT
    cd.customer_segment,
    SUM(sf.sales_amount) as total_sales
FROM
    sales_fact sf
JOIN
    customer_dimension cd ON sf.customer_key = cd.customer_key
WHERE
    sf.date_key BETWEEN 20230101 AND 20230131
    AND cd.customer_segment = 'Enterprise'
GROUP BY
    cd.customer_segment;

-- With explicit pushdown using CTEs
WITH filtered_sales AS (
    SELECT
        customer_key,
        sales_amount
    FROM
        sales_fact
    WHERE
        date_key BETWEEN 20230101 AND 20230131
),
filtered_customers AS (
    SELECT
        customer_key,
        customer_segment
    FROM
        customer_dimension
    WHERE
        customer_segment = 'Enterprise'
)
SELECT
    fc.customer_segment,
    SUM(fs.sales_amount) as total_sales
FROM

```

```

        filtered_sales fs
JOIN
    filtered_customers fc ON fs.customer_key = fc.customer_key
GROUP BY
    fc.customer_segment;

```

Query Splitting for Complex Analytics Break complex queries into manageable parts:

-- Complex query with multiple joins and aggregations

-- Inefficient approach:

```

SELECT
    d.year,
    d.quarter,
    p.category,
    c.customer_segment,
    SUM(s.sales_amount) as total_sales,
    COUNT(DISTINCT s.customer_key) as unique_customers,
    SUM(s.sales_amount) / COUNT(DISTINCT s.customer_key) as sales_per_customer,
    SUM(CASE WHEN s.sales_amount > 1000 THEN 1 ELSE 0 END) as high_value_transactions
FROM
    sales_fact s
JOIN
    date_dimension d ON s.date_key = d.date_key
JOIN
    product_dimension p ON s.product_key = p.product_key
JOIN
    customer_dimension c ON s.customer_key = c.customer_key
WHERE
    d.year = 2023
GROUP BY
    d.year, d.quarter, p.category, c.customer_segment;

```

-- More efficient: Split into stages

-- Stage 1: Create base aggregation

```

CREATE TEMPORARY TABLE stage1_aggregation AS
SELECT
    date_key,
    product_key,
    customer_key,
    SUM(sales_amount) as total_sales,
    COUNT(*) as transaction_count,
    SUM(CASE WHEN sales_amount > 1000 THEN 1 ELSE 0 END) as high_value_count
FROM
    sales_fact
WHERE

```

```

        date_key BETWEEN 20230101 AND 20231231
GROUP BY
    date_key, product_key, customer_key;

-- Stage 2: Join dimensions and create final result
SELECT
    d.year,
    d.quarter,
    p.category,
    c.customer_segment,
    SUM(s.total_sales) as total_sales,
    COUNT(DISTINCT s.customer_key) as unique_customers,
    SUM(s.total_sales) / COUNT(DISTINCT s.customer_key) as sales_per_customer,
    SUM(s.high_value_count) as high_value_transactions
FROM
    stage1_aggregation s
JOIN
    date_dimension d ON s.date_key = d.date_key
JOIN
    product_dimension p ON s.product_key = p.product_key
JOIN
    customer_dimension c ON s.customer_key = c.customer_key
WHERE
    d.year = 2023
GROUP BY
    d.year, d.quarter, p.category, c.customer_segment;

```

Optimizing Data Loading For analytics workloads, bulk loading is more efficient than row-by-row inserts:

```

-- Example of efficient bulk loading in PostgreSQL/Redshift
COPY sales_fact
FROM 's3://mybucket/sales/data.csv'
IAM_ROLE 'arn:aws:iam::0123456789012:role/MyRedshiftRole'
DELIMITER ','
REGION 'us-west-2';

-- Snowflake bulk loading
COPY INTO sales_fact
FROM @my_s3_stage/sales/data.csv
FILE_FORMAT = (TYPE = 'CSV', SKIP_HEADER = 1);

-- Load directly into partitioned tables
COPY INTO sales_fact_2023
FROM @my_s3_stage/sales/2023/
FILE_FORMAT = (TYPE = 'CSV', SKIP_HEADER = 1);

```

Multi-Statement Transactions for Complex Data Pipelines For complex analytics workflows, using transactions ensures atomicity:

```
-- Complex ETL transaction example
BEGIN TRANSACTION;

-- 1. Create a temporary staging table
CREATE TEMPORARY TABLE temp_monthly_sales AS
SELECT
    DATE_TRUNC('month', d.date_value) as month_date,
    p.category,
    SUM(s.sales_amount) as total_sales,
    COUNT(*) as transaction_count
FROM
    sales_fact s
JOIN
    date_dimension d ON s.date_key = d.date_key
JOIN
    product_dimension p ON s.product_key = p.product_key
WHERE
    s.sales_amount > 0
    AND d.date_value >= '2023-01-01'
    AND d.date_value < '2024-01-01'
GROUP BY
    DATE_TRUNC('month', d.date_value), p.category;

-- 2. Delete existing data that will be replaced
DELETE FROM monthly_sales_report
WHERE month_date >= '2023-01-01' AND month_date < '2024-01-01';

-- 3. Insert new data
INSERT INTO monthly_sales_report (
    month_date,
    category,
    total_sales,
    transaction_count,
    sales_per_transaction,
    last_updated
)
SELECT
    month_date,
    category,
    total_sales,
    transaction_count,
    CASE
        WHEN transaction_count > 0 THEN total_sales / transaction_count
```

```

        ELSE 0
    END as sales_per_transaction,
    CURRENT_TIMESTAMP as last_updated
FROM
    temp_monthly_sales;

-- 4. Update the data processing log
INSERT INTO etl_process_log (
    process_name,
    start_date,
    end_date,
    records_processed,
    status,
    comments
)
VALUES (
    'monthly_sales_refresh',
    '2023-01-01',
    '2023-12-31',
    (SELECT COUNT(*) FROM temp_monthly_sales),
    'SUCCESS',
    'Monthly sales refresh completed successfully'
);

COMMIT;

```

By understanding and applying these distributed SQL principles and techniques, you can build highly scalable and efficient analytics solutions across a variety of modern data platforms. # SQL for Analytics: A Comprehensive Guide

Part II: Advanced SQL for Analytics

1. Advanced Filtering and Logic

Advanced filtering is essential for isolating exactly the data subsets needed for analysis.

Complex Filtering with CASE, COALESCE, and NULLIF

```

-- CASE for complex categorization
SELECT
    transaction_id,
    sales_amount,
    CASE
        WHEN sales_amount < 100 THEN 'Small'
        WHEN sales_amount BETWEEN 100 AND 999 THEN 'Medium'
        WHEN sales_amount BETWEEN 1000 AND 9999 THEN 'Large'
    END as category

```

```

        WHEN sales_amount >= 10000 THEN 'Enterprise'
        ELSE 'Unknown'
    END as sale_size_category
FROM
    sales_transactions;

-- COALESCE: Returns the first non-NULL expression
-- Useful for handling missing values
SELECT
    customer_id,
    COALESCE(email, phone, 'No contact info') as contact_info,
    COALESCE(preferred_name, first_name || ' ' || last_name) as display_name,
    COALESCE(annual_spend, 0) as customer_spend
FROM
    customers;

-- NULLIF: Returns NULL if the two expressions are equal
-- Useful for avoiding division by zero and detecting unchanged values
SELECT
    product_id,
    current_month_sales,
    previous_month_sales,
    current_month_sales / NULLIF(previous_month_sales, 0) as sales_ratio,
    NULLIF(current_price, original_price) as price_changed
FROM
    product_performance;

```

Advanced Subquery Techniques

```

-- Scalar subqueries (return a single value)
SELECT
    product_id,
    product_name,
    sales_amount,
    sales_amount / (SELECT AVG(sales_amount) FROM sales_transactions) as ratio_to_avg
FROM
    sales_transactions
JOIN
    products USING (product_id);

-- Correlated subqueries (reference the outer query)
SELECT
    customer_id,
    customer_name,
    annual_spend,
    (SELECT AVG(annual_spend)

```



```

        FROM customers c2
        WHERE c2.customer_segment = c1.customer_segment) as segment_avg_spend
FROM
    customers c1;

```

```

-- Subqueries in the FROM clause
SELECT
    customer_segment,
    AVG(customer_ltv) as avg_segment_ltv
FROM (
    SELECT
        c.customer_id,
        c.customer_name,
        c.customer_segment,
        SUM(s.sales_amount) as customer_ltv
    FROM
        customers c
    JOIN
        sales_transactions s ON c.customer_id = s.customer_id
    GROUP BY
        c.customer_id, c.customer_name, c.customer_segment
) customer_lifetime_value
GROUP BY
    customer_segment;

```

```

-- Subqueries with aggregation
SELECT
    product_id,
    product_name
FROM
    products
WHERE
    product_id IN (
        SELECT product_id
        FROM sales_transactions
        GROUP BY product_id
        HAVING COUNT(*) > 100
    );

```

Exists, Not Exists, and Anti-joins

```

-- EXISTS: Check for the existence of related rows
-- Find customers who purchased a specific product category
SELECT
    c.customer_id,
    c.customer_name

```

```

FROM
    customers c
WHERE
    EXISTS (
        SELECT 1
        FROM sales_transactions s
        JOIN products p ON s.product_id = p.product_id
        WHERE s.customer_id = c.customer_id
        AND p.product_category = 'Electronics'
    );

-- NOT EXISTS: Check for the absence of related rows
-- Find products with no sales in the last 90 days
SELECT
    p.product_id,
    p.product_name
FROM
    products p
WHERE
    NOT EXISTS (
        SELECT 1
        FROM sales_transactions s
        WHERE s.product_id = p.product_id
        AND s.transaction_date >= CURRENT_DATE - INTERVAL '90 days'
    );

-- Anti-join: Finding rows in one table with no match in another
-- Alternative to NOT EXISTS
SELECT
    p.product_id,
    p.product_name
FROM
    products p
LEFT JOIN
    sales_transactions s ON p.product_id = s.product_id
    AND s.transaction_date >= CURRENT_DATE - INTERVAL '90 days'
WHERE
    s.transaction_id IS NULL;

```

Analytic Boolean Logic

```

-- Complex filtering logic for segmentation
SELECT
    customer_id,
    customer_name,
    CASE

```

```

        WHEN
            (annual_spend > 10000 OR lifetime_purchases > 50)
            AND (last_purchase_date >= CURRENT_DATE - INTERVAL '90 days')
            AND customer_status = 'Active'
        THEN 'High Value'
    WHEN
        annual_spend BETWEEN 5000 AND 10000
        AND last_purchase_date >= CURRENT_DATE - INTERVAL '180 days'
    THEN 'Medium Value'
    WHEN customer_status = 'Active' THEN 'Regular'
    ELSE 'At Risk'
END as customer_value_segment
FROM
    customers;

-- Multiple conditions with grouping
SELECT
    product_category,
    COUNT(*) as product_count,
    SUM(CASE
        WHEN price > 100 AND stock_quantity > 0 THEN 1
        ELSE 0
    END) as premium_in_stock,
    SUM(CASE
        WHEN price <= 100 AND stock_quantity > 0 THEN 1
        ELSE 0
    END) as budget_in_stock,
    SUM(CASE
        WHEN stock_quantity = 0 THEN 1
        ELSE 0
    END) as out_of_stock
FROM
    products
GROUP BY
    product_category;

```

2. Advanced Join Techniques

Analytics often requires sophisticated approaches to joining data from multiple sources.

Self Joins for Hierarchical and Sequential Analysis

```

-- Self join for employee hierarchy analysis
SELECT
    e.employee_id,

```

```

        e.employee_name,
        e.title,
        m.employee_name as manager_name,
        m.title as manager_title
FROM
    employees e
LEFT JOIN
    employees m ON e.manager_id = m.employee_id;

-- Sequential events analysis
-- Finding time between a customer's purchases
SELECT
    current.customer_id,
    current.transaction_id as current_transaction,
    current.transaction_date as current_date,
    previous.transaction_id as previous_transaction,
    previous.transaction_date as previous_date,
    current.transaction_date - previous.transaction_date as days_between_purchases
FROM
    sales_transactions current
LEFT JOIN
    sales_transactions previous
    ON current.customer_id = previous.customer_id
    AND previous.transaction_date < current.transaction_date
    AND NOT EXISTS (
        SELECT 1
        FROM sales_transactions middle
        WHERE middle.customer_id = current.customer_id
        AND middle.transaction_date > previous.transaction_date
        AND middle.transaction_date < current.transaction_date
    );

```

Lateral Joins Lateral joins allow subqueries in the FROM clause to reference columns from preceding tables, enabling powerful analytics patterns:

```

-- Finding top 3 products for each customer
SELECT
    c.customer_id,
    c.customer_name,
    top_products.product_id,
    top_products.product_name,
    top_products.total_spent,
    top_products.product_rank
FROM
    customers c
CROSS JOIN LATERAL (

```

```

SELECT
    p.product_id,
    p.product_name,
    SUM(s.sales_amount) as total_spent,
    RANK() OVER (ORDER BY SUM(s.sales_amount) DESC) as product_rank
FROM
    sales_transactions s
JOIN
    products p ON s.product_id = p.product_id
WHERE
    s.customer_id = c.customer_id
GROUP BY
    p.product_id, p.product_name
ORDER BY
    total_spent DESC
LIMIT 3
) top_products;

-- Time-based cohort analysis with lateral joins
SELECT
    first_month,
    COUNT(DISTINCT customer_id) as cohort_size,
    retention.month_number,
    retention.active_customers,
    ROUND(retention.active_customers * 100.0 / COUNT(DISTINCT customer_id), 2) as retention
FROM (
    SELECT
        customer_id,
        DATE_TRUNC('month', MIN(transaction_date)) as first_month
    FROM
        sales_transactions
    GROUP BY
        customer_id
) cohorts
CROSS JOIN LATERAL (
    SELECT
        months.month_number,
        COUNT(DISTINCT s.customer_id) as active_customers
    FROM (
        SELECT generate_series(0, 11) as month_number
    ) months
    LEFT JOIN
        sales_transactions s
        ON s.customer_id = cohorts.customer_id
        AND DATE_TRUNC('month', s.transaction_date) = cohorts.first_month + (months.month_nu
    GROUP BY

```

```

        months.month_number
    ) retention
GROUP BY
    first_month, retention.month_number, retention.active_customers
ORDER BY
    first_month, retention.month_number;

```

Advanced Set Operations for Analysis

```

-- Finding customer overlap between product categories
WITH electronics_customers AS (
    SELECT DISTINCT customer_id
    FROM sales_transactions s
    JOIN products p ON s.product_id = p.product_id
    WHERE p.product_category = 'Electronics'
),
appliance_customers AS (
    SELECT DISTINCT customer_id
    FROM sales_transactions s
    JOIN products p ON s.product_id = p.product_id
    WHERE p.product_category = 'Appliances'
)
SELECT
    'Electronics only' as customer_segment,
    COUNT(*) as customer_count
FROM
    electronics_customers
WHERE
    customer_id NOT IN (SELECT customer_id FROM appliance_customers)

UNION ALL

SELECT
    'Appliances only' as customer_segment,
    COUNT(*) as customer_count
FROM
    appliance_customers
WHERE
    customer_id NOT IN (SELECT customer_id FROM electronics_customers)

UNION ALL

SELECT
    'Both categories' as customer_segment,
    COUNT(*) as customer_count
FROM

```

```

        electronics_customers
WHERE
    customer_id IN (SELECT customer_id FROM appliance_customers);

-- Time-period comparison using UNION ALL
SELECT
    'Current Quarter' as period,
    product_category,
    SUM(sales_amount) as sales_amount
FROM
    sales_transactions s
JOIN
    products p ON s.product_id = p.product_id
WHERE
    transaction_date BETWEEN '2023-01-01' AND '2023-03-31'
GROUP BY
    product_category

UNION ALL

SELECT
    'Previous Quarter' as period,
    product_category,
    SUM(sales_amount) as sales_amount
FROM
    sales_transactions s
JOIN
    products p ON s.product_id = p.product_id
WHERE
    transaction_date BETWEEN '2022-10-01' AND '2022-12-31'
GROUP BY
    product_category

UNION ALL

SELECT
    'Year-over-Year' as period,
    product_category,
    SUM(sales_amount) as sales_amount
FROM
    sales_transactions s
JOIN
    products p ON s.product_id = p.product_id
WHERE
    transaction_date BETWEEN '2022-01-01' AND '2022-03-31'
GROUP BY

```

```

        product_category

ORDER BY
    product_category,
    CASE
        WHEN period = 'Current Quarter' THEN 1
        WHEN period = 'Previous Quarter' THEN 2
        WHEN period = 'Year-over-Year' THEN 3
    END;

### 3. Time-Based Operations

Time-based analysis is central to analytics, enabling trend identification, seasonality analysis, and forecasting.

#### Date and Time Functions

```sql
-- Date extraction and manipulation
SELECT
 transaction_date,
 -- Extract components
 EXTRACT(YEAR FROM transaction_date) AS year,
 EXTRACT(QUARTER FROM transaction_date) AS quarter,
 EXTRACT(MONTH FROM transaction_date) AS month,
 EXTRACT(DAY FROM transaction_date) AS day,
 EXTRACT(DOW FROM transaction_date) AS day_of_week, -- 0 (Sunday) to 6 (Saturday)
 EXTRACT(ISODOW FROM transaction_date) AS iso_day_of_week, -- 1 (Monday) to 7 (Sunday)
 EXTRACT(DOY FROM transaction_date) AS day_of_year,
 EXTRACT(WEEK FROM transaction_date) AS week,

 -- Truncation to specific precision
 DATE_TRUNC('year', transaction_date) AS year_start,
 DATE_TRUNC('quarter', transaction_date) AS quarter_start,
 DATE_TRUNC('month', transaction_date) AS month_start,
 DATE_TRUNC('week', transaction_date) AS week_start,
 DATE_TRUNC('day', transaction_date) AS day_start,

 -- Date math
 transaction_date + INTERVAL '1 month' AS one_month_later,
 transaction_date - INTERVAL '1 year' AS one_year_ago,

 -- Date difference
 CURRENT_DATE - transaction_date AS days_since_transaction,
 EXTRACT(EPOCH FROM (CURRENT_TIMESTAMP - transaction_date)) / 86400 AS days_since_decimal
FROM
 sales_transactions;

```



## Time Series Analysis

```
-- Creating a complete time series without gaps
WITH date_spine AS (
 SELECT
 generate_series(
 DATE_TRUNC('month', MIN(transaction_date)),
 DATE_TRUNC('month', MAX(transaction_date)),
 INTERVAL '1 month'
) AS month_start
 FROM
 sales_transactions
)
SELECT
 date_spine.month_start,
 COALESCE(SUM(s.sales_amount), 0) AS monthly_sales
FROM
 date_spine
LEFT JOIN
 sales_transactions s
 ON DATE_TRUNC('month', s.transaction_date) = date_spine.month_start
GROUP BY
 date_spine.month_start
ORDER BY
 date_spine.month_start;

-- Moving averages (3-month rolling average)
WITH monthly_sales AS (
 SELECT
 DATE_TRUNC('month', transaction_date) AS month,
 SUM(sales_amount) AS total_sales
 FROM
 sales_transactions
 GROUP BY
 DATE_TRUNC('month', transaction_date)
)
SELECT
 month,
 total_sales,
 AVG(total_sales) OVER (
 ORDER BY month
 ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
) AS moving_avg_3month
FROM
 monthly_sales
ORDER BY
```

```

 month;

-- Year-over-year comparison
WITH monthly_sales AS (
 SELECT
 EXTRACT(YEAR FROM transaction_date) AS year,
 EXTRACT(MONTH FROM transaction_date) AS month,
 SUM(sales_amount) AS monthly_sales
 FROM
 sales_transactions
 GROUP BY
 EXTRACT(YEAR FROM transaction_date),
 EXTRACT(MONTH FROM transaction_date)
)
SELECT
 current_year.year,
 current_year.month,
 current_year.monthly_sales AS current_year_sales,
 previous_year.monthly_sales AS previous_year_sales,
 current_year.monthly_sales - previous_year.monthly_sales AS absolute_difference,
 CASE
 WHEN previous_year.monthly_sales = 0 THEN NULL
 ELSE ROUND((current_year.monthly_sales - previous_year.monthly_sales) * 100.0 / previous_year.monthly_sales) AS percentage_growth
 END AS percentage_growth
FROM
 monthly_sales current_year
LEFT JOIN
 monthly_sales previous_year
 ON current_year.month = previous_year.month
 AND current_year.year = previous_year.year + 1
ORDER BY
 current_year.year, current_year.month;

```

## Cohort Analysis

```

-- Customer cohort retention analysis
WITH first_purchases AS (
 -- Get the first purchase month for each customer
 SELECT
 customer_id,
 DATE_TRUNC('month', MIN(transaction_date)) AS first_purchase_month
 FROM
 sales_transactions
 GROUP BY
 customer_id
),

```

```

cohort_activity AS (
 -- Calculate activity for each customer by month
 SELECT
 fp.customer_id,
 fp.first_purchase_month AS cohort_month,
 DATE_TRUNC('month', s.transaction_date) AS activity_month,
 -- Calculate the number of months between first purchase and this activity
 EXTRACT(YEAR FROM DATE_TRUNC('month', s.transaction_date)) * 12 +
 EXTRACT(MONTH FROM DATE_TRUNC('month', s.transaction_date)) -
 (EXTRACT(YEAR FROM fp.first_purchase_month) * 12 +
 EXTRACT(MONTH FROM fp.first_purchase_month)) AS months_since_first_purchase
 FROM
 first_purchases fp
 JOIN
 sales_transactions s ON fp.customer_id = s.customer_id
),
cohort_size AS (
 -- Count of customers in each cohort
 SELECT
 cohort_month,
 COUNT(DISTINCT customer_id) AS num_customers
 FROM
 first_purchases
 GROUP BY
 cohort_month
),
retention_table AS (
 -- Count distinct active customers for each cohort and month
 SELECT
 ca.cohort_month,
 ca.months_since_first_purchase,
 COUNT(DISTINCT ca.customer_id) AS num_customers
 FROM
 cohort_activity ca
 GROUP BY
 ca.cohort_month,
 ca.months_since_first_purchase
)
SELECT
 rt.cohort_month,
 cs.num_customers AS original_cohort_size,
 rt.months_since_first_purchase,
 rt.num_customers AS active_customers,
 ROUND(rt.num_customers * 100.0 / cs.num_customers, 2) AS retention_rate
FROM
 retention_table rt

```

```

JOIN
 cohort_size cs ON rt.cohort_month = cs.cohort_month
ORDER BY
 rt.cohort_month,
 rt.months_since_first_purchase;

```

## Seasonal Analysis

*-- Monthly seasonality analysis*

```

SELECT
 EXTRACT(MONTH FROM transaction_date) AS month,
 TO_CHAR(DATE_TRUNC('month', transaction_date), 'Month') AS month_name,
 EXTRACT(YEAR FROM transaction_date) AS year,
 SUM(sales_amount) AS monthly_sales,

 -- Compare to overall monthly average
 SUM(sales_amount) /
 AVG(SUM(sales_amount)) OVER (PARTITION BY EXTRACT(YEAR FROM transaction_date))
 AS year_seasonality_index
FROM
 sales_transactions
GROUP BY
 EXTRACT(MONTH FROM transaction_date),
 TO_CHAR(DATE_TRUNC('month', transaction_date), 'Month'),
 EXTRACT(YEAR FROM transaction_date)
ORDER BY
 year, month;

```

*-- Day-of-week patterns*

```

SELECT
 EXTRACT(ISODOW FROM transaction_date) AS day_of_week,
 TO_CHAR(transaction_date, 'Day') AS day_name,
 COUNT(*) AS transaction_count,
 SUM(sales_amount) AS total_sales,
 AVG(sales_amount) AS avg_transaction_value
FROM
 sales_transactions
GROUP BY
 EXTRACT(ISODOW FROM transaction_date),
 TO_CHAR(transaction_date, 'Day')
ORDER BY
 day_of_week;

```

*-- Hourly patterns*

```

SELECT
 EXTRACT(HOUR FROM transaction_timestamp) AS hour_of_day,

```

```

COUNT(*) AS transaction_count,
SUM(sales_amount) AS total_sales,
ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 2) AS percent_of_daily_transactions
FROM
 sales_transactions
GROUP BY
 EXTRACT(HOUR FROM transaction_timestamp)
ORDER BY
 hour_of_day;

```

### Time Windows and Intervals

```

-- Activity in the last 30, 60, 90 days
SELECT
 'Last 30 days' AS time_window,
 COUNT(DISTINCT customer_id) AS active_customers,
 COUNT(*) AS transaction_count,
 SUM(sales_amount) AS total_sales
FROM
 sales_transactions
WHERE
 transaction_date >= CURRENT_DATE - INTERVAL '30 days'

UNION ALL

SELECT
 'Last 60 days' AS time_window,
 COUNT(DISTINCT customer_id) AS active_customers,
 COUNT(*) AS transaction_count,
 SUM(sales_amount) AS total_sales
FROM
 sales_transactions
WHERE
 transaction_date >= CURRENT_DATE - INTERVAL '60 days'

UNION ALL

SELECT
 'Last 90 days' AS time_window,
 COUNT(DISTINCT customer_id) AS active_customers,
 COUNT(*) AS transaction_count,
 SUM(sales_amount) AS total_sales
FROM
 sales_transactions
WHERE
 transaction_date >= CURRENT_DATE - INTERVAL '90 days'

```

```

ORDER BY
 CASE
 WHEN time_window = 'Last 30 days' THEN 1
 WHEN time_window = 'Last 60 days' THEN 2
 WHEN time_window = 'Last 90 days' THEN 3
 END;

-- Comparing custom time periods
WITH current_period AS (
 SELECT
 product_category,
 SUM(sales_amount) AS period_sales
 FROM
 sales_transactions s
 JOIN
 products p ON s.product_id = p.product_id
 WHERE
 transaction_date BETWEEN '2023-03-01' AND '2023-03-31'
 GROUP BY
 product_category
),
previous_period AS (
 SELECT
 product_category,
 SUM(sales_amount) AS period_sales
 FROM
 sales_transactions s
 JOIN
 products p ON s.product_id = p.product_id
 WHERE
 transaction_date BETWEEN '2023-02-01' AND '2023-02-28'
 GROUP BY
 product_category
),
year_ago_period AS (
 SELECT
 product_category,
 SUM(sales_amount) AS period_sales
 FROM
 sales_transactions s
 JOIN
 products p ON s.product_id = p.product_id
 WHERE
 transaction_date BETWEEN '2022-03-01' AND '2022-03-31'
 GROUP BY

```

```

 product_category
)
SELECT
 cp.product_category,
 cp.period_sales AS current_period_sales,
 pp.period_sales AS previous_period_sales,
 yap.period_sales AS year_ago_sales,

 -- Month-over-month change
 cp.period_sales - pp.period_sales AS mom_change,
 CASE
 WHEN pp.period_sales = 0 THEN NULL
 ELSE (cp.period_sales - pp.period_sales) * 100.0 / pp.period_sales
 END AS mom_percent_change,

 -- Year-over-year change
 cp.period_sales - yap.period_sales AS yoy_change,
 CASE
 WHEN yap.period_sales = 0 THEN NULL
 ELSE (cp.period_sales - yap.period_sales) * 100.0 / yap.period_sales
 END AS yoy_percent_change
FROM
 current_period cp
LEFT JOIN
 previous_period pp ON cp.product_category = pp.product_category
LEFT JOIN
 year_ago_period yap ON cp.product_category = yap.product_category
ORDER BY
 cp.period_sales DESC;

```

## Working with Timestamps and Time Zones

```

-- Converting between time zones
SELECT
 transaction_timestamp,
 transaction_timestamp AT TIME ZONE 'UTC' AS utc_time,
 transaction_timestamp AT TIME ZONE 'America/New_York' AS eastern_time,
 transaction_timestamp AT TIME ZONE 'America/Los_Angeles' AS pacific_time
FROM
 sales_transactions
LIMIT 10;

-- Aggregating across time zones to a standard time
SELECT
 DATE_TRUNC('day', transaction_timestamp AT TIME ZONE 'UTC') AS day_utc,
 SUM(sales_amount) AS total_sales,

```

```

 COUNT(*) AS transaction_count
FROM
 sales_transactions
GROUP BY
 DATE_TRUNC('day', transaction_timestamp AT TIME ZONE 'UTC')
ORDER BY
 day_utc;

-- Finding peak hours by location's local time
SELECT
 store_timezone,
 EXTRACT(HOUR FROM transaction_timestamp AT TIME ZONE store_timezone) AS local_hour,
 COUNT(*) AS transaction_count,
 SUM(sales_amount) AS total_sales
FROM
 sales_transactions s
JOIN
 stores st ON s.store_id = st.store_id
GROUP BY
 store_timezone,
 EXTRACT(HOUR FROM transaction_timestamp AT TIME ZONE store_timezone)
ORDER BY
 store_timezone,
 local_hour;

```

#### 4. User-Defined Functions for Analytics

User-defined functions (UDFs) can significantly enhance your analytical capabilities by encapsulating complex logic into reusable components.

##### Creating Analytical UDFs

```

-- Simple UDF for age calculation
CREATE OR REPLACE FUNCTION calculate_age(birth_date DATE)
RETURNS INTEGER AS $
BEGIN
 RETURN EXTRACT(YEAR FROM AGE(CURRENT_DATE, birth_date));
END;
$ LANGUAGE plpgsql;

-- Usage
SELECT
 customer_id,
 customer_name,
 birth_date,
 calculate_age(birth_date) AS customer_age

```



```

FROM
 customers;

-- Customer lifetime value function
CREATE OR REPLACE FUNCTION customer_lifetime_value(
 customer_id_param INTEGER,
 start_date DATE DEFAULT '1900-01-01',
 end_date DATE DEFAULT CURRENT_DATE
)
RETURNS NUMERIC AS $
DECLARE
 total_value NUMERIC;
BEGIN
 SELECT COALESCE(SUM(sales_amount), 0)
 INTO total_value
 FROM sales_transactions
 WHERE customer_id = customer_id_param
 AND transaction_date BETWEEN start_date AND end_date;

 RETURN total_value;
END;
$ LANGUAGE plpgsql;

-- Usage
SELECT
 customer_id,
 customer_name,
 customer_lifetime_value(customer_id) AS all_time_value,
 customer_lifetime_value(customer_id, '2023-01-01', '2023-12-31') AS current_year_value
FROM
 customers;

```

## Table-Valued Functions for Analytics

```

-- Product performance analysis function
CREATE OR REPLACE FUNCTION product_performance(
 start_date DATE,
 end_date DATE,
 category_filter TEXT DEFAULT NULL
)
RETURNS TABLE (
 product_id INTEGER,
 product_name TEXT,
 product_category TEXT,
 total_sales NUMERIC,
 transaction_count BIGINT,

```

```

 unique_customers BIGINT,
 avg_sale_value NUMERIC
) AS $
BEGIN
 RETURN QUERY
 SELECT
 p.product_id,
 p.product_name,
 p.product_category,
 COALESCE(SUM(s.sales_amount), 0) AS total_sales,
 COUNT(s.transaction_id) AS transaction_count,
 COUNT(DISTINCT s.customer_id) AS unique_customers,
 CASE
 WHEN COUNT(s.transaction_id) = 0 THEN 0
 ELSE COALESCE(SUM(s.sales_amount), 0) / COUNT(s.transaction_id)
 END AS avg_sale_value
 FROM
 products p
 LEFT JOIN
 sales_transactions s ON p.product_id = s.product_id
 AND s.transaction_date BETWEEN start_date AND end_date
 WHERE
 (category_filter IS NULL OR p.product_category = category_filter)
 GROUP BY
 p.product_id, p.product_name, p.product_category
 ORDER BY
 total_sales DESC;
END;
$ LANGUAGE plpgsql;

-- Usage
SELECT * FROM product_performance('2023-01-01', '2023-03-31', 'Electronics');
SELECT * FROM product_performance('2023-01-01', '2023-03-31');

```

## Statistical UDFs for Advanced Analytics

```

-- Z-score calculation (standardization)
CREATE OR REPLACE FUNCTION z_score(value NUMERIC, mean NUMERIC, std_dev NUMERIC)
RETURNS NUMERIC AS $
BEGIN
 IF std_dev = 0 THEN
 RETURN 0;
 ELSE
 RETURN (value - mean) / std_dev;
 END IF;
END;

```

```

$ LANGUAGE plpgsql;

-- Using the z-score function to identify outliers
WITH product_stats AS (
 SELECT
 product_category,
 AVG(sales_amount) AS avg_sale,
 STDDEV(sales_amount) AS stddev_sale
 FROM
 sales_transactions s
 JOIN
 products p ON s.product_id = p.product_id
 GROUP BY
 product_category
)
SELECT
 s.transaction_id,
 p.product_id,
 p.product_name,
 p.product_category,
 s.sales_amount,
 ps.avg_sale,
 ps.stddev_sale,
 z_score(s.sales_amount, ps.avg_sale, ps.stddev_sale) AS z_score_value
FROM
 sales_transactions s
JOIN
 products p ON s.product_id = p.product_id
JOIN
 product_stats ps ON p.product_category = ps.product_category
WHERE
 ABS(z_score(s.sales_amount, ps.avg_sale, ps.stddev_sale)) > 2
ORDER BY
 ABS(z_score(s.sales_amount, ps.avg_sale, ps.stddev_sale)) DESC;

-- Percentile calculation function
CREATE OR REPLACE FUNCTION percentile_disc_array(
 values_array NUMERIC[],
 percentile NUMERIC
)
RETURNS NUMERIC AS $
DECLARE
 sorted_array NUMERIC[];
 array_length INTEGER;
 percentile_position INTEGER;
BEGIN

```

```

-- Sort the array
SELECT array_agg(val ORDER BY val)
INTO sorted_array
FROM unnest(values_array) AS val;

-- Get the array length
array_length := array_length(sorted_array, 1);

-- Calculate the position for the percentile
percentile_position := CEIL(percentile * array_length);

-- Return the value at the percentile position
IF percentile_position = 0 THEN
 RETURN sorted_array[1];
ELSE
 RETURN sorted_array[percentile_position];
END IF;
END;
$ LANGUAGE plpgsql;

-- Using the percentile function for RFM (Recency, Frequency, Monetary) segmentation
WITH customer_rfm AS (
 SELECT
 customer_id,
 CURRENT_DATE - MAX(transaction_date) AS recency,
 COUNT(*) AS frequency,
 SUM(sales_amount) AS monetary
 FROM
 sales_transactions
 GROUP BY
 customer_id
),
rfm_stats AS (
 SELECT
 percentile_disc_array(ARRAY_AGG(recency), 0.2) AS r_20,
 percentile_disc_array(ARRAY_AGG(recency), 0.4) AS r_40,
 percentile_disc_array(ARRAY_AGG(recency), 0.6) AS r_60,
 percentile_disc_array(ARRAY_AGG(recency), 0.8) AS r_80,

 percentile_disc_array(ARRAY_AGG(frequency), 0.2) AS f_20,
 percentile_disc_array(ARRAY_AGG(frequency), 0.4) AS f_40,
 percentile_disc_array(ARRAY_AGG(frequency), 0.6) AS f_60,
 percentile_disc_array(ARRAY_AGG(frequency), 0.8) AS f_80,

 percentile_disc_array(ARRAY_AGG(monetary), 0.2) AS m_20,
 percentile_disc_array(ARRAY_AGG(monetary), 0.4) AS m_40,

```

```

 percentile_disc_array(ARRAY_AGG(monetary), 0.6) AS m_60,
 percentile_disc_array(ARRAY_AGG(monetary), 0.8) AS m_80
FROM
 customer_rfm
)
SELECT
 c.customer_id,
 c.customer_name,
 rfm.recency,
 rfm.frequency,
 rfm.monetary,

 -- Recency score (lower is better)
 CASE
 WHEN rfm.recency <= rs.r_20 THEN 5
 WHEN rfm.recency <= rs.r_40 THEN 4
 WHEN rfm.recency <= rs.r_60 THEN 3
 WHEN rfm.recency <= rs.r_80 THEN 2
 ELSE 1
 END AS r_score,

 -- Frequency score (higher is better)
 CASE
 WHEN rfm.frequency >= rs.f_80 THEN 5
 WHEN rfm.frequency >= rs.f_60 THEN 4
 WHEN rfm.frequency >= rs.f_40 THEN 3
 WHEN rfm.frequency >= rs.f_20 THEN 2
 ELSE 1
 END AS f_score,

 -- Monetary score (higher is better)
 CASE
 WHEN rfm.monetary >= rs.m_80 THEN 5
 WHEN rfm.monetary >= rs.m_60 THEN 4
 WHEN rfm.monetary >= rs.m_40 THEN 3
 WHEN rfm.monetary >= rs.m_20 THEN 2
 ELSE 1
 END AS m_score
FROM
 customer_rfm rfm
JOIN
 customers c ON rfm.customer_id = c.customer_id
CROSS JOIN
 rfm_stats rs
ORDER BY
 (r_score + f_score + m_score) DESC;# SQL for Analytics: A Comprehensive Guide

```

## Part III: Data Modeling for Analytics

### 1. Dimensional Modeling Concepts

Dimensional modeling is a database design technique specifically optimized for data warehouses and analytics. It focuses on delivering data that is intuitive to business users and provides high performance for analytical queries.

#### Facts and Dimensions

```
-- Core concepts of dimensional modeling:
-- 1. Fact tables: contain measurements, metrics, or facts about business processes
-- 2. Dimension tables: contain descriptive attributes about business entities
```

```
-- Example fact table: sales_fact
```

```
CREATE TABLE sales_fact (
 sale_id INTEGER PRIMARY KEY,
 date_key INTEGER REFERENCES date_dimension(date_key),
 product_key INTEGER REFERENCES product_dimension(product_key),
 customer_key INTEGER REFERENCES customer_dimension(customer_key),
 store_key INTEGER REFERENCES store_dimension(store_key),
 promotion_key INTEGER REFERENCES promotion_dimension(promotion_key),
 quantity INTEGER NOT NULL,
 unit_price NUMERIC(12,2) NOT NULL,
 discount_amount NUMERIC(12,2) NOT NULL,
 sales_amount NUMERIC(12,2) NOT NULL,
 cost_amount NUMERIC(12,2) NOT NULL,
 profit_amount NUMERIC(12,2) NOT NULL
);
```

```
-- Example dimension table: product_dimension
```

```
CREATE TABLE product_dimension (
 product_key INTEGER PRIMARY KEY,
 product_id INTEGER NOT NULL, -- business key
 product_name VARCHAR(100) NOT NULL,
 product_description TEXT,
 brand VARCHAR(50),
 category VARCHAR(50) NOT NULL,
 subcategory VARCHAR(50),
 department VARCHAR(50),
 size VARCHAR(20),
 color VARCHAR(20),
 weight NUMERIC(8,2),
 cost NUMERIC(12,2),
 retail_price NUMERIC(12,2),
 effective_date DATE NOT NULL,
 expiration_date DATE,
```

```

 is_current BOOLEAN NOT NULL
);

```

#### #### Key Concepts in Dimensional Modeling

##### \*\*1. Grain\*\*

The grain defines the level of detail in a fact table. It answers the question: "What does a

```

```sql
-- Examples of different grain levels:
-- Transaction grain (most detailed)
CREATE TABLE sales_transaction_fact (
    transaction_id INTEGER,
    transaction_line_id INTEGER,
    date_key INTEGER,
    product_key INTEGER,
    customer_key INTEGER,
    quantity INTEGER,
    sales_amount NUMERIC(12,2),
    PRIMARY KEY (transaction_id, transaction_line_id)
);

```

```

-- Daily product sales grain (aggregated)
CREATE TABLE daily_product_sales_fact (
    date_key INTEGER,
    product_key INTEGER,
    store_key INTEGER,
    total_quantity INTEGER,
    total_sales_amount NUMERIC(12,2),
    PRIMARY KEY (date_key, product_key, store_key)
);

```

```

-- Monthly customer grain (highly aggregated)
CREATE TABLE monthly_customer_sales_fact (
    year_month INTEGER, -- YYYYMM format
    customer_key INTEGER,
    total_transactions INTEGER,
    total_quantity INTEGER,
    total_sales_amount NUMERIC(12,2),
    PRIMARY KEY (year_month, customer_key)
);

```

2. Dimension Table Design

Dimensions provide the context for facts and support filtering, grouping, and labeling.

```

-- Shared dimension pattern (conformed dimension)
-- This customer dimension can be used across multiple fact tables
CREATE TABLE customer_dimension (
    customer_key INTEGER PRIMARY KEY,
    customer_id VARCHAR(20) NOT NULL, -- business key
    customer_name VARCHAR(100) NOT NULL,
    email VARCHAR(100),
    phone VARCHAR(20),
    address_line1 VARCHAR(100),
    address_line2 VARCHAR(100),
    city VARCHAR(50),
    state_province VARCHAR(50),
    postal_code VARCHAR(20),
    country VARCHAR(50),
    customer_since_date DATE,
    customer_segment VARCHAR(30),
    credit_rating VARCHAR(10),
    lifetime_value NUMERIC(12,2)
);

-- Role-playing dimension (same dimension used in different contexts)
CREATE TABLE date_dimension (
    date_key INTEGER PRIMARY KEY,
    full_date DATE UNIQUE NOT NULL,
    day_of_week INTEGER NOT NULL, -- 1-7
    day_of_week_name VARCHAR(10) NOT NULL, -- Monday, Tuesday, etc.
    day_of_month INTEGER NOT NULL, -- 1-31
    day_of_year INTEGER NOT NULL, -- 1-366
    week_of_month INTEGER NOT NULL,
    week_of_year INTEGER NOT NULL,
    month_number INTEGER NOT NULL, -- 1-12
    month_name VARCHAR(10) NOT NULL, -- January, February, etc.
    quarter INTEGER NOT NULL, -- 1-4
    quarter_name VARCHAR(10) NOT NULL, -- Q1, Q2, Q3, Q4
    year INTEGER NOT NULL
);

-- Using the same date dimension in different roles
SELECT
    od.full_date AS order_date,
    sd.full_date AS ship_date,
    pd.full_date AS payment_date,
    f.order_amount,
    f.ship_amount,
    f.payment_amount
FROM

```



```

        order_fact f
JOIN
    date_dimension od ON f.order_date_key = od.date_key
JOIN
    date_dimension sd ON f.ship_date_key = sd.date_key
JOIN
    date_dimension pd ON f.payment_date_key = pd.date_key;

```

3. Fact Table Types

There are three main types of fact tables, each serving different analytical needs:

```

-- 1. Transaction fact table (records individual events)
CREATE TABLE sales_transaction_fact (
    transaction_id INTEGER,
    line_item_id INTEGER,
    date_key INTEGER,
    product_key INTEGER,
    customer_key INTEGER,
    quantity INTEGER NOT NULL,
    sales_amount NUMERIC(12,2) NOT NULL,
    PRIMARY KEY (transaction_id, line_item_id)
);

-- 2. Periodic snapshot fact table (records regular snapshots)
CREATE TABLE monthly_inventory_fact (
    date_key INTEGER, -- last day of month
    product_key INTEGER,
    store_key INTEGER,
    quantity_on_hand INTEGER,
    quantity_on_order INTEGER,
    inventory_value NUMERIC(12,2),
    PRIMARY KEY (date_key, product_key, store_key)
);

-- 3. Accumulating snapshot fact table (records process milestones)
CREATE TABLE order_fulfillment_fact (
    order_key INTEGER PRIMARY KEY,
    customer_key INTEGER,
    product_key INTEGER,
    order_date_key INTEGER,
    approved_date_key INTEGER,
    picked_date_key INTEGER,
    shipped_date_key INTEGER,
    delivered_date_key INTEGER,
    order_amount NUMERIC(12,2),
    shipping_amount NUMERIC(12,2),

```

```

        total_amount NUMERIC(12,2),
        order_status VARCHAR(20)
    );

```

4. Slowly Changing Dimensions

Dimensions can change over time, and there are different strategies for handling these changes:

```

-- Type 1 SCD: Overwrite the old value (no history)
UPDATE product_dimension
SET
    category = 'Home Electronics',
    subcategory = 'Smart Home'
WHERE
    product_id = 12345;

-- Type 2 SCD: Add a new row with the updated values (preserve history)
-- 1. Set expiration on the current record
UPDATE product_dimension
SET
    expiration_date = CURRENT_DATE,
    is_current = FALSE
WHERE
    product_id = 12345
    AND is_current = TRUE;

-- 2. Insert the new version
INSERT INTO product_dimension (
    product_key, product_id, product_name, category, subcategory,
    effective_date, expiration_date, is_current
)
VALUES (
    NEXTVAL('product_key_seq'), 12345, 'Smart Thermostat',
    'Home Electronics', 'Smart Home',
    CURRENT_DATE, NULL, TRUE
);

-- Query current product information
SELECT * FROM product_dimension WHERE is_current = TRUE;

-- Query historical product information
SELECT * FROM product_dimension
WHERE product_id = 12345
ORDER BY effective_date;

-- Type 3 SCD: Add new columns to track a limited history

```

```

CREATE TABLE product_dimension_type3 (
    product_key INTEGER PRIMARY KEY,
    product_id INTEGER NOT NULL,
    product_name VARCHAR(100) NOT NULL,
    current_category VARCHAR(50) NOT NULL,
    previous_category VARCHAR(50),
    category_change_date DATE,
    -- other attributes
    is_current BOOLEAN NOT NULL
);

```

5. Junk Dimensions

A junk dimension combines several low-cardinality flags or attributes into a single dimension to avoid dimension bloat:

```

-- Creating a junk dimension for order flags
CREATE TABLE order_flags_dimension (
    flag_key INTEGER PRIMARY KEY,
    is_rush_order BOOLEAN NOT NULL,
    is_gift BOOLEAN NOT NULL,
    has_gift_wrapping BOOLEAN NOT NULL,
    has_special_instructions BOOLEAN NOT NULL,
    delivery_method VARCHAR(20) NOT NULL,
    payment_method VARCHAR(20) NOT NULL
);

-- Populate with all possible combinations
INSERT INTO order_flags_dimension
(flag_key, is_rush_order, is_gift, has_gift_wrapping,
has_special_instructions, delivery_method, payment_method)
VALUES
(1, FALSE, FALSE, FALSE, FALSE, 'Standard', 'Credit Card'),
(2, TRUE, FALSE, FALSE, FALSE, 'Express', 'Credit Card'),
(3, FALSE, TRUE, TRUE, FALSE, 'Standard', 'Credit Card'),
-- Add all other relevant combinations
(16, TRUE, TRUE, TRUE, TRUE, 'Express', 'PayPal');

-- Use in fact table
CREATE TABLE order_fact (
    order_key INTEGER PRIMARY KEY,
    date_key INTEGER,
    customer_key INTEGER,
    flag_key INTEGER REFERENCES order_flags_dimension(flag_key),
    order_amount NUMERIC(12,2)
);

```

6. Degenerate Dimensions

Degenerate dimensions are attributes that are neither measures (facts) nor dimension attributes but are still useful for analysis:

```
CREATE TABLE sales_fact (  
    date_key INTEGER,  
    product_key INTEGER,  
    store_key INTEGER,  
    invoice_number VARCHAR(20), -- degenerate dimension  
    po_number VARCHAR(20), -- degenerate dimension  
    quantity INTEGER,  
    sales_amount NUMERIC(12,2),  
    PRIMARY KEY (date_key, product_key, store_key, invoice_number)  
);
```

2. Star and Snowflake Schemas

The two primary dimensional modeling patterns are star schemas and snowflake schemas. Both are designed for analytical workloads but have different approaches to dimension normalization.

Star Schema A star schema features a central fact table surrounded by denormalized dimension tables. Each dimension table has a primary key that directly relates to the fact table, creating a star-like pattern.

-- Star Schema Example

-- Dimension tables

```
CREATE TABLE date_dimension (  
    date_key INTEGER PRIMARY KEY,  
    full_date DATE,  
    day_of_week VARCHAR(10),  
    day_of_month INTEGER,  
    month VARCHAR(10),  
    quarter VARCHAR(2),  
    year INTEGER  
);  
  
CREATE TABLE product_dimension (  
    product_key INTEGER PRIMARY KEY,  
    product_id VARCHAR(20),  
    product_name VARCHAR(100),  
    brand VARCHAR(50),  
    category VARCHAR(50),  
    subcategory VARCHAR(50),  
    unit_cost NUMERIC(12,2),  
    unit_price NUMERIC(12,2)  
);
```

```

CREATE TABLE store_dimension (
    store_key INTEGER PRIMARY KEY,
    store_id VARCHAR(20),
    store_name VARCHAR(100),
    store_type VARCHAR(30),
    region VARCHAR(30),
    city VARCHAR(50),
    state VARCHAR(2),
    country VARCHAR(30),
    open_date DATE
);

-- Fact table
CREATE TABLE sales_fact (
    sale_key SERIAL PRIMARY KEY,
    date_key INTEGER REFERENCES date_dimension(date_key),
    product_key INTEGER REFERENCES product_dimension(product_key),
    store_key INTEGER REFERENCES store_dimension(store_key),
    quantity INTEGER,
    unit_price NUMERIC(12,2),
    extended_price NUMERIC(12,2),
    cost NUMERIC(12,2),
    profit NUMERIC(12,2)
);

-- Typical star schema query
SELECT
    d.year,
    d.quarter,
    p.category,
    p.brand,
    s.region,
    SUM(f.quantity) AS total_units,
    SUM(f.extended_price) AS total_sales,
    SUM(f.profit) AS total_profit
FROM
    sales_fact f
JOIN
    date_dimension d ON f.date_key = d.date_key
JOIN
    product_dimension p ON f.product_key = p.product_key
JOIN
    store_dimension s ON f.store_key = s.store_key
WHERE
    d.year = 2023

```

```

GROUP BY
    d.year, d.quarter, p.category, p.brand, s.region
ORDER BY
    d.quarter, total_sales DESC;

```

Snowflake Schema A snowflake schema normalizes dimension tables by creating additional tables that relate to the dimension tables, rather than directly to the fact table. This creates a more complex, snowflake-like structure.

-- Snowflake Schema Example

-- Normalized dimension tables

```

CREATE TABLE category_dimension (
    category_key INTEGER PRIMARY KEY,
    category_name VARCHAR(50)
);

CREATE TABLE subcategory_dimension (
    subcategory_key INTEGER PRIMARY KEY,
    category_key INTEGER REFERENCES category_dimension(category_key),
    subcategory_name VARCHAR(50)
);

CREATE TABLE brand_dimension (
    brand_key INTEGER PRIMARY KEY,
    brand_name VARCHAR(50),
    manufacturer VARCHAR(100)
);

CREATE TABLE product_dimension (
    product_key INTEGER PRIMARY KEY,
    product_id VARCHAR(20),
    product_name VARCHAR(100),
    brand_key INTEGER REFERENCES brand_dimension(brand_key),
    subcategory_key INTEGER REFERENCES subcategory_dimension(subcategory_key),
    unit_cost NUMERIC(12,2),
    unit_price NUMERIC(12,2)
);

CREATE TABLE country_dimension (
    country_key INTEGER PRIMARY KEY,
    country_name VARCHAR(50)
);

CREATE TABLE state_dimension (
    state_key INTEGER PRIMARY KEY,

```

```

        country_key INTEGER REFERENCES country_dimension(country_key),
        state_name VARCHAR(50),
        state_code VARCHAR(10)
    );

CREATE TABLE city_dimension (
    city_key INTEGER PRIMARY KEY,
    state_key INTEGER REFERENCES state_dimension(state_key),
    city_name VARCHAR(50)
);

CREATE TABLE store_dimension (
    store_key INTEGER PRIMARY KEY,
    store_id VARCHAR(20),
    store_name VARCHAR(100),
    store_type VARCHAR(30),
    city_key INTEGER REFERENCES city_dimension(city_key),
    open_date DATE
);

-- Fact table (same as in star schema)
CREATE TABLE sales_fact (
    sale_key SERIAL PRIMARY KEY,
    date_key INTEGER REFERENCES date_dimension(date_key),
    product_key INTEGER REFERENCES product_dimension(product_key),
    store_key INTEGER REFERENCES store_dimension(store_key),
    quantity INTEGER,
    unit_price NUMERIC(12,2),
    extended_price NUMERIC(12,2),
    cost NUMERIC(12,2),
    profit NUMERIC(12,2)
);

-- Snowflake schema query (more joins but same result)
SELECT
    d.year,
    d.quarter,
    c.category_name,
    b.brand_name,
    country.country_name,
    SUM(f.quantity) AS total_units,
    SUM(f.extended_price) AS total_sales,
    SUM(f.profit) AS total_profit
FROM
    sales_fact f
JOIN

```

```

    date_dimension d ON f.date_key = d.date_key
JOIN
    product_dimension p ON f.product_key = p.product_key
JOIN
    brand_dimension b ON p.brand_key = b.brand_key
JOIN
    subcategory_dimension sub ON p.subcategory_key = sub.subcategory_key
JOIN
    category_dimension c ON sub.category_key = c.category_key
JOIN
    store_dimension s ON f.store_key = s.store_key
JOIN
    city_dimension city ON s.city_key = city.city_key
JOIN
    state_dimension state ON city.state_key = state.state_key
JOIN
    country_dimension country ON state.country_key = country.country_key
WHERE
    d.year = 2023
GROUP BY
    d.year, d.quarter, c.category_name, b.brand_name, country.country_name
ORDER BY
    d.quarter, total_sales DESC;

```

Star vs. Snowflake Comparison **Star Schema Advantages:** - Simpler structure with fewer joins required for queries - Generally better query performance - Easier for business users to understand - Simpler ETL processes

Snowflake Schema Advantages: - More normalized structure uses less storage space - Better dimension consistency when attributes are shared - Can better represent complex hierarchical relationships - Often easier to maintain and update dimensions

3. Time and Date Dimensions

Time dimensions are among the most important in an analytical database. Nearly every fact table connects to at least one time dimension, and most analyses include time-based comparisons.

Creating a Comprehensive Date Dimension A well-designed date dimension makes time-based analysis much simpler:

```

-- Create the date dimension table
CREATE TABLE date_dimension (
    date_key INTEGER PRIMARY KEY,
    date_value DATE UNIQUE NOT NULL,

```



```

-- Calendar hierarchical components
year INTEGER NOT NULL,
quarter INTEGER NOT NULL,
quarter_name VARCHAR(10) NOT NULL, -- e.g., "Q1", "Q2"
month_number INTEGER NOT NULL, -- 1 to 12
month_name VARCHAR(10) NOT NULL, -- January, February, etc.
month_name_short VARCHAR(3) NOT NULL, -- Jan, Feb, etc.
week_of_year INTEGER NOT NULL, -- 1 to 53
day_of_year INTEGER NOT NULL, -- 1 to 366
day_of_month INTEGER NOT NULL, -- 1 to 31
day_of_week INTEGER NOT NULL, -- 1 (Monday) to 7 (Sunday)
day_of_week_name VARCHAR(10) NOT NULL, -- Monday, Tuesday, etc.
day_of_week_name_short VARCHAR(3) NOT NULL, -- Mon, Tue, etc.

-- Fiscal components (if fiscal year is different from calendar year)
fiscal_year INTEGER NOT NULL,
fiscal_quarter INTEGER NOT NULL,
fiscal_month INTEGER NOT NULL,

-- Period start/end indicators
is_first_day_of_month BOOLEAN NOT NULL,
is_last_day_of_month BOOLEAN NOT NULL,
is_first_day_of_quarter BOOLEAN NOT NULL,
is_last_day_of_quarter BOOLEAN NOT NULL,
is_first_day_of_year BOOLEAN NOT NULL,
is_last_day_of_year BOOLEAN NOT NULL,

-- Holiday and business indicators
is_weekend BOOLEAN NOT NULL,
is_holiday BOOLEAN NOT NULL,
holiday_name VARCHAR(50),
is_business_day BOOLEAN NOT NULL,

-- Seasons and special periods
season VARCHAR(10), -- Spring, Summer, Fall, Winter

-- Special retail periods
is_black_friday BOOLEAN NOT NULL DEFAULT FALSE,
is_cyber_monday BOOLEAN NOT NULL DEFAULT FALSE,
is_holiday_season BOOLEAN NOT NULL DEFAULT FALSE -- Nov 15 - Dec 31
);

-- Populate the date dimension for 10 years (can be done with a script or procedure)
INSERT INTO date_dimension
SELECT
    TO_CHAR(dt, 'YYYYMMDD')::INTEGER AS date_key,

```

```

dt AS date_value,
EXTRACT(YEAR FROM dt) AS year,
EXTRACT(QUARTER FROM dt) AS quarter,
'Q' || EXTRACT(QUARTER FROM dt) AS quarter_name,
EXTRACT(MONTH FROM dt) AS month_number,
TO_CHAR(dt, 'Month') AS month_name,
TO_CHAR(dt, 'Mon') AS month_name_short,
EXTRACT(WEEK FROM dt) AS week_of_year,
EXTRACT(DOY FROM dt) AS day_of_year,
EXTRACT(DAY FROM dt) AS day_of_month,
EXTRACT(ISODOW FROM dt) AS day_of_week,
TO_CHAR(dt, 'Day') AS day_of_week_name,
TO_CHAR(dt, 'Dy') AS day_of_week_name_short,

-- Fiscal year (example assumes fiscal year starts in July)
CASE
    WHEN EXTRACT(MONTH FROM dt) >= 7 THEN EXTRACT(YEAR FROM dt) + 1
    ELSE EXTRACT(YEAR FROM dt)
END AS fiscal_year,

-- Fiscal quarter
CASE
    WHEN EXTRACT(MONTH FROM dt) BETWEEN 7 AND 9 THEN 1
    WHEN EXTRACT(MONTH FROM dt) BETWEEN 10 AND 12 THEN 2
    WHEN EXTRACT(MONTH FROM dt) BETWEEN 1 AND 3 THEN 3
    ELSE 4
END AS fiscal_quarter,

-- Fiscal month
CASE
    WHEN EXTRACT(MONTH FROM dt) >= 7 THEN EXTRACT(MONTH FROM dt) - 6
    ELSE EXTRACT(MONTH FROM dt) + 6
END AS fiscal_month,

-- First/last day indicators
EXTRACT(DAY FROM dt) = 1 AS is_first_day_of_month,
EXTRACT(DAY FROM dt) = EXTRACT(DAY FROM (DATE_TRUNC('MONTH', dt) + INTERVAL '1 MONTH - 1 DAY')) AS is_last_day_of_month,

EXTRACT(DAY FROM dt) = 1 AND EXTRACT(MONTH FROM dt) IN (1, 4, 7, 10) AS is_first_day_of_year,
EXTRACT(DAY FROM dt) = EXTRACT(DAY FROM (DATE_TRUNC('MONTH', dt) + INTERVAL '1 MONTH - 1 DAY'))
    AND EXTRACT(MONTH FROM dt) IN (3, 6, 9, 12) AS is_last_day_of_year,

EXTRACT(DOY FROM dt) = 1 AS is_first_day_of_year,
EXTRACT(MONTH FROM dt) = 12 AND
EXTRACT(DAY FROM dt) = 31 AS is_last_day_of_year,

```

```

-- Weekend indicator
EXTRACT(ISODOW FROM dt) >= 6 AS is_weekend,

-- Holiday indicators would be populated based on region-specific logic
FALSE AS is_holiday, -- Placeholder, needs more complex logic
NULL AS holiday_name, -- Placeholder

EXTRACT(ISODOW FROM dt) < 6 AS is_business_day, -- Simplified, should also exclude holi

-- Season (Northern Hemisphere)
CASE
    WHEN EXTRACT(MONTH FROM dt) BETWEEN 3 AND 5 THEN 'Spring'
    WHEN EXTRACT(MONTH FROM dt) BETWEEN 6 AND 8 THEN 'Summer'
    WHEN EXTRACT(MONTH FROM dt) BETWEEN 9 AND 11 THEN 'Fall'
    ELSE 'Winter'
END AS season,

-- Retail periods
(EXTRACT(MONTH FROM dt) = 11 AND EXTRACT(DAY FROM dt) =
    (SELECT EXTRACT(DAY FROM (DATE_TRUNC('MONTH', DATE '2023-11-01') +
        ((EXTRACT(ISODOW FROM DATE_TRUNC('MONTH', DATE '2023-11-01') + INTERVAL '21 day')) +
AS is_black_friday,

(EXTRACT(MONTH FROM dt) = 11 AND EXTRACT(DAY FROM dt) =
    (SELECT EXTRACT(DAY FROM (DATE_TRUNC('MONTH', DATE '2023-11-01') +
        ((EXTRACT(ISODOW FROM DATE_TRUNC('MONTH', DATE '2023-11-01') + INTERVAL '21 day')) +
AS is_cyber_monday,

(EXTRACT(MONTH FROM dt) = 11 AND EXTRACT(DAY FROM dt) >= 15) OR
(EXTRACT(MONTH FROM dt) = 12) AS is_holiday_season
FROM
    generate_series(
        '2020-01-01'::date,
        '2030-12-31'::date,
        '1 day'::interval
    ) AS dt;

```

Using the Date Dimension for Analysis

```

-- Year-over-year comparison by quarter
SELECT
    current_year.quarter_name,
    SUM(current_facts.sales_amount) AS current_year_sales,
    SUM(previous_facts.sales_amount) AS previous_year_sales,
    (SUM(current_facts.sales_amount) - SUM(previous_facts.sales_amount)) /
        NULLIF(SUM(previous_facts.sales_amount), 0) * 100 AS percent_change

```

```

FROM
    sales_fact current_facts
JOIN
    date_dimension current_year ON current_facts.date_key = current_year.date_key
JOIN
    date_dimension previous_year ON previous_year.date_value = current_year.date_value - INTERVAL '1 year'
LEFT JOIN
    sales_fact previous_facts ON previous_facts.date_key = previous_year.date_key
WHERE
    current_year.year = 2023
GROUP BY
    current_year.quarter, current_year.quarter_name
ORDER BY
    current_year.quarter;

-- Sales by day of week
SELECT
    d.day_of_week_name,
    SUM(f.sales_amount) AS total_sales,
    COUNT(*) AS transaction_count,
    SUM(f.sales_amount) / COUNT(*) AS avg_transaction_value
FROM
    sales_fact f
JOIN
    date_dimension d ON f.date_key = d.date_key
WHERE
    d.year = 2023
GROUP BY
    d.day_of_week, d.day_of_week_name
ORDER BY
    d.day_of_week;

-- Holiday vs. non-holiday performance
SELECT
    CASE WHEN d.is_holiday THEN 'Holiday' ELSE 'Regular Day' END AS day_type,
    COUNT(DISTINCT d.date_value) AS number_of_days,
    SUM(f.sales_amount) AS total_sales,
    SUM(f.sales_amount) / COUNT(DISTINCT d.date_value) AS avg_daily_sales
FROM
    sales_fact f
JOIN
    date_dimension d ON f.date_key = d.date_key
WHERE
    d.year = 2023
GROUP BY
    CASE WHEN d.is_holiday THEN 'Holiday' ELSE 'Regular Day' END;

```

```

-- Business day vs. weekend performance by category
SELECT
    p.category,
    CASE WHEN d.is_weekend THEN 'Weekend' ELSE 'Business Day' END AS day_type,
    COUNT(DISTINCT d.date_value) AS number_of_days,
    SUM(f.sales_amount) AS total_sales,
    SUM(f.sales_amount) / COUNT(DISTINCT d.date_value) AS avg_daily_sales,
    COUNT(f.sale_id) AS transaction_count,
    COUNT(f.sale_id) / COUNT(DISTINCT d.date_value) AS avg_daily_transactions
FROM
    sales_fact f
JOIN
    date_dimension d ON f.date_key = d.date_key
JOIN
    product_dimension p ON f.product_key = p.product_key
WHERE
    d.year = 2023
GROUP BY
    p.category,
    CASE WHEN d.is_weekend THEN 'Weekend' ELSE 'Business Day' END
ORDER BY
    p.category, day_type;

```

Time-of-Day Dimension For more granular analysis of intraday patterns, a time-of-day dimension can be useful:

```

-- Create the time dimension table
CREATE TABLE time_dimension (
    time_key INTEGER PRIMARY KEY,
    time_value TIME UNIQUE NOT NULL,
    hour_24 INTEGER NOT NULL, -- 0-23
    hour_12 INTEGER NOT NULL, -- 1-12
    am_pm VARCHAR(2) NOT NULL, -- AM/PM
    minute INTEGER NOT NULL, -- 0-59
    second INTEGER NOT NULL, -- 0-59

    -- Time periods
    day_period VARCHAR(20) NOT NULL, -- Morning, Afternoon, Evening, Night
    business_hour BOOLEAN NOT NULL, -- True if during business hours (e.g., 9am-5pm)
    peak_hour BOOLEAN NOT NULL, -- True if during peak hours (defined by business)

    -- Formatted values
    time_format_24 VARCHAR(8) NOT NULL, -- HH:MM:SS
    time_format_12 VARCHAR(11) NOT NULL -- HH:MM:SS AM/PM
);

```

```

-- Populate the time dimension for each minute of the day
INSERT INTO time_dimension
SELECT
    TO_CHAR(tm, 'HH24MI')::INTEGER AS time_key,
    tm::TIME AS time_value,
    EXTRACT(HOUR FROM tm) AS hour_24,
    CASE
        WHEN EXTRACT(HOUR FROM tm) % 12 = 0 THEN 12
        ELSE EXTRACT(HOUR FROM tm) % 12
    END AS hour_12,
    CASE
        WHEN EXTRACT(HOUR FROM tm) < 12 THEN 'AM'
        ELSE 'PM'
    END AS am_pm,
    EXTRACT(MINUTE FROM tm) AS minute,
    EXTRACT(SECOND FROM tm) AS second,

    CASE
        WHEN EXTRACT(HOUR FROM tm) BETWEEN 5 AND 11 THEN 'Morning'
        WHEN EXTRACT(HOUR FROM tm) BETWEEN 12 AND 16 THEN 'Afternoon'
        WHEN EXTRACT(HOUR FROM tm) BETWEEN 17 AND 20 THEN 'Evening'
        ELSE 'Night'
    END AS day_period,

    EXTRACT(HOUR FROM tm) BETWEEN 9 AND 17 AS business_hour,

    (EXTRACT(HOUR FROM tm) BETWEEN 11 AND 13) OR
    (EXTRACT(HOUR FROM tm) BETWEEN 17 AND 19) AS peak_hour,

    TO_CHAR(tm, 'HH24:MI:SS') AS time_format_24,
    TO_CHAR(tm, 'HH12:MI:SS AM') AS time_format_12
FROM
    generate_series(
        '2000-01-01 00:00:00'::timestamp,
        '2000-01-01 23:59:00'::timestamp,
        '1 minute'::interval
    ) AS tm;

```

Combining Date and Time Dimensions To analyze both date and time components together:

```

-- Creating a fact table with both date and time keys
CREATE TABLE transaction_fact (
    transaction_id INTEGER PRIMARY KEY,
    date_key INTEGER REFERENCES date_dimension(date_key),

```

```

    time_key INTEGER REFERENCES time_dimension(time_key),
    store_key INTEGER REFERENCES store_dimension(store_key),
    product_key INTEGER REFERENCES product_dimension(product_key),
    customer_key INTEGER REFERENCES customer_dimension(customer_key),
    quantity INTEGER,
    sales_amount NUMERIC(12,2)
);

```

-- Analyzing sales by time period throughout the week

```

SELECT
    d.day_of_week_name,
    t.day_period,
    COUNT(*) AS transaction_count,
    SUM(f.sales_amount) AS total_sales
FROM
    transaction_fact f
JOIN
    date_dimension d ON f.date_key = d.date_key
JOIN
    time_dimension t ON f.time_key = t.time_key
WHERE
    d.month_number = 3 -- March
    AND d.year = 2023
GROUP BY
    d.day_of_week, d.day_of_week_name, t.day_period
ORDER BY
    d.day_of_week,
    CASE
        WHEN t.day_period = 'Morning' THEN 1
        WHEN t.day_period = 'Afternoon' THEN 2
        WHEN t.day_period = 'Evening' THEN 3
        WHEN t.day_period = 'Night' THEN 4
    END;

```

-- Identifying peak business hours by product category

```

SELECT
    p.category,
    t.hour_24,
    t.am_pm,
    COUNT(*) AS transaction_count,
    SUM(f.quantity) AS units_sold,
    SUM(f.sales_amount) AS total_sales
FROM
    transaction_fact f
JOIN
    product_dimension p ON f.product_key = p.product_key

```

```

JOIN
    time_dimension t ON f.time_key = t.time_key
JOIN
    date_dimension d ON f.date_key = d.date_key
WHERE
    d.is_business_day = TRUE
    AND d.year = 2023
GROUP BY
    p.category, t.hour_24, t.am_pm
ORDER BY
    p.category, total_sales DESC;

```

Holiday and Special Event Analysis

```

-- Impact of holidays on sales
SELECT
    d.holiday_name,
    SUM(f.sales_amount) AS total_sales,
    AVG(f.sales_amount) AS avg_transaction_value,
    COUNT(*) AS transaction_count
FROM
    sales_fact f
JOIN
    date_dimension d ON f.date_key = d.date_key
WHERE
    d.is_holiday = TRUE
    AND d.year = 2023
GROUP BY
    d.holiday_name
ORDER BY
    total_sales DESC;

-- Comparing Black Friday to regular Fridays
SELECT
    CASE
        WHEN d.is_black_friday THEN 'Black Friday'
        ELSE 'Regular Friday'
    END AS friday_type,
    COUNT(DISTINCT d.date_key) AS day_count,
    SUM(f.sales_amount) AS total_sales,
    SUM(f.sales_amount) / COUNT(DISTINCT d.date_key) AS avg_daily_sales,
    COUNT(*) / COUNT(DISTINCT d.date_key) AS avg_daily_transactions
FROM
    sales_fact f
JOIN
    date_dimension d ON f.date_key = d.date_key

```



```

WHERE
    d.day_of_week_name = 'Friday'
    AND d.year = 2023
GROUP BY
    CASE WHEN d.is_black_friday THEN 'Black Friday' ELSE 'Regular Friday' END;

-- Holiday season vs. non-holiday season performance by category
SELECT
    p.category,
    CASE WHEN d.is_holiday_season THEN 'Holiday Season' ELSE 'Regular Season' END AS season,
    COUNT(DISTINCT d.date_key) AS number_of_days,
    SUM(f.sales_amount) AS total_sales,
    SUM(f.sales_amount) / COUNT(DISTINCT d.date_key) AS avg_daily_sales
FROM
    sales_fact f
JOIN
    date_dimension d ON f.date_key = d.date_key
JOIN
    product_dimension p ON f.product_key = p.product_key
WHERE
    d.year = 2023
GROUP BY
    p.category,
    CASE WHEN d.is_holiday_season THEN 'Holiday Season' ELSE 'Regular Season' END
ORDER BY
    p.category,
    CASE WHEN season_type = 'Holiday Season' THEN 1 ELSE 2 END;

```

SQL for Analytics: A Comprehensive Guide

Part IV: Performance Optimization

1. Indexes and Query Plans

Optimizing query performance is critical for analytics, especially when dealing with large datasets. Understanding how to interpret execution plans and use indexes effectively is an essential skill.

Index Types for Analytics While transaction-oriented databases primarily use B-tree indexes, analytical databases employ a wider variety of index types optimized for different query patterns:

```

-- B-tree index (standard index type)
-- Good for: equality, range queries, and sorting
CREATE INDEX idx_product_category
ON product_dimension(category);

```

```

-- Bitmap index
-- Good for: low cardinality columns (columns with few unique values)
-- Note: Explicitly available in some databases like Oracle and PostgreSQL
CREATE INDEX idx_product_category_bitmap
ON product_dimension USING BITMAP(category);

-- Covering index (includes additional columns)
-- Good for: avoiding table lookups by including all needed columns
CREATE INDEX idx_sales_by_date_product
ON sales_fact(date_key, product_key)
INCLUDE (sales_amount);

-- Partial/filtered index (only indexes a subset of rows)
-- Good for: queries that always filter on the same condition
CREATE INDEX idx_high_value_sales
ON sales_fact(date_key, customer_key)
WHERE sales_amount > 1000;

-- Expression index (indexes a computed value)
-- Good for: queries that use functions or expressions
CREATE INDEX idx_sale_month
ON sales_fact(EXTRACT(MONTH FROM transaction_date));

-- GIN (Generalized Inverted Index)
-- Good for: full-text search and array operations
CREATE INDEX idx_product_description_search
ON product_dimension USING GIN (to_tsvector('english', product_description));

```

Understanding EXPLAIN Output The EXPLAIN command shows how the database will execute your query. For analytics, understanding these plans is crucial for optimizing long-running queries:

```

-- Basic EXPLAIN
EXPLAIN
SELECT
    p.category,
    COUNT(*) as sale_count,
    SUM(s.sales_amount) as total_sales
FROM
    sales_fact s
JOIN
    product_dimension p ON s.product_key = p.product_key
WHERE
    s.date_key BETWEEN 20230101 AND 20230131
GROUP BY

```

```

        p.category;

-- Sample output:
/*
HashAggregate (cost=1234.56..1235.67 rows=89 width=24)
  Group Key: p.category
    -> Hash Join (cost=87.65..1122.33 rows=10000 width=16)
        Hash Cond: (s.product_key = p.product_key)
        -> Seq Scan on sales_fact s (cost=0.00..877.00 rows=10000 width=12)
            Filter: ((date_key >= 20230101) AND (date_key <= 20230131))
        -> Hash (cost=54.00..54.00 rows=2692 width=12)
            -> Seq Scan on product_dimension p (cost=0.00..54.00 rows=2692 width=12)

*/

-- EXPLAIN with execution statistics
EXPLAIN ANALYZE
SELECT
    p.category,
    COUNT(*) as sale_count,
    SUM(s.sales_amount) as total_sales
FROM
    sales_fact s
JOIN
    product_dimension p ON s.product_key = p.product_key
WHERE
    s.date_key BETWEEN 20230101 AND 20230131
GROUP BY
    p.category;

-- Sample output with runtime information:
/*
HashAggregate (cost=1234.56..1235.67 rows=89 width=24) (actual time=22.548..22.592 rows=15)
  Group Key: p.category
    -> Hash Join (cost=87.65..1122.33 rows=10000 width=16) (actual time=0.978..19.462 rows=)
        Hash Cond: (s.product_key = p.product_key)
        -> Seq Scan on sales_fact s (cost=0.00..877.00 rows=10000 width=12) (actual time=)
            Filter: ((date_key >= 20230101) AND (date_key <= 20230131))
            Rows Removed by Filter: 23432
        -> Hash (cost=54.00..54.00 rows=2692 width=12) (actual time=0.909..0.909 rows=2692)
            Buckets: 4096 Batches: 1 Memory Usage: 175kB
            -> Seq Scan on product_dimension p (cost=0.00..54.00 rows=2692 width=12) (a

Planning Time: 0.654 ms
Execution Time: 22.744 ms
*/

```

Common Plan Operations and What They Mean Understanding the operations in a query plan helps identify optimization opportunities:

```
-- Sequential Scan: Full table scan
EXPLAIN
SELECT * FROM large_table WHERE column1 > 1000;
-- Output contains: "Seq Scan on large_table"

-- Index Scan: Uses an index to find rows, then fetches data
EXPLAIN
SELECT * FROM large_table WHERE indexed_column = 'value';
-- Output might contain: "Index Scan using idx_column on large_table"

-- Index Only Scan: Gets all data directly from the index
EXPLAIN
SELECT indexed_column FROM large_table WHERE indexed_column = 'value';
-- Output might contain: "Index Only Scan using idx_column on large_table"

-- Bitmap Index Scan: Uses a bitmap to find matching rows
EXPLAIN
SELECT * FROM large_table WHERE category IN ('A', 'B', 'C');
-- Output might contain: "Bitmap Index Scan on idx_category"

-- Hash Join: Build a hash table from one side, probe with the other
EXPLAIN
SELECT * FROM table1 JOIN table2 ON table1.id = table2.id;
-- Output might contain: "Hash Join" with "Hash" and "Hash Cond"

-- Merge Join: Merge two sorted inputs
EXPLAIN
SELECT * FROM table1 JOIN table2 ON table1.id = table2.id
ORDER BY table1.id;
-- Output might contain: "Merge Join" with "Sort" operations

-- Nested Loop: For each row from one table, scan the other table
EXPLAIN
SELECT * FROM small_table s JOIN large_table l ON s.id = l.id;
-- Output might contain: "Nested Loop"

-- Aggregate: Group and calculate aggregate functions
EXPLAIN
SELECT category, SUM(amount) FROM table GROUP BY category;
-- Output might contain: "HashAggregate" or "GroupAggregate"

-- Sort: Sort the result set
EXPLAIN
```

```

SELECT * FROM table ORDER BY column;
-- Output might contain: "Sort"

-- Limit: Restrict the number of returned rows
EXPLAIN
SELECT * FROM table LIMIT 10;
-- Output might contain: "Limit"

```

Comparing Different Query Plans Let's examine how the same analytical query can have different execution plans:

-- Query: Find total sales by category for January 2023

-- Version 1: Simple Query

```

EXPLAIN ANALYZE
SELECT
    p.category,
    SUM(s.sales_amount) as total_sales
FROM
    sales_fact s
JOIN
    product_dimension p ON s.product_key = p.product_key
WHERE
    s.date_key BETWEEN 20230101 AND 20230131
GROUP BY
    p.category;

```

-- Sample output:

```

/*
HashAggregate (cost=1234.56..1235.67 rows=15 width=24) (actual time=22.548..22.592 rows=15)
  Group Key: p.category
    -> Hash Join (cost=87.65..1122.33 rows=11204 width=16) (actual time=0.978..19.462 rows=11204)
      Hash Cond: (s.product_key = p.product_key)
        -> Seq Scan on sales_fact s (cost=0.00..877.00 rows=11204 width=12) (actual time=0.978..19.462 rows=11204)
          Filter: ((date_key >= 20230101) AND (date_key <= 20230131))
          Rows Removed by Filter: 23432
        -> Hash (cost=54.00..54.00 rows=2692 width=12) (actual time=0.909..0.909 rows=2692)
          Buckets: 4096 Batches: 1 Memory Usage: 175kB
          -> Seq Scan on product_dimension p (cost=0.00..54.00 rows=2692 width=12) (actual time=0.909..0.909 rows=2692)
Planning Time: 0.654 ms
Execution Time: 22.744 ms
*/

```

-- Version 2: Pre-aggregation in a CTE

```

EXPLAIN ANALYZE
WITH date_filtered_sales AS (

```

```

        SELECT
            product_key,
            SUM(sales_amount) as total_sales
        FROM
            sales_fact
        WHERE
            date_key BETWEEN 20230101 AND 20230131
        GROUP BY
            product_key
    )
SELECT
    p.category,
    SUM(s.total_sales) as total_sales
FROM
    date_filtered_sales s
JOIN
    product_dimension p ON s.product_key = p.product_key
GROUP BY
    p.category;

-- Sample output:
/*
HashAggregate (cost=923.45..924.56 rows=15 width=24) (actual time=19.332..19.376 rows=15 l
  Group Key: p.category
    -> Hash Join (cost=787.65..899.33 rows=1204 width=16) (actual time=10.978..17.462 rows=
      Hash Cond: (s.product_key = p.product_key)
        -> HashAggregate (cost=700.00..725.00 rows=1204 width=12) (actual time=10.014..12.
          Group Key: sales_fact.product_key
            -> Seq Scan on sales_fact (cost=0.00..600.00 rows=11204 width=12) (actual t
              Filter: ((date_key >= 20230101) AND (date_key <= 20230131))
              Rows Removed by Filter: 23432
            -> Hash (cost=54.00..54.00 rows=2692 width=12) (actual time=0.909..0.909 rows=2692
              Buckets: 4096 Batches: 1 Memory Usage: 175kB
            -> Seq Scan on product_dimension p (cost=0.00..54.00 rows=2692 width=12) (a
Planning Time: 0.712 ms
Execution Time: 19.513 ms
*/

-- Version 3: Using a pre-created index
-- Assume we created: CREATE INDEX idx_sales_date_product ON sales_fact(date_key, product_k
EXPLAIN ANALYZE
SELECT
    p.category,
    SUM(s.sales_amount) as total_sales
FROM
    sales_fact s

```

```

JOIN
    product_dimension p ON s.product_key = p.product_key
WHERE
    s.date_key BETWEEN 20230101 AND 20230131
GROUP BY
    p.category;

-- Sample output with index:
/*
HashAggregate (cost=723.45..724.56 rows=15 width=24) (actual time=15.548..15.592 rows=15 l
Group Key: p.category
-> Hash Join (cost=87.65..622.33 rows=11204 width=16) (actual time=0.978..12.462 rows=1
    Hash Cond: (s.product_key = p.product_key)
    -> Index Scan using idx_sales_date_product on sales_fact s (cost=0.00..377.00 rows=
        Index Cond: ((date_key >= 20230101) AND (date_key <= 20230131))
    -> Hash (cost=54.00..54.00 rows=2692 width=12) (actual time=0.909..0.909 rows=2692
        Buckets: 4096 Batches: 1 Memory Usage: 175kB
        -> Seq Scan on product_dimension p (cost=0.00..54.00 rows=2692 width=12) (a
Planning Time: 0.654 ms
Execution Time: 15.744 ms
*/

```

Analyzing Join Order Impact One of the most important aspects of query optimization is join order. Let's examine how it affects performance:

```

-- Query: Sales by store, product, and date

-- Version 1: Optimizer-determined join order
EXPLAIN
SELECT
    s.store_name,
    p.product_name,
    d.month_name,
    SUM(f.sales_amount) as total_sales
FROM
    sales_fact f
JOIN
    date_dimension d ON f.date_key = d.date_key
JOIN
    product_dimension p ON f.product_key = p.product_key
JOIN
    store_dimension s ON f.store_key = s.store_key
WHERE
    d.year = 2023
    AND s.region = 'Northeast'
    AND p.category = 'Electronics'

```

```

GROUP BY
    s.store_name, p.product_name, d.month_name;

-- Version 2: Explicitly ordered joins (start with the most filtered table)
EXPLAIN
SELECT
    s.store_name,
    p.product_name,
    d.month_name,
    SUM(f.sales_amount) as total_sales
FROM
    store_dimension s
JOIN
    sales_fact f ON s.store_key = f.store_key
JOIN
    product_dimension p ON f.product_key = p.product_key
JOIN
    date_dimension d ON f.date_key = d.date_key
WHERE
    d.year = 2023
    AND s.region = 'Northeast'
    AND p.category = 'Electronics'
GROUP BY
    s.store_name, p.product_name, d.month_name;

```

Examining the Impact of WHERE Clause Order The order of conditions in a WHERE clause can impact the execution plan:

-- Query: Find high-value electronics sales in 2023

```

-- Version 1: Date filter first
EXPLAIN ANALYZE
SELECT
    p.product_name,
    SUM(f.sales_amount) as total_sales
FROM
    sales_fact f
JOIN
    product_dimension p ON f.product_key = p.product_key
JOIN
    date_dimension d ON f.date_key = d.date_key
WHERE
    d.year = 2023
    AND p.category = 'Electronics'
    AND f.sales_amount > 1000
GROUP BY

```



```

        p.product_name;

-- Version 2: Category filter first
EXPLAIN ANALYZE
SELECT
    p.product_name,
    SUM(f.sales_amount) as total_sales
FROM
    sales_fact f
JOIN
    product_dimension p ON f.product_key = p.product_key
JOIN
    date_dimension d ON f.date_key = d.date_key
WHERE
    p.category = 'Electronics'
    AND d.year = 2023
    AND f.sales_amount > 1000
GROUP BY
    p.product_name;

```

Understanding Cardinality and Selectivity For analytics workloads, knowing how many rows your queries will process is crucial:

```

-- Examine the actual vs. estimated row counts
EXPLAIN ANALYZE
SELECT
    p.category,
    COUNT(*) as product_count
FROM
    product_dimension p
WHERE
    p.retail_price > 100
GROUP BY
    p.category;

-- Collect statistics on the table to improve estimates
ANALYZE product_dimension;

-- Re-run the explain to see updated estimates
EXPLAIN ANALYZE
SELECT
    p.category,
    COUNT(*) as product_count
FROM
    product_dimension p
WHERE

```

```

        p.retail_price > 100
GROUP BY
    p.category;

```

2. Query Optimization Techniques

Beyond understanding execution plans, there are specific techniques to optimize analytical queries.

Filtering Early Reducing the amount of data processed as early as possible is one of the most effective optimization strategies:

```

-- Inefficient: Filtering after the join
EXPLAIN
SELECT
    c.customer_name,
    SUM(s.sales_amount) as total_sales
FROM
    sales_fact s
JOIN
    customer_dimension c ON s.customer_key = c.customer_key
WHERE
    s.date_key BETWEEN 20230101 AND 20230131
    AND c.customer_segment = 'Enterprise'
GROUP BY
    c.customer_name;

-- More efficient: Pre-filtering the tables before joining
EXPLAIN
WITH filtered_sales AS (
    SELECT
        customer_key,
        sales_amount
    FROM
        sales_fact
    WHERE
        date_key BETWEEN 20230101 AND 20230131
),
filtered_customers AS (
    SELECT
        customer_key,
        customer_name
    FROM
        customer_dimension
    WHERE
        customer_segment = 'Enterprise'

```

```

)
SELECT
    c.customer_name,
    SUM(s.sales_amount) as total_sales
FROM
    filtered_sales s
JOIN
    filtered_customers c ON s.customer_key = c.customer_key
GROUP BY
    c.customer_name;

```

Aggregating Early For analytical queries, reducing data volume through early aggregation can significantly improve performance:

-- Inefficient: Joining first, then aggregating

```

EXPLAIN
SELECT
    d.year,
    d.quarter,
    p.category,
    SUM(s.sales_amount) as total_sales
FROM
    sales_fact s
JOIN
    date_dimension d ON s.date_key = d.date_key
JOIN
    product_dimension p ON s.product_key = p.product_key
WHERE
    d.year BETWEEN 2020 AND 2023
GROUP BY
    d.year, d.quarter, p.category;

```

-- More efficient: Aggregate before joining when possible

```

EXPLAIN
WITH sales_by_date_product AS (
    SELECT
        date_key,
        product_key,
        SUM(sales_amount) as total_sales
    FROM
        sales_fact
    GROUP BY
        date_key, product_key
)
SELECT
    d.year,

```

```

        d.quarter,
        p.category,
        SUM(s.total_sales) as total_sales
FROM
    sales_by_date_product s
JOIN
    date_dimension d ON s.date_key = d.date_key
JOIN
    product_dimension p ON s.product_key = p.product_key
WHERE
    d.year BETWEEN 2020 AND 2023
GROUP BY
    d.year, d.quarter, p.category;

```

Materialized Views for Analytics Materialized views pre-compute and store the results of a query for faster access:

```

-- Create a materialized view for monthly sales by product category
CREATE MATERIALIZED VIEW monthly_category_sales AS
SELECT
    d.year,
    d.month_number,
    p.category,
    SUM(s.sales_amount) as total_sales,
    COUNT(*) as transaction_count
FROM
    sales_fact s
JOIN
    date_dimension d ON s.date_key = d.date_key
JOIN
    product_dimension p ON s.product_key = p.product_key
GROUP BY
    d.year, d.month_number, p.category;

-- Create an index on the materialized view for faster queries
CREATE INDEX idx_mv_monthly_sales
ON monthly_category_sales(year, month_number, category);

-- Use the materialized view
EXPLAIN
SELECT
    year,
    month_number,
    category,
    total_sales
FROM

```

```

        monthly_category_sales
WHERE
    year = 2023
    AND month_number BETWEEN 1 AND 3
ORDER BY
    month_number, total_sales DESC;

-- Refresh the materialized view when source data changes
REFRESH MATERIALIZED VIEW monthly_category_sales;

```

Query Rewriting Techniques Sometimes, restructuring a query can dramatically improve performance:

```

-- Original query with multiple joins and subqueries
EXPLAIN
SELECT
    c.customer_name,
    SUM(s.sales_amount) as total_sales,
    (
        SELECT AVG(sales_amount)
        FROM sales_fact
        WHERE date_key BETWEEN 20230101 AND 20230131
    ) as average_sale
FROM
    sales_fact s
JOIN
    customer_dimension c ON s.customer_key = c.customer_key
WHERE
    s.date_key BETWEEN 20230101 AND 20230131
    AND s.sales_amount > (
        SELECT AVG(sales_amount) * 2
        FROM sales_fact
        WHERE date_key BETWEEN 20230101 AND 20230131
    )
GROUP BY
    c.customer_name;

-- Rewritten query with CTEs to avoid repeated subqueries
EXPLAIN
WITH period_stats AS (
    SELECT
        AVG(sales_amount) as avg_sale,
        AVG(sales_amount) * 2 as high_value_threshold
    FROM
        sales_fact
    WHERE

```

```

        date_key BETWEEN 20230101 AND 20230131
    )
SELECT
    c.customer_name,
    SUM(s.sales_amount) as total_sales,
    ps.avg_sale as average_sale
FROM
    sales_fact s
JOIN
    customer_dimension c ON s.customer_key = c.customer_key
CROSS JOIN
    period_stats ps
WHERE
    s.date_key BETWEEN 20230101 AND 20230131
    AND s.sales_amount > ps.high_value_threshold
GROUP BY
    c.customer_name, ps.avg_sale;

```

Using Window Functions Instead of Self-Joins Window functions can often replace complex self-joins for analytical calculations:

-- Inefficient: Self-join to calculate year-over-year growth

```

EXPLAIN
SELECT
    current.year,
    current.month_number,
    current.category,
    current.total_sales,
    previous.total_sales as prev_year_sales,
    (current.total_sales - previous.total_sales) / previous.total_sales * 100 as yoy_growth
FROM
    monthly_category_sales current
LEFT JOIN
    monthly_category_sales previous
    ON current.month_number = previous.month_number
    AND current.category = previous.category
    AND previous.year = current.year - 1
WHERE
    current.year = 2023;

```

-- More efficient: Using window functions

```

EXPLAIN
SELECT
    year,
    month_number,
    category,

```

```

total_sales,
LAG(total_sales, 12) OVER (
    PARTITION BY category, month_number
    ORDER BY year, month_number
) as prev_year_sales,
CASE
    WHEN LAG(total_sales, 12) OVER (
        PARTITION BY category, month_number
        ORDER BY year, month_number
    ) IS NULL OR LAG(total_sales, 12) OVER (
        PARTITION BY category, month_number
        ORDER BY year, month_number
    ) = 0
    THEN NULL
    ELSE (total_sales - LAG(total_sales, 12) OVER (
        PARTITION BY category, month_number
        ORDER BY year, month_number
    )) / LAG(total_sales, 12) OVER (
        PARTITION BY category, month_number
        ORDER BY year, month_number
    ) * 100
END as yoy_growth
FROM
    monthly_category_sales
WHERE
    year = 2023;

```

Using EXISTS Instead of IN for Subquery Filtering For large datasets, EXISTS can perform better than IN with subqueries:

```

-- Using IN with a subquery
EXPLAIN
SELECT
    p.product_id,
    p.product_name,
    p.category
FROM
    product_dimension p
WHERE
    p.product_id IN (
        SELECT DISTINCT product_id
        FROM sales_fact
        WHERE date_key BETWEEN 20230101 AND 20230131
    );

-- Using EXISTS instead

```

```

EXPLAIN
SELECT
    p.product_id,
    p.product_name,
    p.category
FROM
    product_dimension p
WHERE
    EXISTS (
        SELECT 1
        FROM sales_fact s
        WHERE s.product_id = p.product_id
        AND s.date_key BETWEEN 20230101 AND 20230131
    );

```

Query Hints Some databases allow query hints to override the optimizer's decisions. Use these sparingly and only when necessary:

```

-- PostgreSQL: Force a specific join order
SELECT /* Leading(dimension1 fact dimension2) */
    d1.attribute,
    d2.attribute,
    SUM(f.measure)
FROM
    dimension1 d1
JOIN
    fact f ON d1.key = f.dimension1_key
JOIN
    dimension2 d2 ON f.dimension2_key = d2.key
GROUP BY
    d1.attribute, d2.attribute;

-- PostgreSQL: Disable specific join types
SELECT /* NoHashJoin(f d1) NoMergeJoin(f d2) */
    d1.attribute,
    d2.attribute,
    SUM(f.measure)
FROM
    fact f
JOIN
    dimension1 d1 ON f.dimension1_key = d1.key
JOIN
    dimension2 d2 ON f.dimension2_key = d2.key
GROUP BY
    d1.attribute, d2.attribute;

```


3. Identifying and Resolving Bottlenecks

Even with well-designed queries, performance issues can arise. Knowing how to identify and resolve bottlenecks is essential.

Identifying Slow Queries First, you need to find problematic queries:

```
-- PostgreSQL: Find slow queries from logs
SELECT
    substring(query, 1, 100) as query_snippet,
    calls,
    total_time,
    mean_time,
    max_time
FROM
    pg_stat_statements
ORDER BY
    total_time DESC
LIMIT 20;

-- PostgreSQL: Find currently running long queries
SELECT
    pid,
    now() - query_start as duration,
    query
FROM
    pg_stat_activity
WHERE
    state = 'active'
    AND now() - query_start > interval '5 minutes'
ORDER BY
    duration DESC;
```

Common Bottlenecks in Analytical Queries Let's examine some common issues and solutions:

1. Inefficient Joins

```
-- Problem: Cartesian join (missing join condition)
EXPLAIN
SELECT
    c.customer_name,
    p.product_name,
    SUM(s.sales_amount) as total_sales
FROM
    sales_fact s,
    customer_dimension c,
```

```

        product_dimension p
WHERE
    s.customer_key = c.customer_key
    -- Missing join condition for product_dimension
GROUP BY
    c.customer_name, p.product_name;

-- Solution: Add proper join conditions
EXPLAIN
SELECT
    c.customer_name,
    p.product_name,
    SUM(s.sales_amount) as total_sales
FROM
    sales_fact s
JOIN
    customer_dimension c ON s.customer_key = c.customer_key
JOIN
    product_dimension p ON s.product_key = p.product_key
GROUP BY
    c.customer_name, p.product_name;

```

2. Too Many Joins

```

-- Problem: Excessive joins for the required analysis
EXPLAIN
SELECT
    d.year,
    d.quarter,
    SUM(s.sales_amount) as total_sales
FROM
    sales_fact s
JOIN
    date_dimension d ON s.date_key = d.date_key
JOIN
    product_dimension p ON s.product_key = p.product_key
JOIN
    customer_dimension c ON s.customer_key = c.customer_key
JOIN
    store_dimension st ON s.store_key = st.store_key
JOIN
    promotion_dimension pr ON s.promotion_key = pr.promotion_key
WHERE
    d.year = 2023
GROUP BY
    d.year, d.quarter;

```

-- Solution: Only join the necessary tables

```
EXPLAIN
SELECT
    d.year,
    d.quarter,
    SUM(s.sales_amount) as total_sales
FROM
    sales_fact s
JOIN
    date_dimension d ON s.date_key = d.date_key
WHERE
    d.year = 2023
GROUP BY
    d.year, d.quarter;
```

3. Inefficient GROUP BY Operations

-- Problem: Grouping by high-cardinality columns

```
EXPLAIN
SELECT
    p.product_id, -- High cardinality
    SUM(s.sales_amount) as total_sales
FROM
    sales_fact s
JOIN
    product_dimension p ON s.product_key = p.product_key
GROUP BY
    p.product_id;
```

-- Solution: Group by appropriate business level

```
EXPLAIN
SELECT
    p.category, -- Lower cardinality
    SUM(s.sales_amount) as total_sales
FROM
    sales_fact s
JOIN
    product_dimension p ON s.product_key = p.product_key
GROUP BY
    p.category;
```

4. Suboptimal Function Usage

-- Problem: Functions in WHERE clause prevent index usage

```
EXPLAIN
SELECT
    customer_key,
    sales_amount
```

```

FROM
    sales_fact
WHERE
    EXTRACT(YEAR FROM transaction_date) = 2023;

-- Solution: Rewrite to allow index usage
EXPLAIN
SELECT
    customer_key,
    sales_amount
FROM
    sales_fact
WHERE
    transaction_date >= '2023-01-01' AND
    transaction_date < '2024-01-01';

-- Alternatively, use a function-based index
CREATE INDEX idx_sales_year
ON sales_fact(EXTRACT(YEAR FROM transaction_date));

```

5. Insufficient Memory for Operations

```

-- Problem: Large sort or hash operations might exceed work_mem
EXPLAIN
SELECT
    c.customer_id,
    p.product_id,
    SUM(s.sales_amount) as total_sales
FROM
    sales_fact s
JOIN
    customer_dimension c ON s.customer_key = c.customer_key
JOIN
    product_dimension p ON s.product_key = p.product_key
GROUP BY
    c.customer_id, p.product_id;

-- Solution: Break into smaller queries or increase work_mem temporarily
SET work_mem = '256MB'; -- Adjust based on your server's capacity

-- Alternative solution: Use CTEs to break down the operations
WITH customer_sales AS (
    SELECT
        customer_key,
        product_key,
        SUM(sales_amount) as total_sales
    FROM

```

```

        sales_fact
    GROUP BY
        customer_key, product_key
)
SELECT
    c.customer_id,
    p.product_id,
    cs.total_sales
FROM
    customer_sales cs
JOIN
    customer_dimension c ON cs.customer_key = c.customer_key
JOIN
    product_dimension p ON cs.product_key = p.product_key;

```

Using EXPLAIN to Identify Issues Let's see how to identify specific problems using EXPLAIN output:

```

-- Identifying a sequential scan when an index should be used
EXPLAIN
SELECT * FROM large_table WHERE indexed_column = 'value';
-- Look for "Seq Scan" instead of "Index Scan"

-- Identifying a hash join when a nested loop would be better for small datasets
EXPLAIN
SELECT * FROM small_table s JOIN large_table l ON s.id = l.id;
-- Look for "Hash Join" instead of "Nested Loop"

-- Identifying poor row count estimates
EXPLAIN
SELECT * FROM table WHERE column1 = 'rare_value';
-- Compare estimated vs. actual rows after running EXPLAIN ANALYZE

-- Identifying a sort operation when using an index could avoid it
EXPLAIN
SELECT * FROM table ORDER BY column1;
-- Look for "Sort" operation instead of "Index Scan using idx_column1"

```

Performance Tuning Solutions 1. Creating Appropriate Indexes

```

-- For common equality filters
CREATE INDEX idx_product_category ON product_dimension(category);

-- For range queries
CREATE INDEX idx_sales_date ON sales_fact(transaction_date);

```

```

-- For common joins
CREATE INDEX idx_sales_product_key ON sales_fact(product_key);

-- Multi-column indexes for compound conditions
CREATE INDEX idx_sales_date_product
ON sales_fact(date_key, product_key);

-- Covering indexes for frequent analytical queries
CREATE INDEX idx_sales_analysis
ON sales_fact(date_key, product_key)
INCLUDE (sales_amount);

-- Indexes for GROUP BY columns
CREATE INDEX idx_sales_store_date
ON sales_fact(store_key, date_key);

```

2. Partitioning Tables

Partitioning large fact tables can significantly improve performance:

```

-- Create a partitioned table by date
CREATE TABLE sales_fact (
    sale_id INTEGER,
    date_key INTEGER,
    product_key INTEGER,
    customer_key INTEGER,
    sales_amount NUMERIC(12,2),
    -- other columns
    PRIMARY KEY (sale_id, date_key)
) PARTITION BY RANGE (date_key);

-- Create partitions for each year
CREATE TABLE sales_fact_2021
PARTITION OF sales_fact
FOR VALUES FROM (20210101) TO (20220101);

CREATE TABLE sales_fact_2022
PARTITION OF sales_fact
FOR VALUES FROM (20220101) TO (20230101);

CREATE TABLE sales_fact_2023
PARTITION OF sales_fact
FOR VALUES FROM (20230101) TO (20240101);

-- Create indexes on each partition
CREATE INDEX idx_sales_2023_product
ON sales_fact_2023(product_key);

```

```

-- Query will automatically use the relevant partition
EXPLAIN
SELECT
    p.category,
    SUM(s.sales_amount) as total_sales
FROM
    sales_fact s
JOIN
    product_dimension p ON s.product_key = p.product_key
WHERE
    s.date_key BETWEEN 20230101 AND 20230331
GROUP BY
    p.category;

```

3. Materialized Views and Summary Tables

Pre-aggregating common analytical queries:

```

-- Create a summary table for daily sales by product category
CREATE TABLE daily_category_sales AS
SELECT
    s.date_key,
    p.category,
    SUM(s.sales_amount) as total_sales,
    COUNT(*) as transaction_count
FROM
    sales_fact s
JOIN
    product_dimension p ON s.product_key = p.product_key
GROUP BY
    s.date_key, p.category;

-- Create indexes on the summary table
CREATE INDEX idx_daily_category_sales_date
ON daily_category_sales(date_key);

CREATE INDEX idx_daily_category_sales_category
ON daily_category_sales(category);

-- Query the summary table instead of base tables
SELECT
    d.month_name,
    dcs.category,
    SUM(dcs.total_sales) as monthly_sales
FROM
    daily_category_sales dcs

```

```

JOIN
    date_dimension d ON dcs.date_key = d.date_key
WHERE
    d.year = 2023
GROUP BY
    d.month_name, dcs.category
ORDER BY
    d.month_name, monthly_sales DESC;

```

4. Query Restructuring

Sometimes, rewriting the query logic can improve performance:

```

-- Original query with complex joins and filtering
EXPLAIN
SELECT
    c.customer_segment,
    p.category,
    d.quarter,
    SUM(f.sales_amount) as total_sales
FROM
    sales_fact f
JOIN
    customer_dimension c ON f.customer_key = c.customer_key
JOIN
    product_dimension p ON f.product_key = p.product_key
JOIN
    date_dimension d ON f.date_key = d.date_key
WHERE
    d.year = 2023
    AND c.customer_segment IN ('Enterprise', 'SMB')
    AND p.category IN ('Electronics', 'Furniture')
GROUP BY
    c.customer_segment, p.category, d.quarter;

-- Restructured query with pre-filtering and explicit join order
EXPLAIN
WITH filtered_dates AS (
    SELECT date_key, quarter
    FROM date_dimension
    WHERE year = 2023
),
filtered_customers AS (
    SELECT customer_key, customer_segment
    FROM customer_dimension
    WHERE customer_segment IN ('Enterprise', 'SMB')
),

```



```

filtered_products AS (
    SELECT product_key, category
    FROM product_dimension
    WHERE category IN ('Electronics', 'Furniture')
)
SELECT
    c.customer_segment,
    p.category,
    d.quarter,
    SUM(f.sales_amount) as total_sales
FROM
    filtered_customers c
JOIN
    sales_fact f ON c.customer_key = f.customer_key
JOIN
    filtered_products p ON f.product_key = p.product_key
JOIN
    filtered_dates d ON f.date_key = d.date_key
GROUP BY
    c.customer_segment, p.category, d.quarter;

```

Part V: Distributed SQL and Large-Scale Analytics

1. Columnar Storage Principles

Modern data warehouses and analytics platforms use columnar storage rather than the row-based storage found in traditional transactional databases. Understanding these principles is essential for optimizing large-scale analytics.

Row-Based vs. Columnar Storage

```

-- Conceptual comparison (not executable SQL)

-- Row-based storage (traditional OLTP databases)
-- Stores all columns of a row together
TABLE sales_fact (
    /* Physically stored as:
    [sale_id=1, date_key=20230101, product_key=101, sales_amount=125.50],
    [sale_id=2, date_key=20230101, product_key=205, sales_amount=89.99],
    ...
    */
);

-- Columnar storage (analytical databases)
-- Stores all values of a column together
TABLE sales_fact (
    /* Physically stored as:

```

```

    sale_id:      [1, 2, 3, 4, 5, ...],
    date_key:     [20230101, 20230101, 20230101, 20230102, 20230102, ...],
    product_key:  [101, 205, 308, 101, 205, ...],
    sales_amount: [125.50, 89.99, 45.25, 130.00, 92.50, ...]
  */
);

```

Benefits of Columnar Storage for Analytics

1. **Data Compression:** Values in the same column often have similar characteristics, allowing for better compression.
2. **I/O Efficiency:** Analytics queries often only need a subset of columns, so you only read what you need.
3. **Vectorized Processing:** Operations can be performed on entire columns at once.
4. **Late Materialization:** Join and filter operations can be pushed down before reconstructing rows.

Columnar Storage Platforms Several major analytical platforms use columnar storage:

- **Amazon Redshift:** Based on PostgreSQL with columnar storage optimizations
- **Snowflake:** Cloud-based data warehouse with columnar storage
- **Google BigQuery:** Serverless columnar data warehouse
- **Apache Parquet:** Columnar file format for data lakes
- **Apache ORC:** Optimized Row Columnar format for Hadoop

Writing SQL for Columnar Systems While the SQL syntax is generally the same, some considerations can help optimize for columnar storage:

```

-- Good practices for columnar databases

-- 1. Select only needed columns (takes advantage of columnar storage)
-- Efficient for columnar storage
SELECT
    date_key,
    product_key,
    SUM(sales_amount) as total_sales
FROM
    sales_fact
WHERE
    date_key BETWEEN 20230101 AND 20230131
GROUP BY
    date_key, product_key;

-- 2. Use partition pruning to limit data scanning

```

```

-- Example for a table partitioned by date_key
SELECT
    product_key,
    SUM(sales_amount) as total_sales
FROM
    sales_fact
WHERE
    date_key BETWEEN 20230101 AND 20230131
GROUP BY
    product_key;

-- 3. Leverage projection pushdown (predicate pushdown)
-- Filtering early reduces the amount of data processed
WITH filtered_sales AS (
    SELECT product_key, sales_amount
    FROM sales_fact
    WHERE date_key BETWEEN 20230101 AND 20230131
)
SELECT
    product_key,
    SUM(sales_amount) as total_sales
FROM
    filtered_sales
GROUP BY
    product_key;

```

2. Partitioning and Distribution Strategies

In distributed environments, how data is split across nodes has a significant impact on query performance.

Table Partitioning Partitioning divides a table into smaller, more manageable pieces based on a specified column:

```

-- Creating a partitioned table in PostgreSQL/Redshift
CREATE TABLE sales_fact (
    sale_id INTEGER,
    date_key INTEGER,
    product_key INTEGER,
    store_key INTEGER,
    customer_key INTEGER,
    sales_amount NUMERIC(12,2),
    -- other columns
    PRIMARY KEY (sale_id, date_key)
) PARTITION BY RANGE (date_key);

```

```

-- Create yearly partitions
CREATE TABLE sales_fact_2021
PARTITION OF sales_fact
FOR VALUES FROM (20210101) TO (20220101);

CREATE TABLE sales_fact_2022
PARTITION OF sales_fact
FOR VALUES FROM (20220101) TO (20230101);

CREATE TABLE sales_fact_2023
PARTITION OF sales_fact
FOR VALUES FROM (20230101) TO (20240101);

-- Snowflake syntax for clustering
CREATE OR REPLACE TABLE sales_fact (
    sale_id INTEGER,
    date_key INTEGER,
    product_key INTEGER,
    store_key INTEGER,
    customer_key INTEGER,
    sales_amount NUMERIC(12,2)
)
CLUSTER BY (date_key);

```

Distribution Styles Different platforms offer various distribution strategies for splitting data across compute nodes:

```

-- Amazon Redshift distribution styles
-- 1. KEY distribution (co-locate related data on the same node)
CREATE TABLE sales_fact (
    sale_id INTEGER,
    date_key INTEGER,
    product_key INTEGER,
    store_key INTEGER,
    customer_key INTEGER,
    sales_amount NUMERIC(12,2)
)
DISTKEY(product_key);

-- 2. EVEN distribution (round-robin distribution)
CREATE TABLE large_fact_table (
    -- columns
)
DISTSTYLE EVEN;

-- 3. ALL distribution (replicate entire table on all nodes)

```

```

CREATE TABLE small_dimension_table (
    -- columns
)
DISTSTYLE ALL;

-- Snowflake doesn't require manual distribution configuration
-- (it handles distribution automatically)

```

Optimizing Join Performance with Distribution Co-locating join keys on the same nodes reduces data movement during joins:

```

-- Amazon Redshift optimization for joins
CREATE TABLE sales_fact (
    sale_id INTEGER,
    date_key INTEGER,
    product_key INTEGER,
    -- other columns
)
DISTKEY(product_key);

CREATE TABLE product_dimension (
    product_key INTEGER,
    product_name VARCHAR(100),
    -- other columns
)
DISTKEY(product_key);

-- This join will be more efficient as both tables are distributed on product_key
SELECT
    p.product_name,
    SUM(s.sales_amount) as total_sales
FROM
    sales_fact s
JOIN
    product_dimension p ON s.product_key = p.product_key
GROUP BY
    p.product_name;

-- Distribution suggestions for fact and dimension tables
-- 1. Distribute fact tables on frequently joined foreign keys
-- 2. Distribute large dimension tables on their primary keys
-- 3. Use ALL distribution for small dimension tables
-- 4. Consider EVEN distribution for staging tables or tables without clear join patterns

```

Sort Keys and Zone Maps Sorting data physically can improve performance for range-based queries:

```

-- Amazon Redshift sort keys
CREATE TABLE sales_fact (
    sale_id INTEGER,
    date_key INTEGER,
    product_key INTEGER,
    -- other columns
)
DISTKEY(product_key)
SORTKEY(date_key); -- Compound sort key (default)

-- Multiple sort keys (compound)
CREATE TABLE sales_fact (
    sale_id INTEGER,
    date_key INTEGER,
    product_key INTEGER,
    -- other columns
)
SORTKEY(date_key, product_key);

-- Interleaved sort keys
CREATE TABLE sales_fact (
    sale_id INTEGER,
    date_key INTEGER,
    product_key INTEGER,
    -- other columns
)
INTERLEAVED SORTKEY(date_key, product_key);

-- Snowflake automatically creates zone maps and doesn't require explicit sort keys

```

3. Large-Scale Aggregation Techniques

Analytics at scale requires efficient aggregation strategies.

Memory-Efficient Aggregation When dealing with very large datasets, standard aggregations may exceed memory limits:

```

-- Two-stage aggregation to reduce memory requirements
-- Stage 1: Pre-aggregate at a granular level
WITH daily_product_sales AS (
    SELECT
        date_key,
        product_key,
        SUM(sales_amount) as total_sales,
        COUNT(*) as transaction_count
    FROM

```

```

        sales_fact
    WHERE
        date_key BETWEEN 20230101 AND 20230331
    GROUP BY
        date_key, product_key
)
-- Stage 2: Aggregate the results further
SELECT
    FLOOR(date_key / 100) as month_key,
    product_key,
    SUM(total_sales) as monthly_sales,
    SUM(transaction_count) as monthly_transactions
FROM
    daily_product_sales
GROUP BY
    FLOOR(date_key / 100), product_key;

```

Approximate Aggregations For very large datasets, approximate calculations can be much faster with acceptable accuracy:

```

-- Approximating distinct counts (PostgreSQL/Redshift)
SELECT
    product_category,
    APPROXIMATE COUNT(DISTINCT customer_key) as approx_unique_customers
FROM
    sales_fact
JOIN
    product_dimension USING (product_key)
GROUP BY
    product_category;

-- Approximate percentile using APPROX_PERCENTILE (BigQuery)
SELECT
    product_category,
    APPROX_PERCENTILE(sales_amount, 0.5) as median_sale,
    APPROX_PERCENTILE(sales_amount, 0.95) as p95_sale
FROM
    sales_fact
JOIN
    product_dimension USING (product_key)
GROUP BY
    product_category;

-- Approximate count distinct (Snowflake)
SELECT
    product_category,

```

```

        APPROX_COUNT_DISTINCT(customer_key) as approx_unique_customers
FROM
    sales_fact
JOIN
    product_dimension USING (product_key)
GROUP BY
    product_category;

```

Handling Skew in Distributed Aggregations Data skew can cause performance issues when certain nodes have more work than others:

```

-- Detecting skew in Redshift
SELECT
    "tbl",
    "col",
    "distkey",
    "skew_rows",
    "skew_rows_ratio"
FROM
    svv_table_info
WHERE
    "table" = 'sales_fact';

-- Handling skewed joins in Redshift (using temporary redistribution)
SET enable_result_cache_for_session TO OFF;
EXPLAIN
SELECT /*+ SHUFFLE */
    p.product_name,
    SUM(s.sales_amount) as total_sales
FROM
    sales_fact s
JOIN
    product_dimension p ON s.product_key = p.product_key
WHERE
    p.product_name = 'High Volume Product'
GROUP BY
    p.product_name;

```

Materialized Views and Result Caching Pre-calculate common aggregations for faster queries:

```

-- Create a materialized view for reporting in PostgreSQL/Redshift
CREATE MATERIALIZED VIEW monthly_category_sales AS
SELECT
    DATE_TRUNC('month', d.date_value) as month_date,
    p.category,

```



```

        SUM(s.sales_amount) as total_sales,
        COUNT(*) as transaction_count
FROM
    sales_fact s
JOIN
    date_dimension d ON s.date_key = d.date_key
JOIN
    product_dimension p ON s.product_key = p.product_key
GROUP BY
    DATE_TRUNC('month', d.date_value), p.category;

-- Create a caching layer in Snowflake
CREATE OR REPLACE TABLE monthly_category_sales AS
SELECT
    DATE_TRUNC('MONTH', d.date_value) as month_date,
    p.category,
    SUM(s.sales_amount) as total_sales,
    COUNT(*) as transaction_count
FROM
    sales_fact s
JOIN
    date_dimension d ON s.date_key = d.date_key
JOIN
    product_dimension p ON s.product_key = p.product_key
GROUP BY
    DATE_TRUNC('MONTH', d.date_value), p.category;

```

4. Incremental Processing

For large-scale analytics, processing only new or changed data is far more efficient than reprocessing everything.

Incremental Aggregation Pattern

```

-- Update an aggregation table incrementally

-- 1. Create the aggregation table
CREATE TABLE monthly_product_sales (
    month_key INTEGER,
    product_key INTEGER,
    total_sales NUMERIC(12,2),
    transaction_count INTEGER,
    last_updated TIMESTAMP,
    PRIMARY KEY (month_key, product_key)
);

```

```

-- 2. Track the last processed date
CREATE TABLE processing_metadata (
    table_name VARCHAR(100) PRIMARY KEY,
    last_processed_date_key INTEGER,
    last_processed_timestamp TIMESTAMP
);

-- 3. Incremental update procedure
-- Pseudocode for incremental update
/*
BEGIN TRANSACTION;

-- Get the last processed date
SELECT last_processed_date_key
INTO last_date
FROM processing_metadata
WHERE table_name = 'monthly_product_sales';

-- Process new data
INSERT INTO monthly_product_sales (month_key, product_key, total_sales, transaction_count, last_updated)
SELECT
    FLOOR(date_key / 100) as month_key,
    product_key,
    SUM(sales_amount) as total_sales,
    COUNT(*) as transaction_count,
    CURRENT_TIMESTAMP as last_updated
FROM
    sales_fact
WHERE
    date_key > last_date
GROUP BY
    FLOOR(date_key / 100), product_key
ON CONFLICT (month_key, product_key)
DO UPDATE SET
    total_sales = monthly_product_sales.total_sales + EXCLUDED.total_sales,
    transaction_count = monthly_product_sales.transaction_count + EXCLUDED.transaction_count,
    last_updated = CURRENT_TIMESTAMP;

-- Update the metadata
UPDATE processing_metadata
SET
    last_processed_date_key = (SELECT MAX(date_key) FROM sales_fact),
    last_processed_timestamp = CURRENT_TIMESTAMP
WHERE
    table_name = 'monthly_product_sales';

```

```
COMMIT;  
*/
```

Handling Late-Arriving Data In distributed systems, data can arrive out of order or be delayed:

```
-- Create a table with effective date ranges  
CREATE TABLE monthly_product_sales_v2 (  
    month_key INTEGER,  
    product_key INTEGER,  
    total_sales NUMERIC(12,2),  
    transaction_count INTEGER,  
    effective_from TIMESTAMP,  
    effective_to TIMESTAMP,  
    is_current BOOLEAN,  
    PRIMARY KEY (month_key, product_key, effective_from)  
);  
  
-- Handle late-arriving data with effective dating  
-- Pseudocode for late-arriving data handling  
/*  
BEGIN TRANSACTION;  
  
-- Mark existing records as not current  
UPDATE monthly_product_sales_v2  
SET  
    effective_to = CURRENT_TIMESTAMP,  
    is_current = FALSE  
WHERE  
    month_key IN (SELECT DISTINCT FLOOR(date_key / 100) FROM staging_sales)  
    AND is_current = TRUE;  
  
-- Insert new records with updated totals  
INSERT INTO monthly_product_sales_v2 (  
    month_key,  
    product_key,  
    total_sales,  
    transaction_count,  
    effective_from,  
    effective_to,  
    is_current  
)  
SELECT  
    FLOOR(date_key / 100) as month_key,  
    product_key,  
    SUM(sales_amount) as total_sales,
```

```

        COUNT(*) as transaction_count,
        CURRENT_TIMESTAMP as effective_from,
        NULL as effective_to,
        TRUE as is_current
FROM (
    -- Combine existing data with new data for affected months
    SELECT
        date_key,
        product_key,
        sales_amount
    FROM
        sales_fact
    WHERE
        FLOOR(date_key / 100) IN (SELECT DISTINCT FLOOR(date_key / 100) FROM staging_sales)

    UNION ALL

    SELECT
        date_key,
        product_key,
        sales_amount
    FROM
        staging_sales
) combined_data
GROUP BY
    FLOOR(date_key / 100), product_key;

COMMIT;
*/

```

SCD Type 2 Processing Slowly Changing Dimension Type 2 (SCD2) is a common pattern for tracking changes over time:

```

-- Create a Type 2 SCD table for product dimension
CREATE TABLE product_dimension_scd2 (
    product_key SERIAL,
    product_id INTEGER NOT NULL, -- business key
    product_name VARCHAR(100) NOT NULL,
    category VARCHAR(50) NOT NULL,
    price NUMERIC(12,2) NOT NULL,
    effective_from DATE NOT NULL,
    effective_to DATE,
    is_current BOOLEAN NOT NULL,
    PRIMARY KEY (product_key)
);

```

```

-- Create a unique index on business key + effective date
CREATE UNIQUE INDEX idx_product_id_effective
ON product_dimension_scd2 (product_id, effective_from);

-- Insert initial product data
INSERT INTO product_dimension_scd2 (
    product_id, product_name, category, price, effective_from, effective_to, is_current
)
SELECT
    product_id,
    product_name,
    category,
    price,
    '2023-01-01', -- effective date
    NULL, -- no end date for current records
    TRUE -- is current
FROM
    source_products;

-- SCD Type 2 incremental load procedure (pseudocode)
/*
BEGIN TRANSACTION;

-- Identify new products (not in dimension yet)
INSERT INTO product_dimension_scd2 (
    product_id, product_name, category, price, effective_from, effective_to, is_current
)
SELECT
    s.product_id,
    s.product_name,
    s.category,
    s.price,
    CURRENT_DATE, -- effective from today
    NULL, -- no end date
    TRUE -- is current
FROM
    staging_products s
LEFT JOIN
    product_dimension_scd2 d
    ON s.product_id = d.product_id
    AND d.is_current = TRUE
WHERE
    d.product_id IS NULL;

-- Identify changed products
WITH changed_products AS (

```

```

SELECT
    s.product_id,
    s.product_name,
    s.category,
    s.price
FROM
    staging_products s
JOIN
    product_dimension_scd2 d
    ON s.product_id = d.product_id
    AND d.is_current = TRUE
WHERE
    s.product_name != d.product_name OR
    s.category != d.category OR
    s.price != d.price
)
-- Update current records to be not current anymore
UPDATE product_dimension_scd2 d
SET
    effective_to = CURRENT_DATE - INTERVAL '1 day',
    is_current = FALSE
FROM
    changed_products c
WHERE
    d.product_id = c.product_id
    AND d.is_current = TRUE;

-- Insert new records for changed products
INSERT INTO product_dimension_scd2 (
    product_id, product_name, category, price, effective_from, effective_to, is_current
)
SELECT
    product_id,
    product_name,
    category,
    price,
    CURRENT_DATE, -- effective from today
    NULL, -- no end date
    TRUE -- is current
FROM
    changed_products;

COMMIT;
*/

```

Merge Patterns for Upserts In data pipelines, you often need to insert new records and update existing ones in a single operation:

```
-- PostgreSQL's MERGE equivalent using ON CONFLICT (upsert)
INSERT INTO monthly_sales_summary (
    month_key,
    product_key,
    total_sales,
    transaction_count,
    last_updated
)
SELECT
    FLOOR(date_key / 100) as month_key,
    product_key,
    SUM(sales_amount) as total_sales,
    COUNT(*) as transaction_count,
    CURRENT_TIMESTAMP as last_updated
FROM
    new_sales_data
GROUP BY
    FLOOR(date_key / 100), product_key
ON CONFLICT (month_key, product_key)
DO UPDATE SET
    total_sales = monthly_sales_summary.total_sales + EXCLUDED.total_sales,
    transaction_count = monthly_sales_summary.transaction_count + EXCLUDED.transaction_count,
    last_updated = CURRENT_TIMESTAMP;

-- Snowflake MERGE statement
MERGE INTO monthly_sales_summary tgt
USING (
    SELECT
        FLOOR(date_key / 100) as month_key,
        product_key,
        SUM(sales_amount) as total_sales,
        COUNT(*) as transaction_count
    FROM
        new_sales_data
    GROUP BY
        FLOOR(date_key / 100), product_key
) src
ON tgt.month_key = src.month_key AND tgt.product_key = src.product_key
WHEN MATCHED THEN UPDATE SET
    tgt.total_sales = tgt.total_sales + src.total_sales,
    tgt.transaction_count = tgt.transaction_count + src.transaction_count,
    tgt.last_updated = CURRENT_TIMESTAMP()
WHEN NOT MATCHED THEN INSERT (
```

```

        month_key,
        product_key,
        total_sales,
        transaction_count,
        last_updated
    )
VALUES (
    src.month_key,
    src.product_key,
    src.total_sales,
    src.transaction_count,
    CURRENT_TIMESTAMP()
);

```

5. Modern Data Formats and Integration

Modern analytics platforms often integrate with various data storage formats and systems.

Working with Parquet and ORC Files Many data lake solutions use columnar file formats like Parquet and ORC:

```

-- Redshift Spectrum query on S3 Parquet files
SELECT
    year,
    month,
    product_category,
    SUM(sales_amount) as total_sales
FROM
    spectrum.sales_parquet
WHERE
    year = 2023
    AND month BETWEEN 1 AND 3
GROUP BY
    year, month, product_category
ORDER BY
    month, total_sales DESC;

-- Creating an external table in Redshift
CREATE EXTERNAL TABLE spectrum.sales_parquet (
    sale_id INTEGER,
    date_key INTEGER,
    product_key INTEGER,
    customer_key INTEGER,
    sales_amount DECIMAL(12,2),
    year INTEGER,

```



```

        month INTEGER
    )
    PARTITIONED BY (year INT, month INT)
    STORED AS PARQUET
    LOCATION 's3://analytics-bucket/sales/';

-- Adding partitions
ALTER TABLE spectrum.sales_parquet ADD PARTITION(year=2023, month=1)
LOCATION 's3://analytics-bucket/sales/year=2023/month=1/';

-- BigQuery external table on GCS Parquet files
CREATE EXTERNAL TABLE `project.dataset.sales_parquet`
OPTIONS (
    format = 'PARQUET',
    uris = ['gs://analytics-bucket/sales/*.parquet']
);

-- Snowflake external table
CREATE EXTERNAL TABLE sales_external (
    sale_id INTEGER,
    date_key INTEGER,
    product_key INTEGER,
    customer_key INTEGER,
    sales_amount DECIMAL(12,2)
)
STORAGE_INTEGRATION = s3_integration
PARTITION BY (TO_VARCHAR(date_key))
FILE_FORMAT = (TYPE = PARQUET)
LOCATION = 's3://analytics-bucket/sales/';

```

Query Federation Modern analytics platforms can query data from various sources in a single query:

```

-- PostgreSQL foreign table (using postgres_fdw)
CREATE EXTENSION postgres_fdw;

CREATE SERVER remote_server
FOREIGN DATA WRAPPER postgres_fdw
OPTIONS (host 'remote-db-server', port '5432', dbname 'products_db');

CREATE USER MAPPING FOR current_user
SERVER remote_server
OPTIONS (user 'remote_user', password 'remote_password');

CREATE FOREIGN TABLE remote_products (
    product_id INTEGER,

```

```

        product_name VARCHAR(100),
        category VARCHAR(50),
        price NUMERIC(12,2)
    )
SERVER remote_server
OPTIONS (schema_name 'public', table_name 'products');

-- Query combining local and remote data
SELECT
    rp.product_name,
    SUM(sf.sales_amount) as total_sales
FROM
    sales_fact sf
JOIN
    remote_products rp ON sf.product_key = rp.product_id
WHERE
    sf.date_key BETWEEN 20230101 AND 20230331
GROUP BY
    rp.product_name;

-- Amazon Redshift federated query
-- Query Aurora PostgreSQL and Redshift in a single query
SELECT
    rp.product_name,
    SUM(sf.sales_amount) as total_sales
FROM
    sales_fact sf
JOIN
    mydb.public.products rp ON sf.product_key = rp.product_id
WHERE
    sf.date_key BETWEEN 20230101 AND 20230331
GROUP BY
    rp.product_name;

```

Working with Semi-Structured Data Modern analytics often involves JSON, XML, or other semi-structured data:

```

-- Querying JSON data in PostgreSQL
CREATE TABLE customer_activities (
    customer_id INTEGER,
    activity_date DATE,
    activity_data JSONB
);

-- Query JSON fields
SELECT

```

```

        customer_id,
        activity_data->>'activity_type' as activity_type,
        (activity_data->>'amount')::NUMERIC as amount
FROM
    customer_activities
WHERE
    activity_data->>'activity_type' = 'purchase'
    AND activity_date >= '2023-01-01';

-- Aggregate JSON array elements
SELECT
    customer_id,
    jsonb_array_elements(activity_data->'products')->>'product_id' as product_id,
    jsonb_array_elements(activity_data->'products')->>'price' as price
FROM
    customer_activities
WHERE
    activity_data->>'activity_type' = 'purchase'
    AND activity_date >= '2023-01-01';

-- Snowflake semi-structured data
CREATE TABLE customer_activities (
    customer_id INTEGER,
    activity_date DATE,
    activity_data VARIANT
);

-- Query variant data
SELECT
    customer_id,
    activity_data:activity_type::STRING as activity_type,
    activity_data:amount::NUMERIC as amount
FROM
    customer_activities
WHERE
    activity_data:activity_type::STRING = 'purchase'
    AND activity_date >= '2023-01-01';

```

6. Practical Optimization Techniques for Distributed Analytics

Predicate Pushdown Pushing filters to the data source level reduces the amount of data that needs to be processed:

```

-- Example query without explicit pushdown
SELECT
    cd.customer_segment,

```

```

        SUM(sf.sales_amount) as total_sales
FROM
    sales_fact sf
JOIN
    customer_dimension cd ON sf.customer_key = cd.customer_key
WHERE
    sf.date_key BETWEEN 20230101 AND 20230131
    AND cd.customer_segment = 'Enterprise'
GROUP BY
    cd.customer_segment;

-- With explicit pushdown using CTEs
WITH filtered_sales AS (
    SELECT
        customer_key,
        sales_amount
    FROM
        sales_fact
    WHERE
        date_key BETWEEN 20230101 AND 20230131
),
filtered_customers AS (
    SELECT
        customer_key,
        customer_segment
    FROM
        customer_dimension
    WHERE
        customer_segment = 'Enterprise'
)
SELECT
    fc.customer_segment,
    SUM(fs.sales_amount) as total_sales
FROM
    filtered_sales fs
JOIN
    filtered_customers fc ON fs.customer_key = fc.customer_key
GROUP BY
    fc.customer_segment;

```

Query Splitting for Complex Analytics Break complex queries into manageable parts:

```

-- Complex query with multiple joins and aggregations
-- Inefficient approach:
SELECT

```

```

        d.year,
        d.quarter,
        p.category,
        c.customer_segment,
        SUM(s.sales_amount) as total_sales,
        COUNT(DISTINCT s.customer_key) as unique_customers,
        SUM(s.sales_amount) / COUNT(DISTINCT s.customer_key) as sales_per_customer,
        SUM(CASE WHEN s.sales_amount > 1000 THEN 1 ELSE 0 END) as high_value_transactions
FROM
    sales_fact s
JOIN
    date_dimension d ON s.date_key = d.date_key
JOIN
    product_dimension p ON s.product_key = p.product_key
JOIN
    customer_dimension c ON s.customer_key = c.customer_key
WHERE
    d.year = 2023
GROUP BY
    d.year, d.quarter, p.category, c.customer_segment;

-- More efficient: Split into stages
-- Stage 1: Create base aggregation
CREATE TEMPORARY TABLE stage1_aggregation AS
SELECT
    date_key,
    product_key,
    customer_key,
    SUM(sales_amount) as total_sales,
    COUNT(*) as transaction_count,
    SUM(CASE WHEN sales_amount > 1000 THEN 1 ELSE 0 END) as high_value_count
FROM
    sales_fact
WHERE
    date_key BETWEEN 20230101 AND 20231231
GROUP BY
    date_key, product_key, customer_key;

-- Stage 2: Join dimensions and create final result
SELECT
    d.year,
    d.quarter,
    p.category,
    c.customer_segment,
    SUM(s.total_sales) as total_sales,
    COUNT(DISTINCT s.customer_key) as unique_customers,

```

```

        SUM(s.total_sales) / COUNT(DISTINCT s.customer_key) as sales_per_customer,
        SUM(s.high_value_count) as high_value_transactions
FROM
    stage1_aggregation s
JOIN
    date_dimension d ON s.date_key = d.date_key
JOIN
    product_dimension p ON s.product_key = p.product_key
JOIN
    customer_dimension c ON s.customer_key = c.customer_key
WHERE
    d.year = 2023
GROUP BY
    d.year, d.quarter, p.category, c.customer_segment;

```

Optimizing Data Loading For analytics workloads, bulk loading is more efficient than row-by-row inserts:

```

-- Example of efficient bulk loading in PostgreSQL/Redshift
COPY sales_fact
FROM 's3://mybucket/sales/data.csv'
IAM_ROLE 'arn:aws:iam::0123456789012:role/MyRedshiftRole'
DELIMITER ','
REGION 'us-west-2';

-- Snowflake bulk loading
COPY INTO sales_fact
FROM @my_s3_stage/sales/data.csv
FILE_FORMAT = (TYPE = 'CSV', SKIP_HEADER = 1);

-- Load directly into partitioned tables
COPY INTO sales_fact_2023
FROM @my_s3_stage/sales/2023/
FILE_FORMAT = (TYPE = 'CSV', SKIP_HEADER = 1);

```

Multi-Statement Transactions for Complex Data Pipelines For complex analytics workflows, using transactions ensures atomicity:

```

-- Complex ETL transaction example
BEGIN TRANSACTION;

-- 1. Create a temporary staging table
CREATE TEMPORARY TABLE temp_monthly_sales AS
SELECT
    DATE_TRUNC('month', d.date_value) as month_date,
    p.category,

```

```

        SUM(s.sales_amount) as total_sales,
        COUNT(*) as transaction_count
FROM
    sales_fact s
JOIN
    date_dimension d ON s.date_key = d.date_key
JOIN
    product_dimension p ON s.product_key = p.product_key
WHERE
    s.sales_amount > 0
    AND d.date_value >= '2023-01-01'
    AND d.date_value < '2024-01-01'
GROUP BY
    DATE_TRUNC('month', d.date_value), p.category;

-- 2. Delete existing data that will be replaced
DELETE FROM monthly_sales_report
WHERE month_date >= '2023-01-01' AND month_date < '2024-01-01';

-- 3. Insert new data
INSERT INTO monthly_sales_report (
    month_date,
    category,
    total_sales,
    transaction_count,
    sales_per_transaction,
    last_updated
)
SELECT
    month_date,
    category,
    total_sales,
    transaction_count,
    CASE
        WHEN transaction_count > 0 THEN total_sales / transaction_count
        ELSE 0
    END as sales_per_transaction,
    CURRENT_TIMESTAMP as last_updated
FROM
    temp_monthly_sales;

-- 4. Update the data processing log
INSERT INTO etl_process_log (
    process_name,
    start_date,
    end_date,

```

```

        records_processed,
        status,
        comments
    )
VALUES (
    'monthly_sales_refresh',
    '2023-01-01',
    '2023-12-31',
    (SELECT COUNT(*) FROM temp_monthly_sales),
    'SUCCESS',
    'Monthly sales refresh completed successfully'
);

COMMIT;

```

By understanding and applying these distributed SQL principles and techniques, you can build highly scalable and efficient analytics solutions across a variety of modern data platforms.